

Where to Stop: Tile-Adaptive Early Exit in a Learned Image Decoder

Anonymous CVPR submission · Paper ID ****

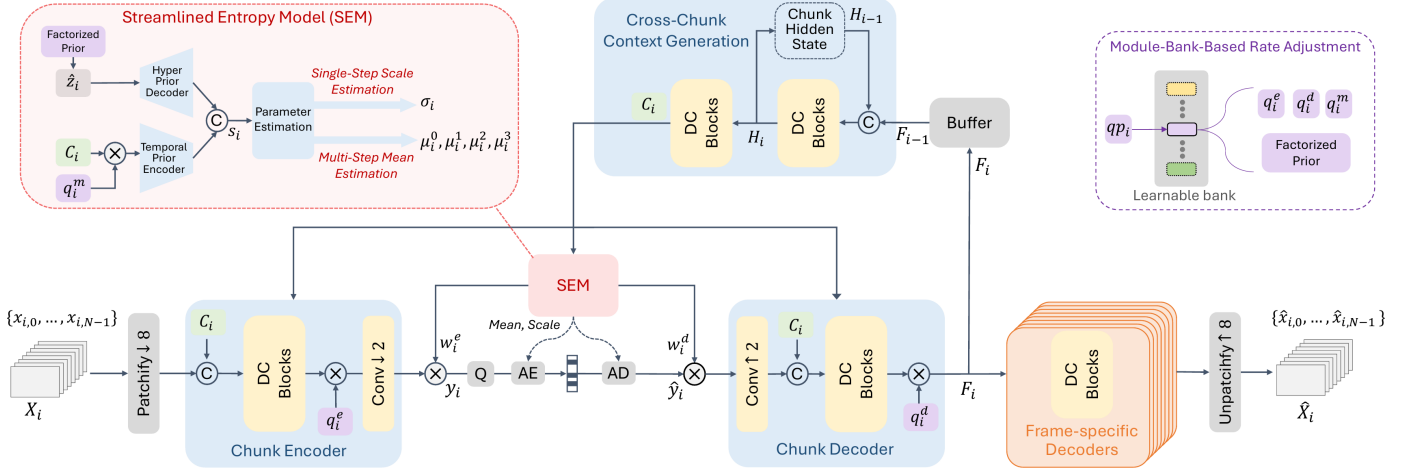


Figure 1. The decoder we modify. Figure 3 of DCVC-UF [14], reproduced. Everything up to the reconstruction stays frozen in this work: the patch embedding, the chunk encoder, the entropy model and the coded payload. FLEX-UF replaces the frame-specific decoders on the right with a ladder of exits taken per tile.

Abstract

Learned image decoders spend the same computation on every region of a frame, whatever that region contains. We show that decoding compute can instead be allocated spatially, by content, with the encoder, the entropy model and the transmitted representation left untouched. We add a multi-exit ladder to the intra decoder of DCVC-UF, so that tiles of one frame leave a shared reconstruction trunk at different depths. On 53 CTC intra frames, one from each sequence, under a 0.1 dB quality constraint this removes 29.2% to 15.2% of decoder multiply-accumulates across the rate range, 21.5% on average, at a BD-Rate cost of 1.03%. At 0.2 dB the mean saving is 33.7% and at 0.3 dB it is 37.4%, within 0.9 points of the ladder’s 38.3% architectural ceiling.

Adaptive decoding works only inside a finite window. Tiling sets a rate-dependent quality floor below which no allocation is feasible, and a saturation point above which every tile already sits on the cheapest path. Rescaling the budget between those two limits collapses five rate curves that differ by 15.5 points onto one within 1.7. The dominant cost of adapting is set by per-tile decoding depth rather than by the spatial extent of boundary contamination. Finally, the bits the entropy model has already spent on a tile predict how deep that tile must decode better than a trained 144 K-parameter router does, by up to 3.4 points, with no parameters and nothing added to the file. A compressed representation carries not only what to reconstruct, but how much computation the reconstruction needs.

1. Introduction

Learned codecs are compared on rate and distortion, with complexity given as a single number, so many GMAC per frame or so many milliseconds. That number is a constant. A learned decoder runs the same graph on a page of text as on a cloudless sky, although the sky is much the easier picture.

Classification networks abandoned this a decade ago. Early-exit architectures attach classifiers at intermediate depths and stop as soon as the prediction is confident [3, 15, 20], and the literature around them is by now mature. Super-resolution adopted the same idea spatially. ClassSR [12] routes image patches to networks of different capacity by difficulty, and APE [1] exits patches at different depths of one network.

Learned compression has taken a different route. Slimmable autoencoders [22, 23] give one model several complexity levels, but the level is chosen *per stream* rather than per region, and switching it changes the bitstream.

We ask the spatial-adaptivity question inside a learned decoder, under a constraint that makes the answer deployable: **the encoder is frozen and the coded payload is unchanged.** Whatever we do has to consume the bitstream the released encoder already produces. That rules out re-training the analysis transform, changing the entropy model, or altering the latent. It leaves one place to spend adaptivity, the synthesis transform.

Our method, FLEX-UF, attaches exits at intervals through the twelve residual blocks of the DCVC-UF intra decoder; we call that ordered set of exits the *ladder*. The first j blocks run over the whole frame. The rest run per tile, over a set of tiles that shrinks with depth, and the shrinkage is where the saving comes from. Each exit hands its feature to the shared head through a small pointwise adapter, zero-initialised so that at step zero the deepest exit reproduces the released decoder bit-exactly.

Getting this to work depended less on the ladder than on two problems that have little to do with early exit as it is usually studied.

Tiling has a large price.

Spatial adaptivity needs tiles, and once a frame has been cut into tiles, every 3×3 convolution at a tile border reads invented values. The tile-boundary artefact that follows is what we call the *seam*, and under the default padding rule it costs 0.677 dB on this decoder at high rate. That is several times the entire budget the method works to, and it is paid before any tile has saved a single operation. In Section 4 we treat padding as an *estimator* of the unseen neighbour and measure four of them. The ordering we get is not the obvious one.

A distortion budget only works inside a band.

Tiling costs something even when nothing exits early, so there is a *floor*, and budgets below it admit no allocation at all. The ladder also has a shallowest usable exit, so there is a *saturation* point above which every tile already takes that exit and more quality buys nothing. Between the two, in what we call the *band*, the budget genuinely trades quality for

compute. We measure both ends at every rate in Section 5.4, where the 0.1 dB budget uses 45% of the band at low rate and 9% at high rate.

Contributions.

(i) An early-exit ladder for a learned image decoder that picks a depth per tile, which is spatial adaptivity at the granularity of a tile. The encoder, the entropy model and the coded latent are untouched in both modes we report. *Signalled* FLEX-UF leaves the latent unchanged and adds 89 bits per 1080p frame of routing metadata, or 0.021% of a typical bitrate, and saves 29.2% to 15.2% of decoder MACs for 0.1 dB on the common test set at a BD-Rate cost of 1.03%. *Bitstream-identical* FLEX-UF adds nothing at all, decodes a byte-identical file, and reaches 23.6% to 10.7%.

(ii) The band a distortion budget works in: a floor below which no allocation is feasible, a saturation point above which none improves, both in closed form, and seven propositions verified numerically rather than asserted. Measuring the budget in units of that band accounts for most of the rate dependence, on two independently trained checkpoints.

(iii) A zero-learned-parameter control that adaptive-inference work is rarely measured against. Routing on the bits the entropy model has already spent per tile costs nothing, adds nothing to the stream, and beats our trained head at every rate, while agreeing with the oracle on fewer tiles than the head does.

2. Related work

Early exit.

BranchyNet [20] and MSDNet [3] established the pattern of intermediate classifiers with a confidence rule; SDN [15] framed it as mitigating overthinking. We train all exits jointly, after Scardapane et al. [18], and distil between exits as in Phuong and Lampert [17]. The caveat in [19] is that too large a student-teacher gap hurts the shallowest exits, so our distillation runs between *adjacent* exits. All of this work exits on a confidence signal computed from the network's own output. A decoder has no such signal. There is no class posterior, and the quantity that would decide is the error against a source the decoder cannot see (Section 3.5).

Spatially adaptive inference.

ClassSR [12] sorts super-resolution patches into easy/medium/hard and runs a different network on each; APE [1] exits patches at different depths; Glance-and-Focus [8] spends resolution adaptively. We share the per-region premise with all three, and we inherit the same tiling problem, though the cost of tile borders is rarely quantified in that line of work. To our knowledge the estimator view of border padding and its measured ordering (Section 4) is new. The closest prior work is the per-channel AR(1) padding of Kaseva et al. [11], which we implement and evaluate.

Method	Varies	Where	Latent	Side	Dec.
				data	only
SlimCAE [22]	width	per stream	new	—	no
Slimmable video [23]	width	per stream	new	—	no
EVC [22]	mask / pruning	per model	new	—	no
Spatial competition [27]	which codec	per region	new	yes	no
DCVC-RT [33]	architecture	fixed	new	—	no
FLEX-UF signalled	decoder depth	per region	same	89 b	no
FLEX-UF bitstream-identical	decoder depth	per region	same	none	yes

Table 1. Where this sits. What each method varies, at what granularity, and what that costs the stream. A single new-bitstream column cannot represent our own two modes, so the last three ask what actually changes: whether the coded latent is the same, what side data travels with it, and whether the decoder alone can do it.

Method	kMAC/px	Par. (M)	BD-Rate (%)
CHARM [51]	496	58.5	+0.86
STF [45]	511	99.9	-2.06
ELIC [44]	574	36.9	-3.22
WeConvene [50]	2343	107.2	-6.98
MambaVC [49]	814	47.9	-8.72
DCVC-DC intra [52]	542	45.5	-9.18
TCM [46]	1824	76.6	-10.70
FLIC [48]	1096	71.0	-13.20
MLIC++ [47]	1283	116.7	-15.15
HPCM-Base [43]	919	68.5	-15.31
HPCM-Large [43]	1261	89.7	-19.19

Table 2. What a learned image decoder costs. Eleven codecs as published, measured on one machine by Li et al. [43]: BD-Rate against VTM-22.0 on Kodak, arithmetic per pixel and parameters. The spread in cost is an order of magnitude, and every entry pays its cost on every pixel of every image.

Method	GMAC	Par. (M)	BD-Rate (%)	Adaptive
DCVC-DC [52]	—	18.3	+14.5	no
DCVC-FM [13]	2642	18.3	-21.3	no
DCVC-RT [33]	385	20.7	-21.0	no
DCVC-UF (LD) [14]	170	9.7	-9.5	no
DCVC-UF (HT-S) [14]	211	81.2	-31.6	no
DCVC-UF (HT-L) [14]	343	120.5	-42.2	no
FLEX-UF intra decoder	219 → 172	—	+1.03	yes

Table 3. The family this work modifies, from DCVC-UF [14]: MACs per 1080p frame, parameters, and BD-Rate against VTM-17.0 low delay. Comparable within this table and not against Table 2, which uses a different anchor and a different test set. The last column is the one this paper is about: whether the decoder spends more computation where the picture needs it. Our row is the intra decoder of the last DCVC-UF entry, at the 0.1 dB budget.

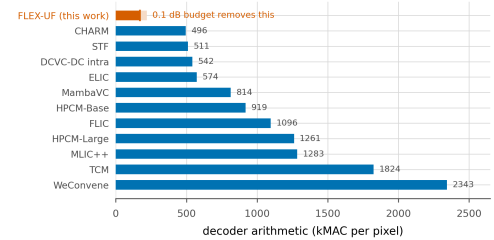


Figure 2. Every published decoder is one number. Arithmetic per pixel for the eleven image codecs of Table 2, and for the intra decoder this work modifies. Costs are comparable here even though the BD-Rate anchors are not, which is why this axis and not a rate-distortion one. The bar at the top is the same decoder as the rest of the paper: a 0.1 dB budget removes the pale section, and how much it removes depends on the picture.

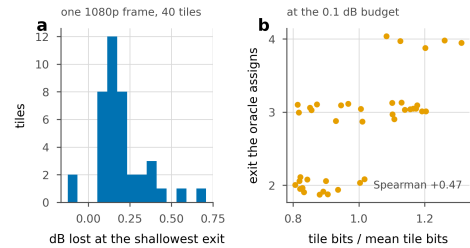


Figure 3. Why depth should not be uniform. One 1080p frame, 40 tiles. **a**, what each tile loses if it stops at the shallowest exit the ladder allows: a few lose nothing measurable and one loses 0.71 dB. A single depth for the frame has to be set by the worst of them. **b**, the bits the entropy model has already spent on a tile against the exit the Lagrangian oracle sends it to, at the 0.1 dB budget. The signal the decoder needs is already in the file.

Together the two tables say where the field spends its decoder budget. Complexity has moved a long way in both directions: TCM [46] and WeConvene [50] buy rate with an order of magnitude more arithmetic per pixel than CHARM [51] or STF [45], and DCVC-UF [14] moves the

other way, cutting a 1080p frame from DCVC-FM's 2642 GMAC to 170. Two things are constant down both tables. The cost is a property of the model, chosen once and paid on every image; and it is uniform over the frame, so a flat sky and a face are decoded at the same price. Those are the two the ladder in this paper changes, and it changes them without touching the model that was shipped.

Complexity control in learned compression.

SlimCAE [22] and slimmable video codecs [23] expose several widths of one model. The choice is per stream and it changes the encoder, so the bitstream is not interchangeable. DCVC-FM [13] and DCVC-UF [14] reduce cost architecturally, for every frame equally. Rate–distortion–complexity has since become an explicit third axis. Gao et al. [35] tune spatial context usage to trade decode cost against rate, Ho et al. [36] survey where conditional residual coding sits on that surface, and Zhang and Gao [37] route *whole frames* to one of several jointly-trained coding paths, spending less computation on cheaper frames. Every one of these changes the model, and with it what the encoder emits. Our axis is orthogonal. The model and the bitstream are both fixed, and only *how much of the decoder runs where* varies.

The closest neighbour.

Blard et al. [27] also partition an image into regions, also choose per region by a rate–distortion cost computed at the encoder, and also transmit a mode map. The differences are in what the regions choose among and in what the map buys. Their regions choose among several *separately trained* codecs, so the decoder must hold all of them and its complexity is that of one codec regardless of the choice; the map buys rate, not compute. Ours choose a prefix length of a single trunk, so the weights are shared by construction and the map buys compute at fixed rate. Because their alternatives are unrelated networks, no decoder-side predictor could stand in for the encoder's search. Our exits are prefixes of one another, and that nesting is what lets a decoder-side predictor stand in at all (Section 3.1).

Signalling versus prediction.

Being *told* a mode decision rather than inferring it is the norm in standardised video coding. HEVC [9] and VVC [21] transmit partitioning, prediction mode and transform tree. We evaluate both, and treat the signalled variant as the conventional design (Section 3.1).

Deferring to an oracle under a budget.

Letting a predictor decide most cases and handing a small budget of the hardest ones to something exact is the shape of selective prediction [38] and learning to defer [39, 40], and of the budgeted variant of the latter [41]. We build on that shape in Section 5.7. Two things differ, and both make our case easier. The expert here is the encoder's own search, so it is exact and always available at no test-time cost. What is scarce is the *bits* needed to say what it decided. The selection rule is not learned either. Because the objective is separable over tiles, the optimal set of size s at a fixed multiplier is exactly the s largest regrets, which the encoder can compute. Who to defer and how to learn it is the open question in that literature, and it has a closed form here. What remains is the question we measure, which is how concentrated the regret is.

Allocating a budget over units.

The construction we use is not new and we do not present it as such. Shoham and Gersho [28] showed that for a finite set of per-unit operating points, a Lagrangian sweep decouples the allocation across units and traces exactly the lower convex hull of the achievable set; Ortega and Ramchandran [29] made it standard practice in image and video coding. What we add is the structure this particular operating set has: a floor below which no allocation is feasible, a saturation point above which none improves, and a measurement of what the convex-hull restriction costs (Section 5.4). The same relaxation has resurfaced for test-time compute in language models [30], with per-instance decoupling

and a binary search on the multiplier. That is the identical structure in a domain with no rate axis, and we read it as evidence that the floor/saturation characterisation is worth stating generally.

How much is there to gain?

Bounding what adaptive inference could achieve is itself a line of work. Hasan et al. [34] derive an oracle bound on efficiency at fixed accuracy, given per-model resource and accuracy, and report 43–121× on ImageNet and 7–81× on HellaSwag. Their bound has a ceiling and no floor, because the largest model in their family reaches the reference accuracy by definition. Spatial adaptivity introduces a floor. Cutting a frame into tiles costs quality even when every tile runs to full depth, so the reference is unreachable at *any* compute and the feasible set of budgets is an interval rather than a ray. Section 5.4 characterises that interval and both of its ends.

Tile boundaries.

Every method that processes an image in independently-computed tiles meets the same artefact. The remedies in the literature are overlap and averaging, local padding from neighbouring patches [31], training with overlaps [32], and fitted extrapolation [11]. Local padding is the closest to the exact remedy we describe in Section 4.4, and the difference is accounting. It pads every convolutional layer and does not report the cost. We pad only the 0.29% of each block that has any spatial extent, which is what makes the exact fix cost 0.032% of the decode rather than a multiplier on all of it. To our knowledge the estimator view of the padding rule, the measured ordering of four estimators, and the dependence of the penalty on per-tile depth (Section 4) have not been reported.

Where the time goes.

DCVC-RT [33] argues that operational rather than computational complexity is the speed bottleneck for neural codecs, and cites channel reductions that yield linear rather than quadratic speedups. Section 5.9 is an instance of that claim inside one loop. Removing 35.3% of the operations buys 29.1% of the time, and 1.2 points of the difference come back from reordering the loop with the arithmetic untouched.

3. Method

3.1 Setting and notation

We set out here the decoder we work on, the notation the rest of the paper uses, and the three configurations we compare.

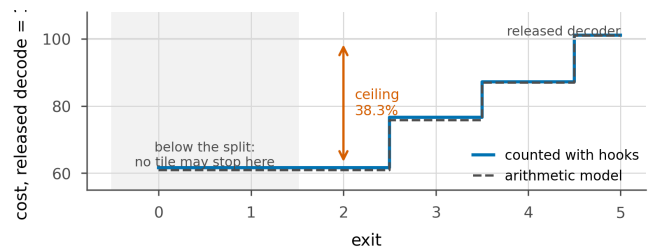


Figure 4. The ladder, priced. What a tile costs at each exit, as a percentage of one released decode, counted with hooks on the executed pass and beside the arithmetic model the argmin uses. Exits below the split depth are not distinct, because the first j groups run for every tile whatever it does. The deepest exit costs slightly more than the release, which is the full-frame deblocking pass. Reported savings in this paper are the hook count.

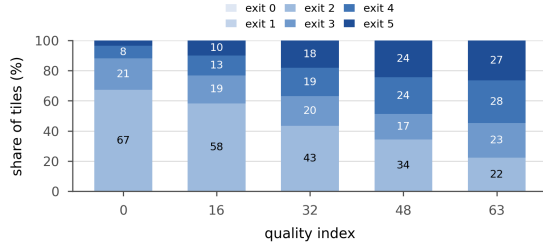


Figure 5. Where the tiles actually go, at the 0.1 dB budget, pooled over the whole test set. At the lowest rate two thirds of tiles take the cheapest available exit; at the highest only a fifth do and a quarter need the deepest. The allocation is not degenerate at either end, which is what makes the choice worth making.

The decoder.

We work on the intra decoder of DCVC-UF [14]. The analysis side stays frozen throughout: the patch embedding, the chunk encoder, the entropy model and the coded payload are never touched (Figure 1). What we replace is the synthesis trunk that turns the decoded latent into a picture. We call the unmodified public decoder the *released decoder*, and one full-frame run of it is the unit of both cost and quality below.

Tiles and exits.

A frame is cut into square tiles, 256 px on a side unless we say otherwise, and each tile is carried through part of the trunk on its own. Tiles are indexed by t and there are N of them, $N=40$ at 1080p. The trunk itself is a stack of twelve blocks; we cut it into K groups and attach an exit to each, so exits are indexed by k and exit $K-1$ is the whole trunk. The first j groups run once over the whole frame and the remaining $K-j$ run per tile, so j is the *split depth*. A tile that leaves at exit k runs groups j to k and skips everything after it. Writing c_k for what a tile costs at exit k , a frame that assigns exit k_t to tile t costs the mean of c over its tiles. Tiles are cut on the trunk's feature grid, which is coarser than the pixel grid, but we quote tile sizes in pixels.

Distortion and the budget.

$D(t,k)$ is the error tile t incurs if it leaves at exit k . It is measured on the path we deploy at inference, a tiled decode, against the released decoder's full-frame decode of the same latent, so what tiling costs is inside it; we call that path the *deployed path* and the table built on it the *deployed table*. Tapping the exits of one full-frame forward pass gives a cheaper *full-frame table* that is not the same quantity, and Section 5 measures the difference. Every result is quoted at a *distortion budget*, the decibels a decoded frame is allowed to fall below the released decoder on that same latent, and the headline budget is 0.1 dB. To allocate is to choose an exit for every tile so that the frame lands on its budget as cheaply as possible. A budget only buys something inside its band, because tiling costs quality before any tile exits early and the shallowest usable exit bounds what can be saved; Section 5.4 measures both ends. Rate is set by a quality index running from q_0 , the lowest rate, to q_63 , the highest, and we report five of them: q_0 , q_{16} , q_{32} , q_{48} and q_{63} .

symbol	meaning
t, N	tile index and tiles per frame; $N=40$ at 1080p
k, K	exit index and number of exits, k in $0..K-1$; $K=6$
j	split depth: groups $0..j-1$ run full frame, $j..K-1$ per tile; $j=2$
c_k	cost of a tile at exit k , as a fraction of one released full-frame decode
$D(t,k)$	error of tile t at exit k , tiled decode against the released full-frame decode of the same latent
λ	multiplier trading c_k against $D(t,k)$, bisected to the budget
β	the same trade applied to the router's logits
q	quality index, q_0 the lowest rate and q_{63} the highest

Three configurations.

All three choose the same object, one exit per tile, on the same weights and the same coded payload. Configurations **A** and **B** differ only in

where that choice is made, and **C** sits between them.

A decides at the encoder. The encoder holds the source, so it knows $D(t,k)$, picks each tile's exit by the Lagrangian of Section 3.5, and transmits the resulting *exit map*, one exit index per tile, which is the analogue of the mode map a standard codec signals. It entropy-codes to ~ 89 bits per 1080p frame, or 0.021% of a typical bitrate. It costs the encoder about one extra decode per frame and the decoder nothing.

B decides at the decoder. A 144 K-parameter head reads decoded data only and predicts the same map (Section 3.5). Nothing is transmitted and the file stays byte-identical to the released one. It costs the decoder 0.163% of its own multiply-accumulates, which is charged inside every **B** number we report. Because that head reads nothing the encoder cannot also read, the encoder can run it too, and so knows tile by tile where it will be wrong.

Two of these are deployment modes and we name them so that no claim in this paper is ambiguous about what is sent. *Signalled* FLEX-UF is configuration **A**: the coded latent is unchanged and a small exit map travels beside it. *Bitstream-identical* FLEX-UF is configuration **B**: nothing is added, and the decoder reads a file that is byte for byte the one the released encoder produced. Every saving quoted in this paper is labelled with the mode it was measured in.

C interpolates between the two. The encoder signals a fraction p of the tiles, the ones where leaving **B** alone is most costly, and **B** decides the rest, so $p=0$ is **B** and $p=1$ is **A**. Any decoder-side predictor can take **B**'s place there, and Section 5.7, which measures **C**, tries a second one. **C** transmits a mask rather than a full map, costs the encoder **A**'s search plus one run of the predictor, and costs the decoder whatever that predictor costs.

3.2 Where the computation is

The DCVC-UF intra decoder is one upsampling block, twelve DepthConvBlocks and a head, costing 453.5 GMAC per 1080p frame. The twelve blocks are 89.4% of that, which is why we build the ladder across them. Inside one block at $C=384$ channels, the only operator with any spatial extent is a 3×3 depthwise convolution, and it costs 9C against the block's $8C^2+9C$. That is **0.29%**. We come back to the fraction several times below. Tile borders damage the picture through it, and it is what makes the exact remedy of Section 4.4 affordable at all. It is also why we kept the adapters pointwise.

3.3 The exit ladder

Two consequences follow from the ladder, and both are easy to state wrongly. Exits shallower than j are indistinguishable from each other, because the first j groups run for every tile regardless, and the usable ladder therefore has $K-j$ distinct costs. The second consequence is the *architectural ceiling* $S_{\max} = 100(1-c_j)\%$, reached when every tile takes exit j . For our shipped setting ($K=6$, $j=2$, 256 px tiles) $S_{\max} = 38.3\%$. We show in Section 5.4 that this bound is *reached* in practice at low rate, so it behaves as an operating point and not as an asymptote.

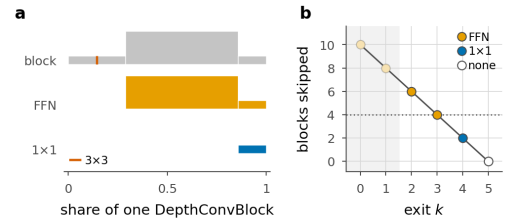


Figure 6. Inside an exit adapter. **a**, one DepthConvBlock and the two adapters, drawn to scale: bar length is a share of the block, bar height the channel width written. Neither adapter contains the block's only 3×3 (the hairline rule), so neither adds receptive field or seam penalty. **b**, blocks skipped per exit, coloured by adapter; the rule switches to the FFN for four skipped blocks. Lengths counted with hooks off the modules themselves.

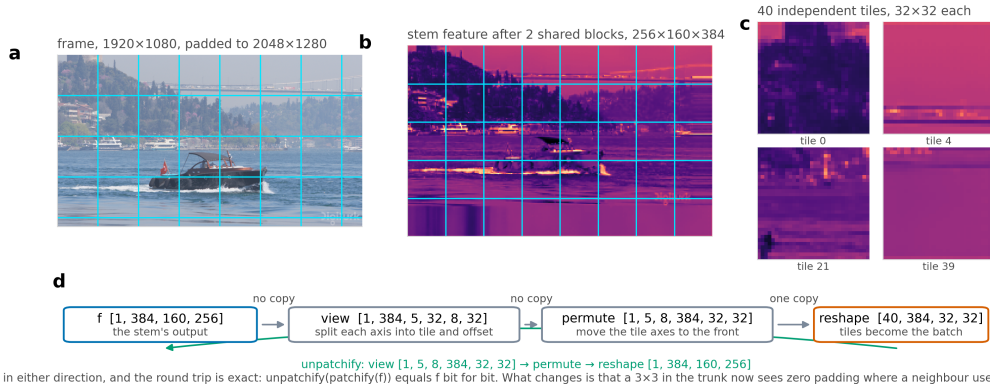


Figure 7. What patchify does. Captured from one real decode. **a**, the frame, padded to a whole number of tiles, with the grid the decoder will impose. **b**, the feature map after the shared stem, at one eighth of frame resolution, so a 256 pixel tile of picture is 32x32 of features; the tiling happens here and not in the pixel domain. **c**, four of the 40 tiles as the trunk now sees them, each its own batch element. **d**, the operation itself, and its inverse. It costs no arithmetic and the round trip is exact.

3.4 Exit adapters

An early exit hands the shared head a feature the head was not fitted to, and the adapter is the correction. For exits that skip little we use a residual 1x1, $Ad(f) = f + Wf$ with W zero-initialised. For exits that skip four blocks or more we use the pointwise expand/activate/contract pair of the block's own FFN. Both are *pointwise by design*. A 3x3 inside an adapter would add seam damage at the tiles that took an early exit, and those are the tiles least able to afford it. Zero-initialising the last layer makes every adapter exactly the identity at step zero, so the deepest exit is bit-exactly the released decoder before training begins and every shallow exit starts from “the decoder as it is”.

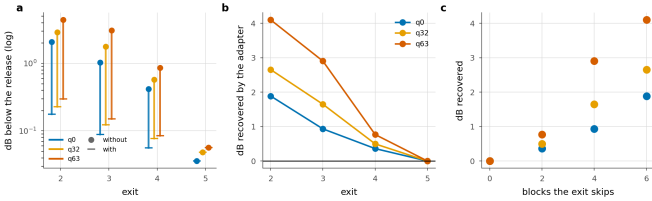


Figure 8. What the adapters are worth. **a**, each exit with and without its adapter. **b**, the dB the adapter recovers. **c**, the same against the number of blocks the exit skips, which is the design rationale measured.

	q0		q32		q63	
Exit	with	without	with	without	with	without
2	0.177	2.065	0.228	2.884	0.297	4.398
3	0.089	1.023	0.123	1.772	0.151	3.054
4	0.056	0.416	0.077	0.574	0.084	0.853
5 †	0.036	0.036	0.048	0.048	0.056	0.056

Table 4. What the adapters are worth. dB below the released decoder with every tile at that exit, with the trained adapters and with each set back to the identity it was initialised to. † the deepest exit has no adapter by construction and is the control.

Zeroing the adapters measures what they learned, since the identity is exactly what they were initialised to. Without them the shallowest exit costs 4.40 dB at q63, forty-four times the budget, and the adapter buys 4.10 dB of that back. We find the gain grows with rate and with the number of blocks the exit skips, so the design rationale here is a measurement rather than an argument, and the deepest exit moves by exactly zero. We would not describe the adapters as a refinement of the ladder, because without them it has no usable exit.

3.5 Allocation

The ladder and its costs are fixed by this point, and one decision is left. Every tile has to be given an exit, drawn from the usable ones, $k \geq j$, and we want the set of choices that keeps the frame inside its distortion budget for the least compute. There are $K-j$ choices per tile and N tiles, so searching assignments one by one is out of the question. What makes

the problem tractable is that the tiles do not interact: each contributes its own error $D(t, k)$ and its own cost c_k , and the only thing they share is the single budget the frame has to meet.

Problems of that shape are solved with a Lagrangian. Put a price λ on compute, add λc_k to the error of exit k , and the budget constraint drops out; what is left is N separate one-tile problems, each a minimum over $K-j$ numbers. Shoham and Gersho [28] showed that sweeping the price this way traces the lower convex hull of what the units can achieve together, and Ortega and Ramchandran [29] made the construction standard in image and video coding.

What λ does is set the exchange rate between compute and error. At $\lambda=0$ compute is free and every tile takes whichever exit has the smallest error. As λ rises, compute becomes expensive relative to error, and a tile drops to a shallower exit as soon as what that exit saves is worth more than the error it adds. Compute falls and error grows as λ grows, neither of them turning back, so one value of λ puts the frame exactly on its budget and we find that value by bisection. The rule each tile follows is

$$k^*(t) = \arg \min_k [D(t, k) + \lambda c_k] \quad (1)$$

with the minimum taken over $k \geq j$. We call $k^*(t)$ the choice of the *Lagrangian oracle*: it is made with the true error of every exit in hand, which no decoder has, but it is optimal only within the family of allocations one multiplier can reach. That qualifier is not cosmetic. Section 5.7 exhibits allocations that are cheaper at the same distortion than anything this rule produces, so *oracle* here means best under a single multiplier and not the global constrained optimum. Where we mean the latter we say so. Both quantities are available to the *encoder*, which holds the source, so configuration A can run this search exactly.

What the search costs the *encoder* depends on how the table of $D(t, k)$ is built, and the two ways of building it are further apart than they look. Take one tiled decode at 1080p as the unit. The deployed table of Section 3.1 needs one tiled decode per exit, which comes to 4.6x the unit rather than K times it, because a shallow exit is cheaper to run than a full one. The full-frame table taps every exit off a single full-frame pass and comes to 1.4x. Section 5 shows that the full-frame table must not be used to *report* quality, but it can still be used to *rank*. Running the argmin on it, and the deployed path only to check the budget, we find the two searches agree on 85% of tiles and land within 0.4 points of saving of each other (19.0% against 18.6%, both under budget). The exact search therefore costs 4.6 decodes and a practical encoder need not pay it; ranking on the full-frame table is the one extra decode per frame we quote for A.

The decoder cannot run the argmin. $D(t, k)$ is the error against the source, and the source is the one thing a decoder never receives. The gap is one of information: the errors the argmin compares depend on a picture that

is not in the bitstream, so no decoder-side model recovers them, however large it is. What a decoder can do is estimate which exit the Lagrangian oracle would have picked, and configuration B therefore *predicts*. Its head reads what the decoder already holds, the feature at the split point, the decoded latent $\hat{y}$, the entropy model's scales and the quality index. For each tile t it emits a vector of logits z_t , one entry per exit, and the tile takes

$$\hat{k}_t = \arg \max_k [\log \text{softmax}(z_t)_k - \beta c_k] \quad (2)$$

The first term is the head's score for exit k , on a log scale. The second is the price on compute that λ carries in the Lagrangian: raising β takes more away from the deep exits than from the shallow ones, so more tiles are handed to cheap exits and the frame again gets cheaper and worse. We bisect β against the budget just as we bisect λ .

Where β comes from at deployment. A real decoder cannot run that bisection. It would need to know the delivered distortion, and the distortion is measured against a source the decoder never receives, which is the same missing variable that stopped it running the Lagrangian in the first place. Saying otherwise would smuggle the source back in at test time.

So β is not chosen at test time. It is calibrated offline, once, on a held-out set, and shipped as a table indexed by the quality index and the budget. The decoder reads the quality index out of the bitstream, looks β up, and runs the argmax above; nothing in the loop needs the source. The table is 64 quality indices by however many budgets a deployment offers, which is a few hundred floats.

We calibrate it on the 512 held-out Open Images validation frames, which are disjoint from the training images and from the test sequences: β is bisected to the budget on those, once per quality index, and the resulting table is then applied unchanged to the test set. Section 5.5 reports both that number and the one obtained by bisecting on the test set itself, because the difference between them is the only honest measure of whether the table transfers.

Why a logarithm. The oracle's rule adds a distortion to a price. The head does not produce a distortion; it produces a distribution over exits, and the quantity that plays the same additive role is the surprisal $-\log p$, which is what the head is trained to make small on the oracle's label. Writing the rule with log softmax puts the two terms in one currency: what exit k costs in belief against what it costs in compute, with β the exchange rate between them. It also makes the comparison invariant to a constant added to every logit, which the softmax removes, so βc_k is the only thing tilting it. Using the probability itself would set a number bounded in $[0,1]$ against a cost measured in decodes, and would compress every distinction between confident exits into a range near one, where β would have to move by orders of magnitude to change any decision.

The bracket for the bisection is $[-2 \times 10^4, 2 \times 10^4]$, which is wider than it looks like it needs to be. A trained head is confident: its log-probability gaps reach the hundreds, and β has to be able to outweigh them before the allocation moves at all. A narrower bracket we tried first, $[-50, 50]$, never reached the all-deepest allocation and so never bracketed the budget from below.

The head, exactly. Three things enter. The first is the feature at the split point, 384 channels at one eighth of frame resolution, which is the last thing every tile shares. The second is the decoded latent $\hat{y}$ concatenated with the entropy model's scales σ , the predicted Gaussian width used to code each latent position: σ is already computed during the decode and is, position by position, an estimate of how hard that position was to code. The third is the quality index. Each of the first two is projected by a 1×1 convolution, to 48 and 32 channels, and then reduced to one vector per tile by taking the mean and the standard deviation over the tile. That gives $96 + 64 + 1 = 161$ numbers per tile, which pass through LayerNorm and a three-layer perceptron of width 256 with SiLU

activations, ending in K logits. The whole head is 144,030 parameters and 0.162% of the decode it is deciding about, almost all of it in the 1×1 on the stem, which is the only part that runs per pixel. The perceptron runs once per tile, forty times for a 1080p frame, so its width is nearly free.

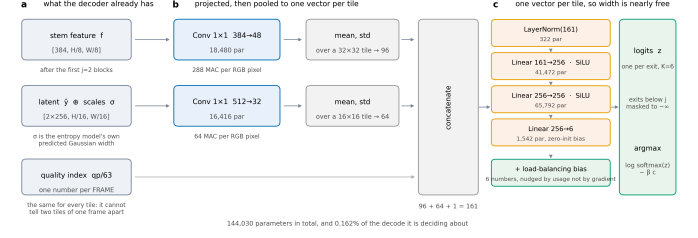


Figure 9. The router head. a, the three things it reads, all already held by the decoder. b, each projected by a 1×1 , then reduced to one vector per tile by its mean and standard deviation. c, a three-layer perceptron, one pass per tile. Every shape and parameter count is read off the module when the figure is drawn.

Two details matter for reproducing it. Exits below the split depth do not exist, and their logits are masked to $-\infty$ rather than to a large negative constant; nothing in the objective penalises a constant offset, the raw logits drifted to around -10^4 during training, and a -10^4 mask then became the *largest* entry in the row. The balancing term is a per-exit bias added to the logits before the decision and updated by usage rather than by a gradient, which is what makes it loss-free.

One head for every quality index. The index is an input, not a selector. During training it is drawn uniformly at random for each batch, so one set of weights covers all 64 operating points, and at test time the same head runs at every rate. It enters as a single number per frame, which means it can tell the head which operating point it is at and cannot, even in principle, distinguish two tiles of the same frame. That is why it stays live in every variant of the input ablation: the variants then differ in per-tile information and in nothing else, and the variant with only the quality index is the control that measures what agreement is reachable with no per-tile information at all.

The head is trained against a *frozen* decoder, so the exits it chooses between do not move while it learns. Its target is the oracle's choice $k^*(t)$ and the loss is a cross-entropy to that target, with each tile weighted by its *regret*, meaning by how much the bracket in the Lagrangian grows if the head's exit is used in place of the oracle's. A tile whose two best exits are nearly tied then counts for less than one where the wrong choice is expensive. A router like this can collapse onto a single exit during training, so it carries a balancing term; ours is loss-free [16] and is biased toward the oracle's own exit distribution instead of toward uniform. At a high λ the oracle genuinely does send every tile to one exit, and forcing spread there would force mistakes. Section 5.5 measures what B gives up against A.

3.6 Training

All exits are decoded every step and the objective is

$$\mathcal{L} = \mathcal{L}_{RD} + w_a \mathcal{L}_{\text{anchor}} + w_d \mathcal{L}_{\text{distill}} \quad (3)$$

where $\mathcal{L}_{RD}$ is the released rate-distortion loss averaged over exits, weighted per exit by fixed α_k and scaled by λ_{rd} , the codec's own rate-distortion weight, which is a different multiplier from the λ of the Lagrangian above. The anchor term $\mathcal{L}_{\text{anchor}}$ pins the deepest exit's reconstruction to the released decoder's, so the reference every saving is quoted against cannot drift away underneath the measurement. The distillation term $\mathcal{L}_{\text{distill}}$ supervises the adapters in feature space, matching the feature at exit k to a stop-graduated copy of the feature at exit $k+1$. We work in feature space and not in pixels because the head is a fixed map from feature to RGB, so matching the deeper feature is the stronger constraint, with 384 dense channels of target instead of 3. Each exit imitates its neighbour, exit k following exit $k+1$, and not the deepest

exit, for the reason given in [19].

We train the adapters *through the tiled decode path they are deployed in*. Training them full frame and tiling only at inference loses 0.14–0.24 dB; training through the deployed path gains 0.51–0.90 dB, so the sign of the effect flips.

4. The price of tiles

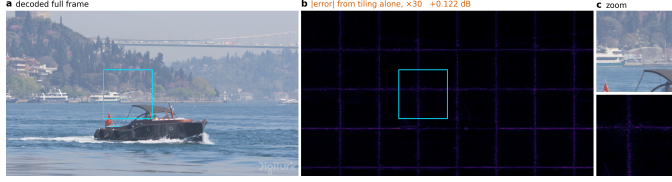


Figure 10. The artefact, before anything is done about it. Bosphorus at 1080p, quality index 63, zero padding at the tile border. **a**, the frame decoded in one piece. **b**, the same bitstream and weights decoded in 256 pixel tiles, every tile at full depth: the difference between the two, amplified 30 times. **c**, the boxed region of each.

A 3×3 depthwise at feature position (x,y) computes a weighted sum over its neighbours. Decoded full frame, those neighbours exist. Decoded per tile they do not, and the kernel is handed whatever the padding rule invents. Each convolution extends the affected region by one ring, so with b per-tile blocks on a tile of side F the fraction of the tile within reach of an invented value is

$$1 - \left(\frac{F-2b}{F}\right)^2$$

At our shipped $F=32$, $b=8$ that comes to 0.750. Three quarters of the tile is affected, so what we are looking at is a structured error over most of the tile and not a thin border at its edge.

That fraction tells us which pixels are affected and not how badly, and it turns out to be the wrong predictor of the penalty. We swept the split depth, which sweeps b from 12 to 0 with $b=0$ as an exact zero-seam control, and measured

$$\text{seam} \propto b^\alpha, \quad \alpha \approx 2 \quad (5)$$

with $\alpha = 2.38$ at q0, 2.22 at q32 and 1.93 at q63, and $r = 0.98$ –0.995 in log-log. Fitted with one free scale, the area fraction is wrong by 160–264% where the power law is wrong by 12–22%. Fixing the exponent at exactly two costs a factor of two, 31–38%, so the square is the right shape without being quite the right law. The exponent has a reading. The count of affected pixels grows like perimeter times depth, and the error accumulated in each of them grows with how many convolutions reached it. Once every pixel has been touched the area law saturates and the penalty does not, which is where the area law fails as a predictor.

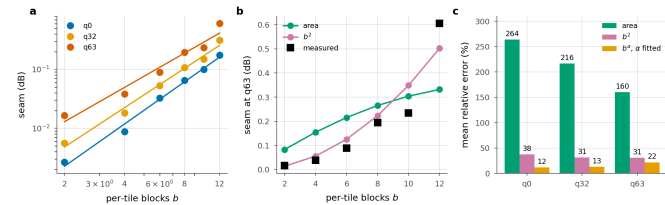


Figure 11. The seam against per-tile depth. **a**, the measurement with the fitted power law. **b**, both models against q63, one free scale each. **c**, mean relative error. The area fraction saturates once the border reaches every pixel; the penalty does not.

4.1 Padding is an estimator

Border padding is an *estimator* of the unseen neighbour, and the seam is its error. The table below measures four of them, with early exit switched off so that tiling is the only difference from a full-frame decode.

Border estimator	q0	q32	q63
Zeros (stock)	0.126	0.287	0.677
Replicate	0.071	0.123	0.218
Linear extrap.	0.198	0.286	0.384
AR(1) per channel [11]	0.061	0.107	0.197

Table 5. Border estimators, dB below the released decoder on the same latent, 256 px tiles, full CTC. Lower is better. Replication, a zero-order hold, beats linear extrapolation by a wide margin, and the fitted AR(1) wins on quality while costing +10.7% of decode wall-clock.

A higher-order estimator is worse. Extrapolating the local gradient past a boundary amplifies whatever noise sits on that boundary, and assuming local constancy does not. The gap is wide, 0.384 dB against 0.218 dB at q63. **The best estimator loses on cost.** The per-channel AR(1) fit of [11] reaches 0.197 dB, about 0.02 dB better than replication, and it costs 10.7% of decode wall-clock. Against a 0.1 dB budget and a ~24% saving that trade does not close, so we drop it.

4.2 What actually removes the seam

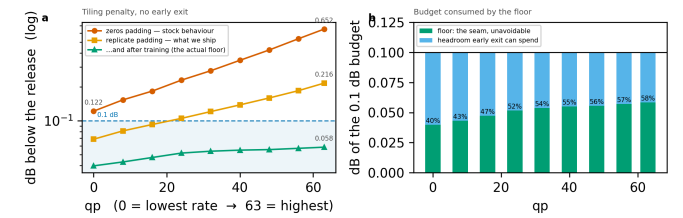


Figure 12. The tiling penalty across the whole rate range, every tile at full depth. The two steps that matter cost nothing, and the largest single factor is training the ladder with the seam present. Panel (b): the floor is charged *inside* the distortion budget and consumes a growing share of it.

Tile size is free in arithmetic. A tiled decode's MAC count does not depend on it at all, and the damage scales with the tile perimeter, as the table below shows. What tile size costs is routing granularity, 40 tiles per 1080p frame at 256 px against 160 at 128 px.

Estimator, q63	128 px	256 px	ratio
zeros	1.249	0.677	×0.54
replicate	0.372	0.218	×0.58
arls	0.332	0.197	×0.59

Table 6. Tile size, dB at q63. Doubling the side roughly halves the penalty, as the power law above predicts, and costs no computation.

The largest factor of all is the easiest one to miss. On the untrained ladder, replicate padding leaves 0.218 dB at q63; after training, the same setting leaves 0.072 dB. So more than two thirds of what the estimator could not fix is absorbed by weights learning to live with it. That single change removes more of the seam than any module in this paper does.

4.3 A learned deblocking filter, and why we reject it

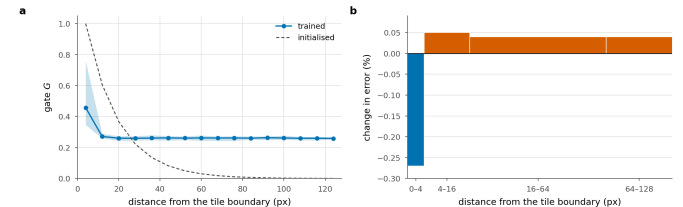


Figure 13. A learned deblocking filter, gated by position within a tile and applied once to the stitched frame, for 0.95% of the decode. **a**, the trained gate against distance from the boundary, beside its initialisation. **b**, the change in error by that distance; bar width is the share of pixels, so bar area is the contribution to the frame. It wins on the 6.2% of pixels nearest a boundary and loses on the 94% beyond.

What a standard codec does about a partition boundary is deblock it, with a filter applied after reconstruction [9, 21]. The tile lattice is known

exactly at training and at inference, so ours can be *told* where to look instead of having to infer it

$$\text{Rep}(f) = f + G[x \bmod P, y \bmod P] \cdot \text{PW}(\text{WSiLU}(\text{DW}_{3 \times 3}(f))) \quad (6)$$

with PW a pointwise convolution, DW a depthwise one, WSiLU the weighted SiLU of the DCVC line [33], G a $P \times P$ gate shared over channels, P the tile pitch in feature samples, and G initialised at $\exp(-d/\tau)$ in the distance d to the nearest tile boundary with τ a fixed decay length. It costs 0.95% of the decode.

It does not earn that. Splitting the per-pixel error by distance from the nearest tile boundary, we find the filter gains 0.27% in the 0–4 px ring, which is 6.2% of pixels, and loses 0.04–0.05% everywhere else. Give it a *perfect* gate, with zero correction in the interior and the boundary gain unchanged, and the ceiling on what it could earn is ≈ 0.0008 dB, for 0.95% of the decode. We rejected AR(1) padding at 0.0019 dB per point of decode, and this is worse by an order of magnitude. Tightening the gate cannot rescue it, because there is almost nothing left to win.

4.4 Removing the cause, and why it does not help

Only 0.29% of each block has spatial extent, so the exact fix is affordable. Give the 3×3 its real neighbours across the tile border, a *halo exchange* narrowed to the one operator that needs it; local padding [31] does the same at every layer. The cost is +0.032% of the decode, thirty times less than the deblocking filter. The fix is also exact. At uniform depth a tiled decode with the exchange is bit-identical to a full-frame one, once the comparison is not confounded by the filter, which runs on the stitched frame and has no full-frame counterpart. Measured: $1.13\text{e-}2$ with the filter on, **exactly 0** with it off, against $6.06\text{e-}2$ for replicate padding.

The exchange also removes most of the floor. Switched on at inference, it drops the floor from 0.036 to 0.003 dB at q0 and from 0.056 to 0.030 at q63, which is 91% and 47% of the tiling penalty.

And it destroys the allocation. At the same 0.1 dB budget the saving falls from 25.9% to 4.2% at q32 and from 19.2% to 0.4% at q63.

The reason is structural, and it is written into the condition on the exactness claim. The exchange is exact when every tile is at the *same* depth, and routing deliberately violates that condition. With replicate padding a tile is entirely independent of its neighbours, so the allocation is free to give adjacent tiles any depths it likes. The exchange makes a tile depend on its neighbours' features, and under routing those neighbours ran a different number of blocks. A shallow tile reading a deep neighbour's activation is an arrangement the weights have never seen, and that mismatch costs more than the seam it removed.

The natural reading of the exactness result is that the exact fix should simply be adopted, and that reading is wrong. We report the negative result for that reason, and because the tension between the halo exchange and routing belongs to the combination itself, so any spatially-adaptive decoder will inherit it. One caveat keeps the finding from being final, the same one that applies to the deblocking filter. This model was trained with padding, and a run trained *with* the exchange could reverse the result. The burden of proof lies with that run.

5. Experiments

Setup.

We fine-tune the decoder on 512×512 crops from OpenImages with the encoder frozen ($\max|\Delta| = 0$ is asserted every run). One λ_{rd} is drawn per sample from a log-spaced range covering all 64 quality indices, so a single set of weights covers the whole rate range. We evaluate on one intra frame from each of the 53 sequences of the common test set (CTC: UVG, MCL-JCV and HEVC classes B, C, D and E), one intra frame each.

Measurement protocol.

Two details of the protocol move the numbers enough that we state them here. The per-tile distortion table has to be built on the *deployed* decode path, that is, one tiled decode per exit, rather than by tapping the exits of a single full-frame forward pass. Tapping is the natural implementation and it is correct for training. But with a full-frame reference on the other side of the ratio, the tiling penalty cancels, and the quality reported is that of a decoder nobody ships. On our model the gap is +0.035 dB at the deepest exit and +0.008 dB at the shallowest. It is not a constant offset; it grows with how many blocks ran per tile, so we cannot correct for it after the fact. The second detail is that once the allocation is chosen, we decode that *mixed* map once and report the distortion it actually produces.

Reporting conventions.

Every dB is measured against the *released* decoder's full-frame decode of the *same* latent, and our side is the deployed tiled decode, so the tiling penalty sits inside every number. Savings are fractions of the released decoder's cost. Our own deepest exit costs 1.0095 of it, and using that as the denominator would flatter every result by 0.6–0.8 points.

5.1 Main result



Figure 14. What the saving looks like. The same bitstream decoded by the released decoder and by ours at the 0.1 dB operating point, 30.5% fewer multiply-accumulates. The crop is the tile that gave up the most quality, chosen automatically, so it shows the method's worst case on this frame rather than a flattering one.

Budget	q0	q16	q32	q48	q63	mean
0.10 dB	29.2	25.0	20.4	17.9	15.2	21.5
0.20 dB	38.3	37.0	33.0	31.0	28.9	33.7
0.30 dB	38.3	38.3	38.3	37.1	35.0	37.4
0.50 dB	38.3	38.3	38.3	38.3	38.3	38.3

Table 7. Decoder MACs saved (%) against the released decoder, per quality index and distortion budget, configuration A. The 0.3 and 0.5 dB rows sit on the architectural ceiling almost everywhere; past that point a looser budget buys nothing.

One word on the budget before the numbers. A tenth of a decibel is an engineering convention, chosen because it is small against the spacing of the rate points and because codec work has long used differences of this size as a working tolerance. It is not evidence that the difference is invisible, and we make no perceptual claim for it. The 0.2, 0.3 and 0.5 dB rows are there so a reader who disagrees with the choice can read off another one.

The table carries the headline numbers. At 0.1 dB the method saves 29.2% at the lowest rate and 15.2% at the highest, and 21.5% on average, at a BD-Rate cost of 1.03%; that is, the compute is worth about the same as a 1.03% increase in bitrate. We do not read the fall with rate as an artefact of the ladder. High-rate reconstructions carry detail the shallow exits cannot reproduce, and the floor rises with rate as well; both effects push the same way.

Decoder	GMAC	% of release	ms
Released DCVC-UF	454	100.0	111
FLEX-UF, all tiles deepest	458	101.0	114
FLEX-UF, 0.1 dB budget	356	78.5	78
FLEX-UF, architectural ceiling	263	58.1	—

Table 8. Decoder complexity at 1080p. Our deepest exit costs slightly more than the released decoder because it still pays the deblocking filter; that 1.0095, not 1.0, is what every saving in this paper is *not* divided by. Wall-clock is the sorted loop of Section 5.9.

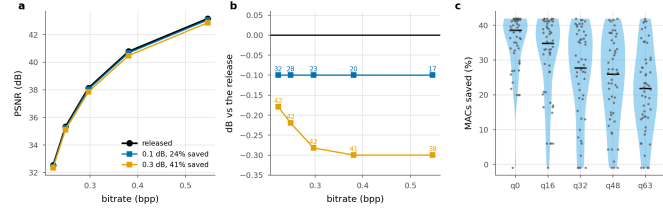


Figure 15. The plane a codec is read on, and the spread behind the mean. a, operating points against the released curve. b, the same with the quality axis expanded; labels are compute saved. c, per sequence at a matched point near 0.1 dB; one dot per sequence, bar is the median.

Panel c shows the distribution that the headline averages over, and it is wide. At q0 the median sequence saves 38.6%, with an interquartile range of 32.0–41.5, while the worst saves –1.0%. The worst cases are the low-resolution sequences of Section 5.3, where two tiles leave nothing to allocate. Reporting the mean alone would hide both ends.

5.2 Is per-tile adaptivity necessary?

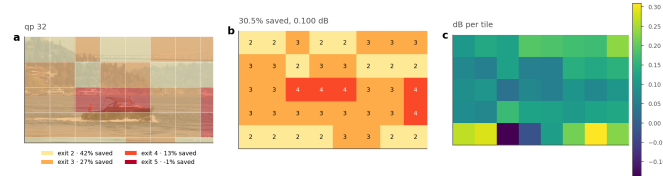


Figure 16. Where the decoder spends. Bosphorus at q32, a 0.1 dB budget. (a) the assignment overlaid on the frame, (b) the exit index per tile, (c) the quality each tile gives up, against its own full-depth reference. Water and sky leave at the shallowest exit; the boat and the shoreline run deep.

q	Allocation	Δ PSNR (dB)	MACs saved (%)
0	uniform, exit 2†	0.179	39.1
	uniform, exit 3	0.093	24.2
	uniform, exit 4	0.059	13.0
	uniform, exit 5	0.036	-1.0
	random (matched mix)	0.125	29.9
	rate-ranked (free)	0.107	29.9
	oracle	0.100	29.9
16	uniform, exit 2†	0.219	39.1
	uniform, exit 3†	0.118	24.2
	uniform, exit 4	0.075	13.0
	uniform, exit 5	0.045	-1.0
	random (matched mix)	0.133	25.6
	rate-ranked (free)	0.106	25.6
	oracle	0.100	25.6
32	uniform, exit 2†	0.282	39.1
	uniform, exit 3†	0.147	24.2
	uniform, exit 4	0.090	13.0
	uniform, exit 5	0.054	-1.0
	random (matched mix)	0.141	20.9
	rate-ranked (free)	0.107	20.9
	oracle	0.100	20.9
48	uniform, exit 2†	0.333	39.1

q	Allocation	Δ PSNR (dB)	MACs saved (%)
63	uniform, exit 3†	0.176	24.2
	uniform, exit 4†	0.103	13.0
	uniform, exit 5	0.062	-1.0
	random (matched mix)	0.145	18.3
	rate-ranked (free)	0.107	18.3
	oracle	0.100	18.3
	uniform, exit 2†	0.396	39.1
	uniform, exit 3†	0.206	24.2
	uniform, exit 4†	0.116	13.0
	uniform, exit 5	0.072	-1.0
	random (matched mix)	0.141	15.6
	rate-ranked (free)	0.110	15.6
	oracle	0.100	15.6

Table 9. Adaptivity against two controls at the 0.1 dB budget. † marks uniform depths whose distortion exceeds the budget, so they are not admissible allocations at all. “Random” draws each frame’s map from the oracle’s own exit histogram and shuffles it across tiles, which preserves the average cost and the mix of depths while discarding the content dependence.

The obvious alternative to routing tiles is to run every tile at the same shallower exit and accept the loss. The table puts that option, together with two stronger controls, at matched compute.

The uniform rows answer the question directly, and the answer sharpens with rate. At the lowest rate a static decoder can reach exit 3 inside the budget and save 24.2%, against the Lagrangian oracle’s 29.2%. At q48 and q63 the only uniform depth that fits is the deepest one, which costs 1.0% instead of saving anything. At those two rates, then, the comparison is between 18.3% and 15.6% against nothing at all, not against something smaller. Averaged over rates, the best static allocation saves 9.7% where the adaptive one saves 21.5%.

The two shuffled rows separate effects that are easy to conflate. We keep the oracle’s own exit histogram, so the mix of depths and hence the average cost are unchanged, and assign it to tiles at random. Quality falls sharply. What the allocation buys is knowing *which* tiles can afford to run shallower; running some fraction of them shallower is worth nothing by itself. The oracle-histogram bit ranking keeps the same histogram and orders it by a signal the decoder already holds, namely the bits the entropy model spent on each tile. This is the cheapest router we can think of. It has no parameters and no training, and the number is available before the trunk starts. It is also the natural analogue of the confidence rules used by early-exit classifiers [3, 20], which likewise read a signal off the network’s own output.

It recovers most of the gap. At q63, random costs 0.141 dB and the oracle 0.100 dB at identical compute, while the oracle-histogram bit ranking costs 0.110 dB. That is 75% of the oracle’s advantage over chance, at 0.68–0.79 agreement with the oracle’s map. A learned router therefore has to beat a calibrated bit rule that is already three quarters of the way there. We would not have run that comparison without this control, and we suggest that any adaptive-inference paper report it. Given the Lagrangian oracle’s histogram, what we measure here is *ranking* and nothing else. Section 5.6 removes that crutch and turns the same signal into a complete routing rule, which matches the trained head across the whole rate range.

5.3 Resolution, and the granularity of a tile

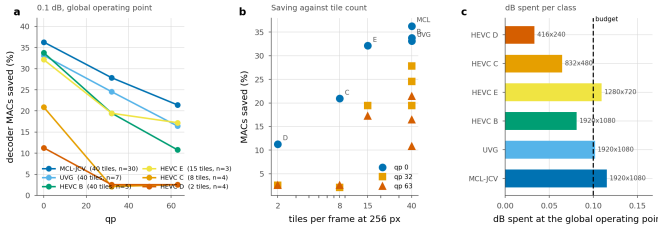


Figure 17. Saving by test class at one global operating point. λ is bisected once so the whole set lands on the budget, exactly as it would be in deployment; each class is then reported at that λ . The ordering follows tile count, not content.

Class	Resolution	Tiles	n	q0	q32	q63
MCL-JCV	1920x1080	40	30	33.7	26.2	20.2
UVG	1920x1080	40	7	30.6	23.1	15.5
HEVC_B	1920x1080	40	5	31.1	18.2	9.7
HEVC_E	1280x720	15	3	29.1	18.4	15.5
HEVC_C	832x480	8	4	18.0	1.7	2.5
HEVC_D	416x240	2	4	8.8	2.5	2.5

Table 10. Saving by test class (%), at a 0.1 dB budget set globally over all 53 sequences rather than per class. “Tiles” is how many 256 px tiles a frame of that resolution contains. The saving tracks tile count much more closely than it tracks content.

Completing the test set exposed a dependence that the 1080p-only subset had hidden. The table groups the saving by class. At 1080p, where a frame is 40 tiles, we save 33.7% (MCL-JCV), 33.1% (UVG) and 33.8% (HEVC B) at the lowest rate, three classes of very different content within three points of each other. At 832×480 a frame is 8 tiles and the saving is 18.0%; at 416×240 it is 2 tiles and 8.8%. At high rate the small resolutions fall to 2.5% against 20.2% for 1080p.

The natural explanation is granularity. With two tiles per frame there is almost no allocation left to make, and the Lagrangian degenerates towards a uniform choice. That suggests a fix. Choose the tile size relative to the frame, since the MAC count does not depend on tile size at all and only the seam does.

We tested that fix and it mostly does not work. We compare the 256 px tiling against a 128 px one at q0, which multiplies the tile count by 3.4. MCL-JCV (1080p) goes from 36.3 to 34.3, HEVC B (1080p) from 33.8 to 32.1, HEVC C (832×480) from 20.9 to 17.6, and HEVC D (416×240) from 11.3 to 13.7.

Smaller tiles help only at the smallest resolution, where the count goes from two to eight. Everywhere else they cost between 1.7 and 3.3 points, including at 832×480, where the count rises from 8 to 28 and the saving still falls. Beyond a modest number of tiles, the extra seam outweighs the extra granularity. We report the comparison as indicative, since the two tile sizes come from different training runs and tile size is therefore confounded with training. But the direction is consistent, and it is enough for us to say that a resolution-adaptive tile size is not the easy win the granularity argument suggests.

5.4 The band a distortion budget works in

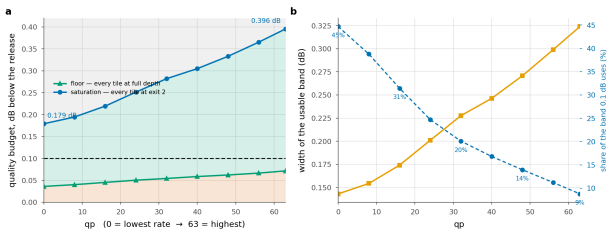


Figure 18. Three regions, and only the middle one is a design choice. Below the floor no allocation meets the budget; above saturation every tile already takes the cheapest exit. The 0.1 dB budget uses 45% of the band at the lowest rate and 9% at the highest.

q	0	16	32	48	63
Floor D _{min}	0.036	0.045	0.054	0.062	0.072
Saturation D _{sat}	0.179	0.219	0.282	0.333	0.396
Usable band	0.143	0.174	0.228	0.271	0.324
0.1 dB uses	45%	31%	20%	14%	9%

Table 11. Floor and saturation, dB below the released decoder. The floor is what tiling costs with no early exit at all; saturation is where every tile reaches exit j and the ceiling is attained.

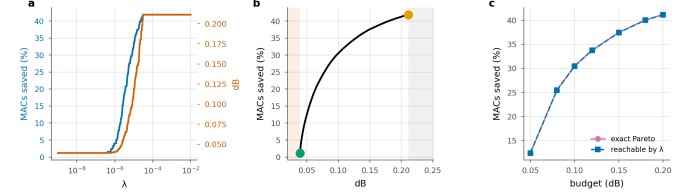


Figure 19. The structure of the allocation, on one frame. **a**, compute and distortion are monotone in the multiplier. **b**, the frontier between floor and saturation; shaded regions are infeasible or wasted. **c**, what the Lagrangian reaches against the exact Pareto set, enumerated by dynamic programming.

One assumption is worth naming before any of this. The argument treats the frame's distortion as a sum over tiles, while the deblocking pass runs on the stitched frame and therefore sees the whole exit map at once, so the tiles are not strictly independent. We measured the size of that coupling over 60 allocations and the largest mismatch between the separable prediction and a real decode is 5.5×10^{-5} dB, four orders of magnitude below the budget. We proceed as if the objective were separable and treat that as measured rather than assumed.

The construction has enough structure to state as propositions, and we check each one numerically instead of asserting it (scripts/verify_theory.py, seven of seven). The allocation decouples per tile; compute is non-increasing and distortion non-decreasing in λ ; the sweep traces the lower convex hull and therefore cannot reach an interior point of the achievable set; $\lambda=0$ reaches the least distortion the ladder can produce; beyond a finite λ , computable in closed form, the allocation is the constant map to exit j at cost exactly c_j ; and the ceiling is a function of the split depth alone.

The third of those has a practical edge. The sweep reaches only hull vertices, so a budget that falls between two of them cannot be met exactly. We measured what that costs by enumerating the *exact* Pareto set with dynamic programming over tiles, which is feasible because there are only four distinct exit costs. The sweep reaches 95 allocations against a Pareto set of 635, and the loss from convexity is at most 0.05 saving points. For this problem the standard construction is essentially optimal.

It matters *why* that loss is small, since the reason is structural and points at what would make it smaller. The reachable costs form a lattice, and its spacing is set by how much the mean cost moves when the multiplier crosses a breakpoint. If m tiles each move to the next exit there, the spacing is at most $m \cdot \max(c_{k+1} - c_k) / T$ saving points. The convexity loss cannot exceed the largest spacing, because a budget between two levels is served by the lower one. Here $m=2$, since tiles with identical rows switch together, and the bound is 0.745 points; the measured spacing sits exactly on it. Nothing in the expression depends on content or rate, and the bound falls as $1/T$: halving the tile side puts four times as many tiles on the lattice, each carrying a quarter of the step. Fine granularity is what makes the Lagrangian relaxation lossless in practice, and no property of the images is doing that work.

Two numbers bound what any budget can do. The **floor** is the distortion of a tiled decode with every tile at full depth, which is pure tiling penalty: 0.036 dB at q0, rising to 0.072 dB at q63. A budget below it admits no allocation. The **saturation** point is the distortion when every tile takes exit j; any budget at or above it reaches the ceiling, and a larger one reaches nothing more.

One consequence of that is worth stating plainly. At q0 the ceiling is reached at 0.179 dB, so the architectural bound behaves as an operating point rather than as an asymptote. Past it, the limiting factor is the *ladder* and not the budget, which is an argument for a finer ladder before a looser budget. The same two numbers pick out a narrow regime: budgets in [0.179, 0.194] dB saturate the lowest rate and nothing else, a range 15 thousandths of a decibel wide.

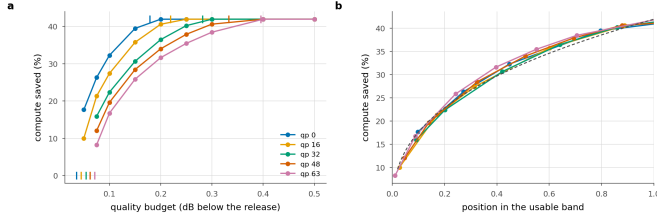


Figure 20. The band is the rate dependence. **a** Saving against the budget, with each rate's floor and saturation point ticked. **b** The same with the budget axis rescaled onto each rate's own band. Five curves become one; dashed is the one-parameter power law fitted to all of them.

And the band is almost all of the rate dependence. At a matched decibel the five rates are 15.5 points apart. Rescale the budget axis onto each rate's own band, with the floor at 0 and saturation at 1, and they collapse onto a single master curve, 1.7 points apart on average and 2.2 at worst. The answer to “how much does a 0.1 dB budget buy at this rate” is therefore, to within a couple of points, “where does 0.1 dB sit in this rate's band”. The floor and the saturation point are both in closed form and both cheap to measure. What they leave over is small enough that a deployment could calibrate the two ends and read the rest off one curve. That curve is a one-parameter power law, saving $\approx C \cdot u^{0.38}$, with C the architectural ceiling and u the position in the band. We fit it in log space over all five rates and get $R^2 = 0.989$, worst residual 2.0 points. The claim we make is the *rescaling*: measuring the budget in units of each rate's own band is what collapses the curves. The exponent is an empirical fit to that collapse and not a law. Inverse fits to the same frontier disagree in it by up to 40%, so we report it as a description of these curves and read nothing into its value.

The collapse appears robust across two checkpoints. We repeated the measurement on a different training run, BEST, a separate recipe taken at a different epoch, and the same thing happens. The rates are 16.9 points apart at a matched decibel and 1.6 after rescaling, and the collapsed curve is again a power law, $R^2 = 0.984$. The *exponent* does not carry over: 0.33 there against 0.38 here, so it describes a trained decoder and not the architecture. What replicates is the collapse. Within a checkpoint, the rate dependence of the trade-off is the rate dependence of the band.

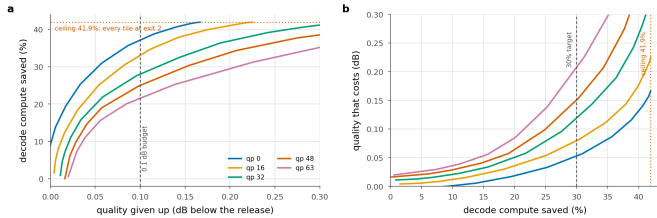


Figure 21. The trade-off, whole. **a** What a budget buys, per rate; the ceiling is 38.3% and q0 reaches it at 0.179 dB. **b** The same relation inverted, so it reads as what a saving target costs. The three budgets reported elsewhere are three points on this curve.

5.5 Signalled versus predicted allocation

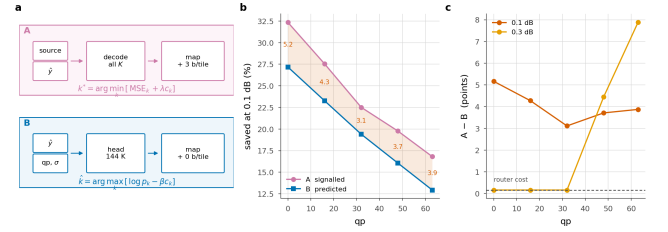


Figure 22. Who decides. **a**, the two decision paths side by side: the encoder's search over all K exits per tile against the head's forward pass over decoded data. **b**, saving at 0.1 dB; shading is what a bit-exact bitstream costs. **c**, that cost is smallest at q32 and grows in both directions.

	0.1 dB	0.3 dB				
q	A	B	gap	A	B	gap
0	29.2	23.6	5.6	38.3	38.2	0.2
16	25.0	20.3	4.8	38.3	38.2	0.2
32	20.4	16.8	3.6	38.3	38.2	0.2
48	17.9	13.6	4.2	37.1	32.6	4.5
63	15.2	10.7	4.5	35.0	26.9	8.1

Table 12. Signalled against predicted at two budgets, same checkpoint and test set, with one router trained against the oracle on the deployed table at a single λ .

The table above compares the two, and what it prices is exactness against bits.

Not signalling costs 3.6–5.6 points, roughly flat across rate, for zero added bits and a byte-identical file. The gap is smallest at q32 and widens toward both ends of the rate range.

It tracks $|\beta|$, the cost multiplier the bisection has to apply to move a router trained at one λ onto another operating point. At q32 the required β is 10, essentially none, since that is where the training λ lands, and there the gap is at its minimum. At q0 it is 146 and the gap is 5.6. A large β in either direction lets the cost term dominate the logits, which discards the content ranking the router learned. The remedy is a router per operating point, and a deployment would have one anyway: a single set of weights covers all rates, and a 144 K head per rate is 0.3% of the model each.

Loosening the budget closes the gap only where the budget saturates the ladder. At 0.3 dB the gap is 0.2 points at the three lowest rates. That is exactly the router's own 0.163% of decode, so its *prediction* is free there; both configurations send every tile to the cheapest exit, and nothing is left to predict wrongly. At the two highest rates 0.3 dB does not saturate, and the gap *widens* to 8.1 points. A looser budget gives the allocation more room to be wrong in as well as more room to be right.

For a system that trains once, the single-router number is the honest one, so that is what we report. It understates what B can do.

β held out		β bisected on the test set					
q	β	saving	dB	β	saving	dB	at equal dB
0	146	23.6	0.098	146	23.6	0.099	-0.0
16	133	21.3	0.105	100	20.3	0.100	+0.0
32	38	18.1	0.109	10	16.8	0.100	+0.0
48	-34	14.9	0.108	-46	13.6	0.100	+0.0
63	—	—	—	-57	10.7	0.100	—

Table 13. Choosing β without the test set. Left, β bisected on the 512 held-out validation frames and then applied to the test frames unchanged; right, β bisected on the test frames themselves, which is what the signalled-against-predicted table reports. Savings are hook counts on the routed decode with the router's own compute charged against them. The last column re-bisects on the test set to the quality the held-out β delivered, so the two allocations are differenced at one distortion rather than across two.

Where the β above came from. Every β in the signalled-against-predicted table was bisected against the budget on the test frames themselves, which is the one thing Section 3.4 says a decoder cannot do. The table immediately above repeats the measurement with

the table a deployment would ship: β bisected once per quality index on the 512 held-out Open Images validation frames, then applied to the CTC frames without being touched again. Checkpoint, head, frames and budget are identical on both sides, so what separates the two columns is where β came from.

At q0 the two agree, 146 against 146 and 23.6% against 23.6%, which is the same measurement to a hundredth of a point. At the rates in between, the held-out β misses the budget on the high side: it delivers 0.109 dB at q32 where 0.1 dB was asked for, and 3 of the 4 rates it covers overshoot. Distortion and saving move together, so those rows also report more saving, and reading one column straight against the other would credit the router with compute it bought using quality the budget did not allow. The last column takes that back out, re-bisecting on the test set to the quality the held-out β actually delivered: at equal delivered quality the two allocations agree to within 0.0 points. What moves between the two sets is the decibel a given β delivers.

The highest rate fails harder than that. On the calibration frames the floor, the distortion tiling costs with every tile already at the deepest exit, is 0.104 dB at q63. That is above the budget, so no allocation on that set meets 0.1 dB and the bisection has nothing to return. The shipped table has a hole where the highest rate should be, and a decoder holding it falls back to the deepest allocation, which is our own full-depth path and costs slightly more than the release, so it saves nothing. The test frames, whose floor at that rate is 0.072 dB, would have supported 10.7 points. What fails there is the calibration set rather than the router: a budget written as an absolute decibel sits a different distance above the floor on 512px photographs than on 1080p video, and at the highest rate it sits below it.

We keep the test-bisected figures as B's headline, since the comparison against A is built on them, and the held-out table is the correction a reader should apply. Calibrating on frames whose tiling penalty matches the ones the decoder will meet is the fix, and for a video decoder that means calibrating on video.

Retraining with the mask fixed raises agreement and does not simply raise the saving. The head above was trained against a suppression term sitting above its own logits (below). We retrained it with the mask fixed, same recipe and same λ . Held-out agreement rises from 0.718 to 0.808, and the deployed saving moves by +3.1 points at q0 and -3.4 at q63. We find the retrained head ahead where the required β is small and behind where it is large, which is the same axis the gap runs along. That is the second time in this paper that agreement with the oracle has moved opposite to the quantity that matters. We keep the original head for every other measurement so the comparisons stay on one system, and we read the retrain as evidence that the mask cost the head real capacity, and that agreement is not the objective.

An earlier version of this measurement put the gap at 1.7 points at q0, rising to 13.3 at q63. The exit mask was at fault. It suppresses exits below the split depth by assigning -1e4, the head's own logits had drifted to that scale, and the suppressed entries were therefore the largest in every row. A large share of every tile went to the cheapest exit for a reason unrelated to its content. The mask is now negative infinity. Fixing it costs 3.5 points at q0, where the accident happened to agree with the oracle, and buys 9.4 at q63, where it did not.

5.6 A router with no parameters

Before a 144 K head is worth its 0.163% of the decode, it has to beat what the decoder already knows. The entropy model has produced one number per tile before the trunk runs, and at no cost: how many bits that tile's latents took.

We turn it into a routing rule with no learned parameters. We model the per-tile distortion as rank-1 in the log domain

$$\log D(t, k) \approx a \log b(t) + c + \log \varphi_k \quad (7)$$

where b is the tile's bit count normalised by the frame mean and ϕ is a K-vector saying what each exit costs on an average tile. We fit (α, c, ϕ) by least squares, leave-one-sequence-out, so that no sequence contributes to the profile that routes it. The same Lagrangian the oracle runs is then run on the surrogate. Nothing is signalled and nothing is trained; the arithmetic is one scalar per tile.

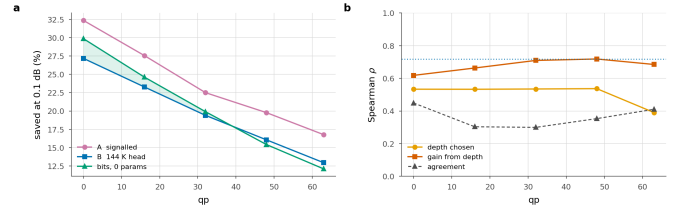


Figure 23. The calibrated bit rule. **a** Saving at 0.1 dB; shaded where the calibrated bit rule beats the trained head. **b** What a tile's bit count is correlated with, against how often the rule agrees with the oracle outright; dotted is the head's own held-out agreement.

q	rate-rank	B router	A signalled	ρ depth	ρ spread
0	27.1	23.6	29.2	+0.54	+0.61
16	22.0	20.3	25.0	+0.52	+0.65
32	17.7	16.8	20.4	+0.53	+0.71
48	14.2	13.6	17.9	+0.53	+0.72
63	11.6	10.7	15.2	+0.39	+0.68

Table 14. The calibrated bit rule, 0.1 dB, same test set; it is the "rate-rank" column. Bold where the rule beats the trained head. ρ are Spearman correlations between a tile's bit count and, respectively, the depth the oracle assigns it and the distortion it stands to gain from that depth.

It beats the trained head at every rate. The zero-learned-parameter rule is ahead of the 144 K router at all 5 measured rates, by margins that run from half a point at q48 to 3.4 points at q0. A head trained on this decoder against this oracle therefore returns nothing over a rule with no parameters at all, and it carries 0.163% of the decode that the calibrated bit rule does not.

Why it works is not that it agrees with the oracle. It agrees on 0.28–0.45 of tiles, well below the router's 0.718, and still saves more at every rate. Agreement weighs a disagreement on a tile where two exits are within a hair of each other exactly as heavily as one where the choice is most of the frame's error, and most tiles are the former. The ordering is what the rule gets right. Bits correlate with the *spread* across the ladder, that is, with how much a tile stands to gain from depth, at $\rho_{\text{spread}} = 0.61$ –0.72 at every rate.

At a looser budget it stops being a baseline and becomes the answer.

At 0.3 dB the calibrated bit rule matches the oracle exactly at the three lowest rates, where both reach the same architectural ceiling, comes within 0.2 points of it at q48, and gives up 34.3% against the oracle's 35.0% at q63. That is ahead of the trained head at every rate, by up to 7.4 points. The head is charged for itself and the rule is not. At a loose budget the head's ordering also has less left to contribute, because once the budget saturates the ladder there is little ordering left to get right, and the rule gets it. At 0.5 dB it reaches the ceiling at every rate and its assignment is *identical* to the oracle's on every tile.

What it cannot do is see anything beyond that ordering. A rank-1 model in the level assigns every tile the same relative profile over exits, so b only decides where on the ladder a tile falls, never the shape of its trade-off. That is the ceiling this baseline sits at, and it is the part a learned head should be earning its parameters on. Ours does not earn them at any rate we measured.

q	A signalled	B router	bits, 0 params
0	34.0	31.6	30.9
16	27.7	25.1	27.1
32	22.3	16.5	22.4
48	19.8	13.3	19.1
63	17.2	9.0	16.2

Table 15. The same comparison on a second training run (BEST), 0.1 dB, same test set. Bold where the calibrated bit rule beats that run’s own trained head.

It replicates on a second training run. BEST is a separate recipe at a different epoch, with its own router trained the same way. There the calibrated bit rule beats the trained head at 4 of 5 rates, by up to 7.2 points, and comes within 3.1 points of the *oracle* everywhere. The margins there are wider than here, and the one rate it concedes is the only rate on either checkpoint where the head finishes ahead, by under a point. We do not read that as showing a learned head cannot be worth its parameters, only that neither of our training runs produced one that is. The calibrated bit rule stays close to the oracle on both.

Per-block bit allocation is a standard quantity in learned compression, where it is something to *choose*; block-level rate control sets it so that complex regions get more bits [42]. We read the same number in the other direction, after the fact and at the decoder, as a statement about how hard a region was. It costs nothing because someone else has already paid for it.

q	bits	0.1	0.25	0.5	0.75	0.9	head
0	29.8	27.9	27.9	27.9	28.0	27.9	28.0
16	24.1	23.3	23.0	23.0	22.8	22.7	22.8
32	19.6	19.5	19.4	19.3	19.3	19.3	19.3
48	14.9	16.9	17.0	16.8	16.7	16.7	16.6
63	12.4	13.2	12.4	11.7	11.6	11.4	11.4

Table 16. Blending the two decoder-side signals at 0.1 dB, both normalised to unit mean, weight w from the calibrated bit rule to the head’s ordering. Bold is the best per rate.

Are the two signals complementary? Barely. We normalise both surrogates to unit mean and blend them with one weight w , where $w=0$ is the calibrated bit rule and $w=1$ the head’s ordering. The best blend is $w=0$ at the three lowest rates and a small head weight at the two highest, worth +2.1 points at q48 and +0.9 at q63. The head reads the entropy model’s scales, so it already has most of what the bit count carries; what it adds is confined to q48 and q63.

A learned component should be measured against the free alternative, and it rarely is. In adaptive inference the usual controls are a uniform allocation and a random one. Both are much weaker than a decoder-side signal that is already lying around.

5.7 Signalling only what the router gets wrong

The encoder knows tile by tile where the router will be wrong, because it can run that router itself (Section 3.1), and nothing obliges it to correct every one of them. That is the room configuration C works in.

We choose the ρN overridden tiles by Lagrangian regret, $\Delta(t) = L(t, \hat{k}) - L(t, k^*)$ with $L(t, k) = D(t, k) + \lambda c_k$ the objective of the Lagrangian in Section 3.5, so $\Delta(t)$ is exactly what that objective loses on tile t by staying silent. We hold β at B’s value and bisect λ over the overrides to land back on the budget. The map costs an entropy-coded mask, $N \cdot H(\rho)$ bits with H the binary entropy, plus 3 per override.

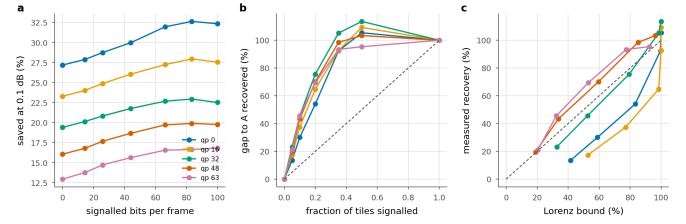


Figure 24. Partial signalling. **a**, saving against the bits spent on the map; each line runs from B on the left to A on the right. **b**, the same normalised by the A-to-B gap; dashed is proportional recovery. **c**, against the fixed- λ Lorenz bound; points above the dashed line are allocations one multiplier cannot reach.

q	B	5%	10%	20%	50%	A
bits/frame	0	14	26	44	83	100
0	24.4	25.1	26.0	27.3	30.1	29.9
16	20.9	21.7	22.5	23.7	25.7	25.6
32	17.4	18.1	18.8	19.8	21.1	20.9
48	14.1	14.9	15.8	16.8	18.3	18.3
63	11.1	11.9	12.9	13.9	15.3	15.6

Table 17. Configuration C at 0.1 dB. Columns are the fraction of tiles the encoder overrides; the two end columns reproduce B and A to within 0.1 points, which is the check that the interpolation is real.

The two ends check out. At $\rho=0$ the measurement reproduces configuration B to the second decimal at every rate, and at $\rho=1$ it reproduces A. Neither end is imposed; both fall out of the same code path run against independently measured files, so the columns between them are measuring something real.

Most of the gap is cheap. Overriding a tenth of the tiles for 26 bits per frame recovers 29–39% of the gap, and a fifth recovers 53–62%. Recovery is concave everywhere, so the first bits spent are the most useful ones.

And a partial map can beat a complete one. At 4 of 5 rates, signalling half the tiles for 83 bits saves *more* than signalling all of them, by up to 0.21 points, at the same distortion. Both land within the bisection’s 5e-4 dB tolerance of the budget, and the local exchange rate is about 48 saving points per decibel, so the tolerance is worth 0.02 points against a margin ten to twenty times that. The margin is therefore real, and it does not come from a better predictor. The hybrid has two multipliers where A has one: the oracle’s λ governs the overridden tiles and the router’s fixed β the rest. A two-multiplier allocation reaches points on the distortion–compute plane that a single Lagrangian sweep cannot, for the same reason the sweep misses interior Pareto points at all. Configuration A is optimal among allocations reachable by one multiplier, and that is a smaller set than the word suggests.

Why the recovery is concave, and what bounds it. At a fixed λ the objective is separable, so overriding a set removes *exactly* the sum of that set’s regrets. The recovery of the *objective* is then the Lorenz curve of the per-tile regret distribution, and its curvature is that distribution’s Gini coefficient, 0.49–0.73 across rates. What the table reports is saving at fixed distortion, not the objective, and returning to the budget means re-bisecting λ . That re-bisection is also what lets the two-multiplier allocations above escape the bound.

At the loose budget it matters where the gap is. At 0.3 dB the three lowest rates are saturated, and there is nothing for a partial map to buy. At \$q48\$ and \$q63\$, where the gap is 8.1 points, half the map recovers 68–91% of it. The allocation follows the same rule here as everywhere else, which is to spend the bits where the ladder still has somewhere to go.

A better predictor concentrates its regret, which is what the Gini was for. We rebuild C on the retrained head of Section 5.5. The Gini of the per-tile regret rises at 4 of 5 rates, to 0.68–0.91. Its mistakes are rarer and larger. The same table shows the consequence. C converges on A

faster from a better base and has less left to recover, and the half-beats-all effect disappears at q0, because a base close to A leaves little for the extra multiplier to exploit. The Lorenz reading is therefore worth having as a diagnostic and not as a decoration; it reports what *kind* of predictor one has as well as how good it is.

Which predictor should C be built on? Either will do, and the answer follows the same split as Section 5.6. Running the identical override rule over the calibrated bit rule instead of the head is worth up to 3.0 points at the lowest rate and costs up to 0.2 at the highest. Both converge on A at $\rho=1$ to the second decimal at every rate, which is the third independent check that the two code paths agree. The best single operating point we measured is the calibrated bit rule with half the map signalled, at 30.5% at q0. That is 0.61 points *above* full signalling, with no learned component anywhere in the decoder.

Configuration C is what we would ship where a small map is tolerable and a large one is not. It also reframes the A–B gap. What the gap measures is the price of *silence*, charged tile by tile, rather than the price of prediction.

5.8 Does the map have to be recomputed?

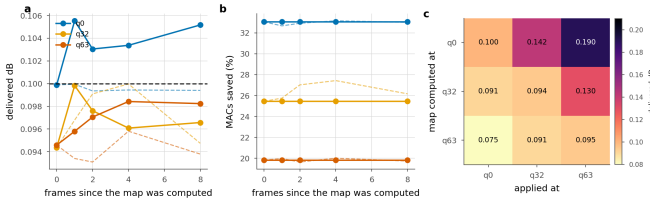


Figure 25. Reusing an exit map. Solid is the transferred map, dashed the one recomputed in place. **a, b:** reuse across frames of the same sequence. **c:** reuse across quality index; the entry is the delivered distortion against a 0.1 dB budget.

How much A’s extra decode at the encoder matters depends on how often the map has to be recomputed, and we have not seen that measured anywhere.

Across frames. Reuse is close to free. We take the map found on frame 0 and apply it eight frames later; at q0 the delivered distortion rises by 0.005 dB, five percent of the budget, and the saving does not move at all: 33.05% transferred against 32.66–33.15% recomputed. The allocation is a property of where the content is hard, and that moves slowly. A codec would search once per group of pictures instead of once per frame, and divide the encoder cost by the group length.

Across rate. Here the map does have to be recomputed, and the direction of the reuse decides how badly. Found at q0 and applied at q63, it delivers 0.190 dB against a 0.1 dB budget, claiming the low-rate saving of 33.05% while spending nearly twice the quality it is allowed. The reverse is safe but wasteful: the q63 map applied at q0 delivers 0.075 dB and only 19.8% where 33.1% was available. Shallow exits are cheap in quality at low rate and expensive at high rate, so a map is calibrated to the rate it was found at, and reusing one upward breaks the quality guarantee without anything in the decode reporting that it has. In our reuse probe, then, an encoder can search once per rate and reuse that search across the frames we tested. How far that carries beyond the offsets and sequences probed here we have not measured.

5.9 Complexity and wall-clock

q	Released	Routed	Saved (%)	
	(ms)	sorted (ms)	measured	MACs
0	111	78	29.1	35.3
32	111	86	22.4	28.0
63	111	94	15.6	20.1

Table 18. Wall-clock, 1080p, median of 40 interleaved iterations on one NVIDIA RTX A6000, 300 W board limit, PyTorch 2.6.0 and CUDA 12.4, single precision, at the 0.1 dB operating point. “MACs” is what the arithmetic predicts; “measured” is the sorted per-tile loop.

A saving in multiply-accumulates is not a saving in time. Tiling costs 3.6% before anything exits early, measured as the deepest-exit tiled decode against the released full-frame one at the same arithmetic and the same weights. On top of that, the routed decode realises 27.9% at q0 where its operations predict 35.3%. Four fifths of the predicted saving arrives. The missing fifth is the tiling overhead together with the bookkeeping of a shrinking active set, and a MAC count sees neither. The shortfall scales with the saving instead of sitting at a fixed offset: at q63 we measure 15.6% realised against 20.1% predicted.

Seconds and joules. Multiply-accumulates are the right unit to optimise, because they do not depend on the machine, but they are not the unit a deployment cares about. We measured both on the padded 1080p frame, sampling board power from the driver at 100 ms and running each condition for a fixed wall time rather than a fixed iteration count, because the sensor updates too slowly for ten iterations of a 100 ms decode to mean anything. Idle power was measured before and after each condition so that a neighbouring process waking up would appear as drift rather than as a saving.

Energy follows time, not arithmetic. At q0, q32 and q63 the routed decode removes 33.0, 22.9 and 17.8% of the arithmetic; it removes 29.1, 19.1 and 14.1% of the wall clock and 29.6, 19.3 and 14.2% of the joules per frame. Board power barely moves, 298 W against 296 W at q0, because the later groups run on a shrinking set of tiles and a partly idle GPU still draws most of its static power. What early exit buys is a shorter decode, not a cooler one. Frame rate at 1080p goes from 9.0 to 11.2, and peak memory rises by 13.3% at every resolution we could measure: tiling turns one feature map into a batch of small ones, so the peak is set by the widest point of the ladder rather than by its deepest.

Sorting the tiles once by exit depth recovers part of it. The obvious implementation of a shrinking active set keeps a boolean mask, gathers the survivors and scatters the finished at every group boundary, and each of those boundaries forces a device-to-host synchronisation. Order the tiles by descending exit depth and “still active at group g” becomes a contiguous prefix: each group is then a slice instead of a gather, and one cumulative count supplies every boundary, so no per-group mask is built at all. The finished tiles come back through the inverse permutation in a single scatter. The arithmetic is unchanged and the output is bit-identical on GPU. At q0 the saving goes from 27.9% to 29.1%, a gain of 1.2 points, or a fifth of the shortfall, for a change that touches no arithmetic.

The same gap appears in our own accounting. The router is charged at its share of the decoder’s multiply-accumulates, 0.163% (Section 3.1). We also timed the head directly, on the padded 2048×1280 frame the decoder actually sees, interleaved against a deepest-exit decode so that a co-tenant’s load falls on both equally. It costs 0.50%, which is 3.0× what its arithmetic predicts, and for the same reason as everything else in this section: a 144 K-parameter head is launch overhead, and a MAC count cannot see a launch. Charged at measured time instead of at operations, every B number in this paper would fall by a further 0.33 points. We leave them charged at MACs because that is the convention the rest of the literature reports in, and we record the correction here so that it does not sit unstated.

A paper that quotes only operations would report 35.3% where the same code, run as written, delivers 29.1%. That is why we give the shortfall as well. The error runs one way: operations are an *optimistic* bound on this

method, and the optimism grows with how much of the frame exits early.

5.10 The right ladder depends on the budget

Ladder	Config	Ceiling	0.1 dB	0.2 dB	0.3 dB	0.5 dB
RECIPE512	K6 j2 256px	38.3	21.5	33.7	37.4	38.3
BEST	K6 j2 256px	38.3	24.2†	–	38.5†	41.3†
BEST128	K6 j2 128px	38.3	19.3†	–	36.6†	40.6†
FINE12	K12 j4 128px	49.5	19.0*†	–	42.0†	48.6†

Table 19. Ladder settings, mean saving (%) over the five rates, on each run’s own latest checkpoint. Ceilings are $100(1-c_j)$ for that ladder, corrected by the offset the hook count shows. * some rate did not reach the budget, so the mean is over the rest. † the saving is from the arithmetic model rather than counted off the decode, and is therefore 0.4 to 0.8 points optimistic; only RECIPE512 has been re-measured with hooks at every budget.

Section 5.4 argued that once a budget saturates a ladder, the only way to spend more is an exit that does not exist. The table measures that. A finer ladder ($K=12$, $j=4$) has a higher ceiling than the shipped one’s 38.3%, and the two orderings cross somewhere between 0.1 and 0.3 dB. At 0.1 dB the coarse ladder wins, and the fine one cannot even reach the budget at the highest rate. Splitting later ($j=4$ against $j=2$) puts twice as many blocks in the per-tile section, which by the power law of Section 4 raises the floor. At 0.5 dB the fine ladder wins by 6.7 points, 48.6% against 38.3%, because the coarse one has been pinned at its ceiling since 0.3 dB and has nothing left to spend.

So the ladder is a choice made at the operating point, not a hyperparameter tuned once and fixed. The band of Section 5.4 tells you which side of the crossover a given budget sits on.

6. Limitations

The seam is reduced, and the exact remedy is not usable as it stands. The halo exchange removes 47–91% of the floor and is bit-exact at uniform depth (Section 4.4). It also destroys the routed saving, because routing puts neighbouring tiles at different depths. We do not know whether a decoder trained with the exchange can recover both at once, and that is the experiment we would run next. Until someone runs it, the floor is a cost this method pays.

Intra frames only. This is the image path of a video codec. Extending the ladder to inter frames raises a question we do not answer here, because an exit map propagates through the reference chain and a shallow tile in one frame is a worse reference for the next one.

Training is not converged. At every checkpoint we have measured more than once, the saving was still going up. We therefore read the numbers reported here as a lower bound on what a converged run would give.

A single decoder. All results are on DCVC-UF’s intra decoder. Nothing in the method looks specific to it, since the ladder needs only a residual trunk with a shared head, but we have not measured a second decoder to check.

7. Conclusion

The intra decoder of DCVC-UF can be run at a depth chosen per tile from what that tile contains. At a 0.1 dB budget this removes 29.2% to 15.2% of its multiply-accumulates for a BD-Rate cost of 1.03%, with no change to the encoder and none to the coded payload.

Three lessons we would carry to any spatially adaptive decoder, and none of them is about early exit.

Tiling is the dominant cost and it is governed by depth. All of that cost traces back to a single 3×3 that is 0.29% of the arithmetic, and the penalty grows as the square of how many such convolutions run per tile. The affected-area fraction usually quoted alongside it predicts that penalty badly.

The exact remedy is in tension with the thing it enables. Giving each convolution its real neighbour is bit-identical at uniform depth, and under routing it destroys the allocation, because routing is the deliberate violation of the condition that makes it exact. We expect any method that combines spatial adaptivity with tiled inference to walk into this.

A distortion budget is only a control variable inside a measurable band. Below the floor the budget admits nothing at all, and above saturation more of it buys nothing. A saving quoted without saying where in that band it sits has left out the part of the result a reader most needs.

We would attach three cautions to the measurements themselves. A saving in operations is an optimistic bound on a saving in time, and the optimism scales with the saving. A learned router should be compared against a free one. Routing on the bits already spent per tile needs no parameters and no training, it adds nothing to the stream, and it beats our trained head at every rate we measured, while agreeing with the Lagrangian oracle on fewer tiles than the head does. We read that as a warning about the metric quite as much as about the head. Finally, a timing harness will report numbers whether or not it is timing the right device. Ours timed the wrong one for months, and what gave it away was a decoder that appeared to slow down with the quality index.

References

- [1] S. Wang et al. Adaptive patch exiting for scalable single image super-resolution. ECCV, 2022.
- [2] J. Ballé et al. Variational image compression with a scale hyperprior. ICLR, 2018.
- [3] G. Huang et al. Multi-scale dense networks for resource efficient image classification. ICLR, 2018.
- [4] G. Bjøntegaard. Calculation of average PSNR differences between RD curves. VCEG-M33, 2001.
- [5] B. Bross et al. Overview of the Versatile Video Coding standard. IEEE TCSVT, 2021.
- [6] Z. Cheng et al. Learned image compression with discretized Gaussian mixture likelihoods. CVPR, 2020.
- [7] W. Fedus et al. Switch Transformers. JMLR, 2022.
- [8] G. Huang et al. Glance and Focus networks for dynamic visual recognition. IEEE TPAMI, 2023.
- [9] G. J. Sullivan et al. Overview of the High Efficiency Video Coding standard. IEEE TCSVT, 2012.
- [10] E. Jang et al. Categorical reparameterization with Gumbel-softmax. ICLR, 2017.
- [11] T. Kaseva et al. Per-channel autoregressive linear prediction padding in tiled CNN processing of 2D spatial data. arXiv:2502.12300, 2025.
- [12] X. Kong et al. ClassSR: a general framework to accelerate super-resolution networks by data characteristic. CVPR, 2021.
- [13] J. Li, B. Li, Y. Lu. Neural video compression with feature modulation. CVPR, 2024.
- [14] J. Li, B. Li, Y. Lu. Uniformly accelerated neural video codec with feature refinement. ACM MM, 2024.
- [15] Y. Kaya et al. Shallow-deep networks: understanding and mitigating network overthinking. ICML, 2019.
- [16] L. Wang et al. Auxiliary-loss-free load balancing strategy for mixture-of-experts. arXiv:2408.15664, 2024.
- [17] M. Phuong, C. H. Lampert. Distillation-based training for multi-exit architectures. ICCV, 2019.
- [18] S. Scardapane et al. Why should we add early exits to neural networks? Cognitive Computation, 2020.
- [19] W. Sun et al. Multi-exit self-distillation with appropriate teachers. FITEE, 2024.
- [20] S. Teerapittayanon et al. BranchyNet: fast inference via early exiting from deep neural networks. ICPR, 2016.
- [21] B. Bross et al. Versatile Video Coding. IEEE TCSVT, 2021.
- [22] F. Yang et al. Slimmable compressive autoencoders for practical neural image compression. CVPR, 2021.
- [23] M. A. Yilmaz et al. Slimmable video codec. CVPRW, 2022.
- [24] A. Kuznetsova et al. The Open Images Dataset V4. IJCV, 2020.
- [25] A. Mercat et al. UVG dataset: 50/120fps 4K sequences for video codec analysis. ACM MMSys, 2020.
- [26] H. Wang et al. MCL-JCV: a JND-based H.264/AVC video quality assessment dataset. ICIP, 2016.

- [27] T. Blard et al. Spatial competition for low-complexity learned image compression. *arXiv:2605.13243*, 2026.
- [28] Y. Shoham, A. Gersho. Efficient bit allocation for an arbitrary set of quantizers. *IEEE TASSP*, 1988.
- [29] A. Ortega, K. Ramchandran. Rate-distortion methods for image and video compression. *IEEE SPM*, 1998.
- [30] Adaptive test-time compute allocation for reasoning LLMs via constrained policy optimization. *arXiv:2604.14853*, 2026.
- [31] H. A. Alhaija et al. Local padding in patch-based GANs for seamless infinite-sized texture synthesis. *arXiv:2309.02340*, 2023.
- [32] C. Innamorati et al. Overlap training to mitigate inconsistencies caused by image tiling in CNNs. *arXiv:1812.02203*, 2018.
- [33] Z. Jia et al. Towards practical real-time neural video compression. *CVPR*, 2025.
- [34] B. A. Hasan et al. Adaptive inference: theoretical limits and unexplored opportunities. *arXiv:2402.04359*, 2024.
- [35] Y. Gao et al. Exploring the rate-distortion-complexity optimization in neural image compression. *CVIU*, 2024.
- [36] Y.-H. Ho et al. On the rate-distortion-complexity trade-offs of neural video coding. *arXiv:2410.03898*, 2024.
- [37] C. Zhang, W. Gao. Learned rate control for frame-level adaptive neural video compression via dynamic neural network. *arXiv:2508.20709*, 2025.
- [38] Y. Geifman, R. El-Yaniv. Selective classification for deep neural networks. *NeurIPS*, 2017.
- [39] D. Madras et al. Predict responsibly: improving fairness and accuracy by learning to defer. *NeurIPS*, 2018.
- [40] H. Mozannar, D. Sontag. Consistent estimators for learning to defer to an expert. *ICML*, 2020.
- [41] G. DeSalvo et al. Budgeted multiple-expert deferral. *arXiv:2510.26706*, 2025.
- [42] M. Dong, M. Lu, and Z. Ma. Accelerating block-level rate control for learned image compression. *arXiv:2409.01009*, 2024.
- [43] Y. Li et al. Learned image compression with hierarchical progressive context modeling. *ICCV*, 2025.
- [44] D. He, Z. Yang, W. Peng, R. Ma, H. Qin, Y. Wang. ELIC: efficient learned image compression with unevenly grouped space-channel contextual adaptive coding. *CVPR*, 2022.
- [45] R. Zou, C. Song, Z. Zhang. The devil is in the details: window-based attention for image compression. *CVPR*, 2022.
- [46] J. Liu, H. Sun, J. Katto. Learned image compression with mixed transformer-CNN architectures. *CVPR*, 2023.
- [47] W. Jiang, R. Wang. MLIC++: linear complexity multi-reference entropy modeling for learned image compression. *ICML Neural Compression Workshop*, 2023.
- [48] H. Li, S. Li, W. Dai, C. Li, J. Zou, H. Xiong. Frequency-aware transformer for learned image compression. *ICLR*, 2024.
- [49] S. Qin et al. MambaVC: learned visual compression with selective state spaces. *arXiv:2405.15413*, 2024.
- [50] H. Fu, J. Liang, Z. Fang, J. Han, F. Liang, G. Zhang. WeConvenc: learned image compression with wavelet-domain convolution and entropy model. *ECCV*, 2024.
- [51] D. Minnen, S. Singh. Channel-wise autoregressive entropy models for learned image compression. *ICIP*, 2020.
- [52] J. Li, B. Li, Y. Lu. Neural video compression with diverse contexts. *CVPR*, 2023.

Where to Stop:

Tile-Adaptive Early Exit in a Learned Image Decoder

Supplementary Material

A. Test settings, implementation and reproducibility

A.1. What this section covers

This section is the recipe and the protocol: the released codec the work starts from and the exact file it is, the test set, the conventions every decibel is measured under, every module with its shape and its parameter count, the two algorithms a reader would have to re-implement, the training configuration with a column naming where each choice was measured, the hardware, the wall clock, the seeds and what varies between two runs of one command. The sections after it account for compute (B), derive the allocation rule and its bounds (C), report the complete results (D), describe the quality budget and its working range (E) and measure the decoder-side router (F). Numbers are not repeated across sections; where a quantity belongs to a later one, this section names it and stops. One fact governs the rest and is stated here rather than buried: the checkpoint every headline number comes from has seen the training set exactly once, and A.10 reports how much the numbers move with a second pass.

A.2. Test settings: the released codec, and what is frozen

There is no traditional-codec invocation in this work and none is needed. The axis measured here is decoder-side compute against a fixed learned decoder, on a bitstream that decoder produced, so the anchor is a checkpoint rather than a command line, and the released codec's own standing against VTM is published with it. That checkpoint is identified by content and not by path, because the released file is a bare state dict carrying no epoch, no optimiser and no step, so a path plus a hash is the whole of its provenance.

The anchor is `~/DCVC/checkpoints/cvpr2026_image.pth.tar`, sha256 `b3b900de23f30e4fc437010ddffe6a5413a56d6cfd97fb6235adbcd2b6973302`, recorded with that reading in `results/dmc_ld_recon_audit.json`.

The codec is imported rather than reimplemented: `flexuf/model.py` sets `DCVC_ROOT` to the `DCVC` checkout and imports `src.models.image_model.DMCI` from it, so the entropy model, the hyperprior, the quantisation-step tables and the 64 quality indices are the released code running unmodified. Our decoder is those weights rearranged: an opening upsampling block, 12 `DepthConvBlocks` at $C = 384$ channels and a head that pixel-shuffles back to RGB, with the 12 blocks grouped into $K = 6$ exits of $b = 2$ blocks each and the first $j = 2$ groups running once over the whole frame before the decode splits into tiles. Exits 0 and 1 therefore lie below the split and cannot be selected; the four reachable exits are 2 to 5.

The encoder is frozen for the whole of training, and that is asserted rather than assumed: `scripts/signalled_curve.py` compares our encoder state dict against the released one tensor by tensor and requires the largest absolute difference to be exactly zero before a single frame is measured. A trained decoder therefore consumes byte for byte the stream the released encoder produces, the bit rate is identical by construction, and the whole of the measured difference is decoder-side.

A.3. The test set, and what tiling does to it

class	sequences	resolution	padded to	tiles	added px
UVG	7	1920x1080	2048x1280	40	26.4%
MCL-JCV	30	1920x1080	2048x1280	40	26.4%
HEVC B	5	1920x1080	2048x1280	40	26.4%
HEVC E	3	1280x720	1280x768	15	6.7%
HEVC C	4	832x480	1024x512	8	31.3%
HEVC D	4	416x240	512x256	2	31.3%
all	53				

Table 1. The test set, and what tiling does to each resolution. Take from it that granularity is not constant across the set: a 1080p frame carries 40 tiles for the allocation to work with and a 416x240 frame carries two, and the small classes also pay the most padding. That single fact explains most of the per-class spread reported in section D.

Sources: `results/per_class_RECIPES12.json` for the class membership and tile counts, `results/tile_definition.json` for the padded sizes; both on `runs/RECIPES12/ckpt_PAPER.pth.tar`. The sequences are UVG [25], MCL-JCV [26] and HEVC classes B, C, D and E [9]; the full list of 53 names is recorded in the measured field of `results/signalled_RECIPES12_ctc53.json`, with `not_measured` empty.

Frames are the leading frames of each sequence, taken consecutively. The headline file uses two frames on each of the 53 sequences of Table 1, so 106 frames; the routed, rate-rank and partial signalling tables use one. Every results file records `frames_per_seq` and every table in this supplement states it. No frame is dropped and no sequence excluded: the `not_measured` field of every curve file is empty.

A.4. The measurement conventions

- Colour.** Sources are YUV 4:2:0, 8 bit, converted to 4:4:4, divided by 255 and shifted by -0.5 for the network. PSNR is the released codec's own metric, $(6 \cdot Y + U + V)/8$, computed after converting the reconstruction back to 4:2:0 on the 0 to 255 scale. Measuring in 4:4:4 would compare against a chroma upsample of the source rather than the source, and would report a chroma PSNR no published DCVC-UF number could be compared with.
- Padding.** Frames are padded by replication, bottom and right only, to a whole number of tiles; `results/tile_definition.json` records the site as the RGB frame before the encoder and the mode as "replicate, bottom and right only: `F.pad(x, (0, pw, 0, ph))`". The stock codec aligns to 16, which is enough for the transform and not for a tile grid. Rate is charged on the true pixel count and PSNR computed after cropping back, so padding can cost and cannot flatter, and it does cost: a tile containing replicated rows gives up about twice the quality of one that does not.
- Decibels.** A budget is a loss against the released decoder on the same latent, averaged per frame and then over frames, which is the convention the released codec's own evaluation uses. Pooling all tiles into one mean squared error first is the other live convention and is not the same number; section E measures the gap at about a third of a 0.1 dB budget. No table mixes the two and every table says which it is on.
- A second metric.** MS-SSIM is reported beside PSNR at the operating point, computed as Wang et al. 2003, 5 scales, 11-tap Gaussian sigma 1.5, valid region, luma plane in 4:2:0 on 0..255. The implementation is self-checked at start-up: identical inputs score 1.000000 and a 3x3 box blur scores 0.7797, both recorded in `results/supp_opquality_PAPER.json`.

- **The map is billed.** The exit map is entropy coded from a per-frame histogram and its cost added to the rate before any saving is computed: 79 to 95 bits per frame at the 0.1 dB budget, at most 9.09e-05 bpp. Every row records map_bits and the bpp it added.
- **The reference.** The released decoder re-expressed in the same ladder and selected by K, so a K = 12 run cannot be quoted against the K = 6 remap.
- **Wall clock.** Every second reported in this work is one frame at a time on a shared card, timed the same way for both decoders. The protocol, what the timer wraps and what is deliberately outside it are stated in section B, where the seconds are reported.

The padding figure and the second metric are both from results/supp_opquality_PAPER.json on the pinned checkpoint, 53 sequences at the 0.1 dB operating point: mean per-tile penalty 0.260 dB over the 547 tiles containing replicated rows against 0.121 dB over the 1,218 that do not, at q0. The map-bit range is from results/signalised_RECIPES12_ctc53.json, field map_bits.

Two limits belong beside that list. Sweeping a multiplier reaches only the lower convex hull of the achievable set [28, 29], so a budget falling in a gap of the hull is met at the nearest reachable point; every row carries a reachability flag and a floor, and rows whose floor already exceeds the budget are reported as unreachable rather than dropped. And 0.1 dB is a reporting convention, not a perceptual threshold. Subjective work measures the smallest noticeable change in quantisation parameter or in a video quality metric, not in tenths of a decibel of PSNR, so nothing published licenses a claim that 0.1 dB is invisible; 0.3 dB and 0.5 dB are reported beside it so that the choice carries no weight.

A.5. The ladder, module by module

module	runs	in → out	params	share
upsample	frame	256 → 384, ×2	1,431,936	10.1
groups 0 to 1, stem	frame	384 → 384	4,154,880	29.2
groups 2 to 5, routable	tile	384 → 384	8,309,760	58.4
one DepthConvBlock	tile	384 → 384	1,038,720	7.3
adapters 0 to 3, FFN	tile, at its exit	384 → 1536 → 384	739,200	5.2
adapter 4, 1×1	tile, at its exit	384 → 384	147,840	1.0
seam repair, grid	after stitching	384 → 384	152,704	1.1
head, PixelShuffle 8	frame	384 → 192 → 3	335,232	2.4
router head (B)	tile	161 → 6	144,024	1.0
decoder, released			14,231,808	100.0
decoder, ours			3,257,344	+22.9
encoder, frozen	encode		27,956,246	196

Table 2. Where every module sits, what shape it is and what it weighs, with the last column as a percentage of the released decoder's own parameters; adapter rows are per adapter and group rows cover the whole run of groups. Take from it three things. The ladder adds 22.9% to the decoder's parameters, almost all of it in the four feed-forward adapters. Of that, 1,478,400 parameters can never run, because at $j = 2$ the decoder clamps every exit to at least 2 and adapters 0 and 1 are unreachable; they still take gradient from the distillation term, which taps every exit, and dropping them would forfeit the ability to re-run the same checkpoint at another split depth. And the frozen encoder is larger than the whole decoder, which is why an encoder-side search is the expensive half of this system and a decoder-side one is not.

Source: results/supp_module_shapes.json, derived by scripts/module_shapes.py from the state dict of runs/RECIPES12/ckpt_PAPER.pth.tar (md5 daa3760937ea0bd8) on the CPU. Parameter counts are tensor sizes read straight from the state dict. The same file also carries each module's multiply-accumulates per feature pixel, and the three aggregate shares that fall out of them, 8.16% for the upsample, 89.44% for the twelve-block trunk and 2.40% for the head, reproduce the forward-hook audit of results/mac_audit.json to four decimals. Section B prices the decode module by module and derives the per-exit cost; none of that is repeated here.

One initialisation detail behind the seam-repair row of Table 2 is load-bearing and is not visible in the arithmetic. The pass multiplies its correction by a learned gate indexed by position within a tile, a 32×32 array of scalars shared across all 384 channels, and that gate starts as a decaying function of the distance to the nearest tile edge. At step zero the module therefore already knows where the seams are and training refines

rather than discovers, while its output convolution is still zero, so the whole module is exactly the identity and the warm-start controls of A.9 still hold.

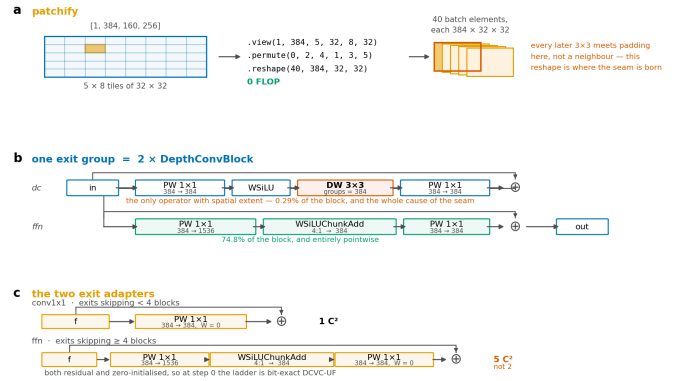


Figure 1. The three pieces a reader has to re-implement. **a**, patchify is a pure reshape and costs nothing, and it is where both the seam and every operation of saving come from. **b**, one exit group is two DepthConvBlocks, whose only operator with spatial extent is a single 3×3 depthwise worth 0.33% of the block (9C of the block's 7C² + 9C, results/mac_audit.json), so an early exit gives up almost purely pointwise capacity. **c**, the two adapters that replace it, both residual and both zero-initialised. Shapes are those of a 1080p frame padded to 2048×1280 and split into 40 tiles.

Two of the five adapters can never run: at $j = 2$ the decoder clamps every exit to at least 2, so adapters 0 and 1 are unreachable at inference. They still take gradient from the distillation term, which taps every exit, and they still occupy 1,478,400 of the 3,257,344 parameters we add. Dropping them is the obvious saving in decoder size and is not done here, because the split depth is a configuration knob and a checkpoint that has dropped them cannot be re-run at another j .

A.6. Tiles, and the exit clamp

patchify is a view, a permute and a reshape with no arithmetic at all: a feature map of shape [1, 384, 160, 256] becomes 40 batch elements of [384, 32, 32], as panel a of Figure 1 shows. Tile order is row major, which is what the exit map is indexed by and what flexuf/eval.py reproduces with the same permutation when it measures per-tile error. One tile is 256 px of RGB, 32 px of feature map and 16 px of latent, at 16 pixels per latent position. patchify asserts divisibility rather than padding: an unpadded 1080p frame gives a 40×40 feature map that is not divisible by 32, and the run stops instead of producing a ragged last row.

The clamp is the most consequential line in the decoder. forward() applies exit_map.clamp(min = j, max = K - 1) before the per-tile loop, so a map naming exit 0 at $j = 2$ runs group 2 and wears exit 2's adapter. Four separate places have to apply the same clamp and each was wrong at some point in this project. Getting any of them wrong is silent, which is why they are listed rather than described.

file and function	what it does
decoder.py, forward	clamps the map to [j, K-1] before the per-tile loop
cost.py, exit_costs	charges max(k, j) blocks, and bills the adapter the clamped exit wears rather than the one the map names
eval.py, tiled_exit_msos	fills the columns below j with the decode at exit j, so an argmin over all K lands on a reachable choice
head2.py, forward and oracle_ce_loss	masks the logits below the split depth with -inf, and clamps the oracle label to the same bound

Table 3. The exit clamp, and the four places that have to repeat it. Take from it the failure mode rather than the rule: with the clamp in the decoder but not in the cost model, a router that assigned the shallowest exit everywhere was billed 58.70% saved where the true figure is 42.9%, and that inflated number made the router appear to beat a bound it cannot beat. The mask in the router is -inf and not a large negative constant, because nothing in a cross-entropy penalises a common offset, the raw logits drifted to about -10^4 , and a -10^4 mask then became the largest entry in the row.

Source: the four files named in the first column, under flexuf/. The two percentages are the comment in flexuf/cost.py that records the measurement; no results file carries them, which A.17 repeats.

A.7. The deployed decode, and how training differs from it

```
input latent y, quantisation step q, exit map m
1 f = upsample(y)
2 for g = 0 ... j-1: f = groups[g](f)  frame, no borders
3 tiles, nh, nw = patchify(f, 32)  reshape, 0 MAC
4 m = clamp(m, j, K-1)  below j cannot run
5 active = all tiles; work = tiles
6 for g = j ... K-1:
7   work = groups[g](work)  borders meet padding
8   leaving = active tiles whose m equals g
9   out = adapters[g](work[leaving])  raw if g = K-1
10  if training: out = out · gate[leaving]
11  canvas[leaving] = out
12  drop leaving from work and from active  the saving
13 canvas = unpatchify(canvas, nh, nw)
14 canvas = canvas + G[r mod P, c mod P] · seam(canvas)
15 return pixel_shuffle(head(canvas · q), 8)
```

Table 4. The deployed routed decode, line for line. Take from it that line 12 is the entire saving: every group runs on fewer tiles than the last, and nothing else in the procedure is cheaper than a full decode. Line 10 is the only difference between training and inference, and it is numerically the identity.

Line 10 of Table 4 deserves the space. A router trained jointly with the decoder needs gradient to reach its logits and an argmax blocks it, so the gate is $p / p.\text{detach}()$ with p the selected exit's probability under a Gumbel-softmax relaxation: its value is exactly 1 and the reconstruction bit-identical to the deployed path, while its gradient is the straight-through estimator. At inference the gate is absent and the exit is a hard argmax or a transmitted symbol. That is the one place where the trained object and the shipped object are not the same computation.

A second execution path exists and is bit-identical. With sorted execution the tiles are ordered by depth with a stable argsort before line 6, so "still active at group g " becomes a contiguous prefix, every group is a slice rather than a masked gather, and the loop costs one host read instead of one synchronisation per group. Only the wall clock moves, by 1.2 points, and section B reports it.

A.8. The training data

Training uses Open Images [24], fetched from the CVDF public mirror by `scripts/fetch_openimages.sh` and prepared by `scripts/prepare_openimages.py` into the flat layout DCVC-UF's own ImageFolder expects. Of 384,795 files scanned, 380,126 survive a minimum-side filter of 512 px, 4,669 are dropped as too small and 0 as unreadable. The filter is not cosmetic: the loader zero-pads any image smaller than the current crop, and an image that is half black border teaches the decoder to reconstruct black borders. Every 400th entry of the sorted survivor list is then held out, giving 512 validation images and 379,614 training images with no overlap, deterministically, so every run and every control sees the same held-out set.

Each sample is a random crop at the schedule's patch size with a random horizontal flip, converted to YCbCr, scaled to [0,1] and shifted by -0.5. One quality index is drawn uniformly over the 64 levels per sample and the matching multiplier comes from the log-spaced table running 10 to 2048, so one set of weights covers the whole rate range rather than a point on it. The crop is 512 px, which is what the released recipe's final phase asks for and also the smallest crop carrying more than one tile: at 256 px tiles a 512 px crop holds 4 and a 256 px crop holds one, and a single tile has no border against a neighbour, so patched training at the smaller crop would train a configuration that cannot occur at inference. Evaluation uses a deterministic centre crop with no flip, because the

loader's random crop swung one model between 29.54 and 30.58 dB at q0 across two runs of one command.

A.9. The warm start, and the controls it asserts

`scripts/warmstart_from_release.py` performs the rearrangement, and it is a key remap and nothing else: the opening upsample takes `dec_1[0]`, group g block i takes `dec_1[1 + g·b + i]`, the head takes `dec_2`, and the adapters and the seam-repair gate are new and zero-initialised. The encoder, hyperprior, entropy model and quantisation-step tables are copied verbatim. The transfer moves 398 tensors and creates 28 adapters at initialisation. Two controls follow from zero initialisation and both are asserted at every build rather than assumed: the deepest exit reproduces the released decoder with a largest absolute difference of 0.0, and every untrained exit equals the released decoder truncated at that depth, also 0.0. A third lives in the checkpoint loader, which tolerates a missing or extra tensor only under the adapter, seam-repair, pad-coefficient and router prefixes and raises otherwise; both sides of a tolerated mismatch are zero-initialised identities, so dropping one is exact rather than approximate.

A.10. The training run

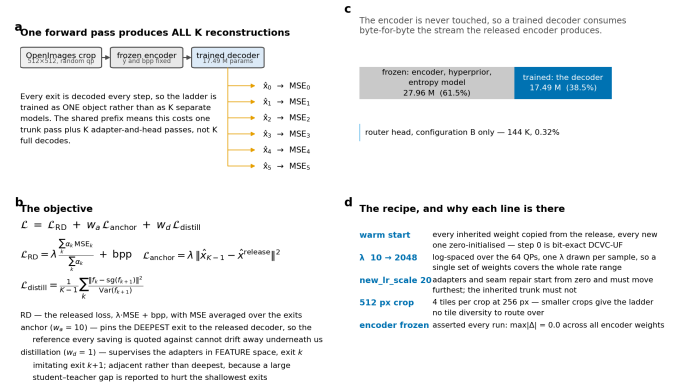


Figure 2. The training scheme. a, one forward pass produces all K reconstructions, so the ladder is trained as one object. b, the three loss terms. c, the encoder is never touched. d, why each line of the recipe is there. The figure is a schematic drawn from the model definition; the parameter counts it carries are the ones tabulated in A.5.

The recipe, laid out in Figure 2, is Microsoft's own, applied at the point in their schedule that matches what we inherit. Their schedule is a 105-entry table of learning rate and crop size indexed by epoch, and the released weights are the output of its last entry, so applying entry zero would kick a converged codec with the from-scratch learning rate. The run enters at offset 99, inside their final 512 px phase at a learning rate of 1×10^{-5} ; all 852 records in the run's log carry that rate and that crop, which is the check that the offset did what it was meant to. Because the offset lands in the phase the experiment needs, no crop-size override is passed at all, where every other warm-started run here raises the crop floor by hand.

One deviation from a plain fine-tune is deliberate. The adapters and the seam-repair gate start at zero and have to move furthest, while the inherited trunk must not move at all if the deepest exit is to stay the released decoder, and the cost of choosing wrongly between them is measured: at the from-scratch rate the trunk drifts 0.20 dB of anchor at q32 in a single epoch. The new modules therefore get their own parameter group at 20 times the schedule's rate, selected by name prefix, with the multiplier re-applied whenever the schedule sets the base rate.

The loss has three terms. The rate-distortion term is the released loss, a multiplier times mean squared error plus bits per pixel, with the error averaged over the exits under fixed weights. The anchor term pins the deepest exit to a frozen copy of the released decoder, because every saving in the paper is quoted against that decoder and a reference drifting

underneath the measurement makes the measurement meaningless. The distillation term supervises each adapter in feature space against the exit one step deeper, normalised by the teacher's own variance, following the adjacent-teacher form rather than the deepest-teacher form [17], since a large gap between student and teacher is reported to hurt the shallowest exits most. Gradients are clipped at a norm of 0.1 and a batch whose norm is not finite is dropped; across all 852 records the count of steps skipped for that reason is 0 and the median gradient norm is 0.273.

An epoch is 47,451 optimiser steps, which is 379,614 images at batch 8. The run logs every 200 steps and the median interval is 299.8 s, so a step costs about 1.5 s and an epoch about 19.8 hours. Training is fp32 throughout on a single NVIDIA RTX A6000 with eight data workers.

quality index	epoch 0	epoch 1	change	floor e0	floor e1
q0	29.90	30.84	+0.93	0.0326	0.0295
q16	25.62	27.06	+1.44	0.0429	0.0383
q32	20.93	21.97	+1.04	0.0528	0.0495
q48	18.33	19.92	+1.59	0.0599	0.0551
q63	15.64	17.20	+1.56	0.0659	0.0595
mean	22.08	23.40	+1.31		

Table 5. One more pass over the training set is worth 1.3 points on average and never less than 0.9. Saving at the 0.1 dB budget against the released decoder, in percent, on the pinned checkpoint and on the epoch-1 pin, measured with the identical protocol on the identical frames. The last two columns are the floor, the quality already given up before any tile exits early; it falls at every rate. The paper reports the left-hand column, which is therefore a lower bound on what this configuration reaches.

Source: results/signalled_RECIP512_ctc53.json (runs/RECIP512/ckpt_PAPER.pth.tar, epoch 0) and results/signalled_RECIP512_e1.json (runs/RECIP512/ckpt_PIN_e1.pth.tar, epoch 1), both 53 sequences.

The run had reached epoch 3, step 27,400 of 47,451, when this was written, against the 16 epochs its launcher was given. Table 5 therefore carries a second reading: no comparison in this paper between two runs at different epochs can be read as a comparison between two configurations, and where such a comparison appears it is labelled with both epochs. Nothing in results/ projects the trajectory to convergence and this section does not either; scripts/convergence.py fits a saturating curve with the ceiling pinned but writes only a figure, so its asymptote is not a number this document can cite.

A.11. Every hyperparameter, and where each was measured

knob	value	searched, and where
ladder geometry		
exits K, blocks per exit b	6, 2	signalled_BEST128 and signalled_FINE12, at K = 12
split depth j	2	supp_seam_vs_split, j from 0 to 6
RGB tile	256 px	ctc_seam_p128 against ctc_seam_p256; tileSize_adaptive
halo, latent and trunk	2, 0	no; the trunk halo was priced at 2.25× and rejected
adapter kind	scaled	adapter_ablation
adapter expansion	4	no; the block's own 4:1
seam repair	grid	supp_seam_vs_split
tile padding	replicate	padding_ablation: zeros, replicate, linear, arls
tile coupling	off	coupling_ablation
full-frame head	on	the per-tile arm exists to measure it
decoder training		
optimiser	AdamW	no; the released recipe's
schedule	105 entries, at offset 99	no; verified by scripts/verify_recipe.py
learning rate	1×10^{-5}	no; set by the schedule
multiplier, new modules	×20	no; set from one anchor-drift measurement

knob	value	searched, and where
crop	512×512	no; set by the schedule
batch, workers, precision	8, 8, fp32	no
gradient clip	0.1, non-finite dropped	no; released recipe's
multiplier range	10 to 2048, 64 indices	no; released
quality index	uniform, per sample	no
anchor weight	10	two settings run; 10 kept
distillation	1.0, adjacent	adjacent against deepest
auxiliary weight	1.0, constant	no
training path	deployed tiled decode	against the full-frame arm
encoder	frozen	no; asserted at each evaluation
seed	none set	no; see A.15
router head, configuration B		
steps, batch, crop	3,000, 6, 512	no
learning rate, decay	1×10^{-3} cosine, 1×10^{-4}	no
regret weight	1.0	no
seed	0	fixed, so the input ablation is comparable
live inputs	stem, latent, scales, qp	router_ablation, six variants
evaluation		
multiplier bisection	60 halvings on [0, 1]	no
outer correction	6 passes, tolerance 5e-4 dB	no
rate-rank bisection	40 halvings	no
router bisection	on beta, bracket [-2e4, 2e4]	no
timing	40 iterations after 20 of warm-up	protocol in section B
batch, when timing	1	supp_latency_batch, at 1, 2 and 4

Table 6. Every knob this work sets, its value, and whether it was searched. Take from it that almost nothing on the training side is tuned: the optimiser, the schedule, the clip, the crop and the multiplier range are the released recipe's, and the four settings that are ours exist to protect a property the measurement depends on rather than to improve a number. What was searched is the geometry, and the third column names the file in results/ that holds each comparison, so a reader can price the choice instead of taking it on trust.

Values come from scripts/launch_recipe512.sh, train_flexuf_image.py, scripts/train_router2.py and the four evaluation scripts; the third column names files in results/. The launcher's flags, without the two leading hyphens the shell needs, are: freeze_encoder train_patched epoch_offset 99 new_lr_scale 20 anchor_weight 10 seam_repair grid adapter_kind scaled distill_weight 1.0 distill_teacher adjacent lambdas 10 2048 batch_size 8 n 8 e 16 num_exits 6 split_depth 2 latent_patch 16 latent_halo 2 aux_weight 1.0 tile_pad replicate.

A.12. Finding the operating point

A budget is a quality target rather than a multiplier, so the multiplier has to be found, and it is found twice over: an inner bisection on a cheap table of per-tile distortions, and an outer correction against what a real decode delivers. The two levels exist because the table measures each tile with every other tile at the same exit, while a routed frame is mixed and a tile's border sees whatever depth its neighbour chose. The residual between them is small and nearly constant, so shifting the inner target by it converges in two or three passes, and each pass costs one decode per frame rather than one per bisection step.

input budget t in dB, frames F , per-tile cost vector C

```

1 for each frame: cache the [tiles, K] table  $M$  of per-tile error,
   one tiled decode per exit, and the released decoder's frame error  $R$ 
2 floor = trueDb(0) one decode per frame
3 if floor >  $t$ : record unreachable and stop
4 inner =  $t$ 
5 repeat at most 6 times:
6   lo, hi = 0, 1
7   repeat 60 times: on the table, no decode
8     mid = (lo + hi) / 2
9      $k$  = argmin over exits of  $M + \text{mid} \cdot C$ 
10    if tableDb( $k$ ) ≤ inner: lo = mid else hi = mid
11    lam = lo; d = trueDb(lam) one decode per frame
12    if |d - t| < 5e-4: stop
13    inner = inner + (t - d)
14  $k$  = argmin of  $M + \text{lam} \cdot C$ , clamped to  $j$ 
15 saving = 100 (1 - routed MACs / released MACs), by hooks
16 bpp = bpp + mapBits( $k$ ) / pixels

```

Table 7. Finding the operating point, as scripts/signalled_curve.py runs it. Take from it that the arithmetic model is used only inside the argmin on line 9, where a decode per candidate is impossible, and that the number reported on line 15 is counted off the forward pass that actually ran. Line 2 is measured on the deployed path rather than on the table, because the floor is exactly where the tiling penalty is largest.

Three details of Table 7 change what the numbers mean. A rate whose floor exceeds its budget is written out with a reachability flag rather than dropped, so a saturated or unreachable row is visible instead of absent. The saving is measured and not modelled: a forward hook on every convolution and linear layer counts the multiply-accumulates the routed decode performed and divides by the same count for the released decode on the same latent, so the ratio needs no calibration constant and a module skipped by an early exit costs nothing because its hook never fires. The gap between model and meter is reported in every row: at the 0.1 dB budget it runs from 0.43 to 0.69 points, always in the direction of the model flattering us, and it is the measured column the paper quotes.

Two saving columns exist and only one is quoted. One divides by our own deepest exit, the other by the released decoder, and the released decoder is the denominator every claim is stated against; it is exactly 1 in the units of the cost table, so the seam-repair tax stays inside the figure it is supposed to be net of. The same two-level scheme with different constants runs in scripts/raterank_curve.py, scripts/router_curve.py and scripts/hybrid_curve.py, whose constants Table 6 lists.

A.13. What the work cost to produce

item	cost	source
one decoder epoch	19.8 h on one A6000	train_log.jsonl
the pinned checkpoint	1 epoch	ckpt_PAPER records epoch 0
the run so far	3 epochs and 27,400 steps	train_log.jsonl
one router head	3,000 steps, 1.03 h	router_ablation_stem
encoder search, per frame	4.52 decodes	supp_encoder_cost
decoder search, per frame	none	the map is signalled

Table 8. What producing these numbers costs, priced rather than asserted. Take from it that the decoder is cheap to build and cheap to run, and that the whole of the allocation cost falls on the encoder, which already has the source and already runs an analysis pass. Sections B and F price the encoder-side search and the router heads against what they buy; this table only says what the work took.

Sources: results/router_ablation_stem.json for one head's wall_s field, results/supp_encoder_cost_PAPER.json for the search cost (pinned checkpoint, q63, 40 tiles, 15 iterations, one tiled decode at 114.9 ms). Decoder wall clock is from runs/RECIPE512/train_log.jsonl; no file in results/ records it.

A.14. The pinned checkpoint, and what every table rests on

Results are measured on runs/RECIPE512/ckpt_PAPER.pth.tar, md5 daa3760937ea0bd89b89c555c2eefa88, which records epoch 0 inside itself and is byte identical to that run's ckpt_epo0.pth.tar. The name exists because a filename is not a checkpoint here: a per-epoch watcher overwrites runs/*/*ckpt_eval.pth.tar as each epoch lands, so a results file naming that path names a file that no longer exists. scripts/make_paper_tables.py and scripts/check_paper.py both hold the pinned name and, among candidates for one quantity, prefer a file whose own ckpt field records it, falling back to the most recently written; hand-written order breaks ties and nothing else. That is a fix rather than a convenience, because an earlier hand-ordered list kept preferring a superseded file after the pinned one had been remeasured.

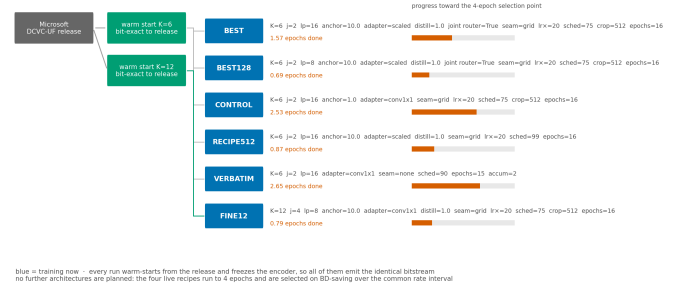


Figure 3. Lineage of the training runs. Every run warm-starts from the released decoder through one of two remaps, one per ladder size, and freezes the encoder, so all of them consume the identical bitstream. The flags shown are the ones that differ between runs. The progress bars are a snapshot taken on 2026-08-18 and the runs have advanced since; the current position of the run this paper reports is given in A.10.

results file	backs	checkpoint	seq×fr
signalled_RECIPE512_ctc53	main results, complexity	pinned, e0	53×1
saturation_RECIPE512_ctc53	operating range	pinned, e0	53 fr
static_RECIPE512_b01	static controls	pinned, e0	53
per_class_RECIPE512	per class	pinned, e0	53 fr
router_RECIPE512_b01	A against B	pinned, e0	53×1
router_RECIPE512_b03	A against B	pinned, e0	53×1
raterank_RECIPE512_b01	rate rank	pinned	53×1
raterank_RECIPE512_b03	rate rank	pinned	53×1
raterank_RECIPE512_b05	rate rank	R512/eval	53×1
hybrid_RECIPE512_b01_fixe d	partial signalling	pinned	53×1
hybrid_raterank_b01	rate rank, second run	pinned	53×1
hybrid_v3_b01	rate rank, second run	R512/eval	53×1
latency_RECIPE512_sorted	latency	R512/eval	-
router_latency	operating range	R512/eval	-
adapter_ablation	adapter ablation	R512/eval	16 fr
combined_RECIPE512_b01	blend	R512/eval	53×1
hybrid_RECIPE512_b03_fixe d	macros only	R512/eval	53×1
hybrid_lorenz_b01	macros only	R512/eval	53×1
coupling_ablation	macros only	R512/eval	16 fr
map_transfer	macros only	R512/eval	-
band_collapse	macros only	none	-
band_collapse_BEST	macros only	none	-
router_retrain_compare	macros only	none	-
signalled_BEST_ctc53	ladder configurations	BEST/eval, e1	53×1
signalled_BEST128_ctc53	ladder configurations	BEST128/step, e1	53×1
signalled_FINE12_ctc53	ladder configurations	FINE12/step, e1	53×1
ctc_seam_p256	padding, tile size	none	-
ctc_seam_p128	tile size	none	-

results file	backs	checkpoint	seq×fr
raterank_BEST_compare	rate rank, second run	none	-
bdrate	BD-rate	none	-
check_paper	claim check	none	-

Table 9. Provenance of every generated table in the main paper. These are the 31 results files scripts/make_paper_tables.py reads, and the checkpoint column is read from each file at build time rather than typed. Take from it that 10 of the 31 are on the pinned checkpoint and the rest are not, so a reader comparing two rows of two different tables should check this column first. R512 abbreviates the run RECIPE512, "eval" is the overwritten per-epoch file, and "none" means the file carries no checkpoint field at all, which is a defect in the script that wrote it.

Three groups in Table 9 deserve separate comment. The pinned group carries the headline saving, the operating range, the per-class breakdown, the static controls and both routed configurations, so the paper's central claim rests on one epoch of one run measured consistently. The eval group is measured on a file that has since been overwritten, so those numbers can be quoted but not reproduced; the latency table and the adapter ablation are in it. The group with no checkpoint field at all cannot even say which decoder it describes.

A.15. Seeds, variation, and the interval that does exist

No seed is set anywhere in the decoder's trainer. Four things vary between two runs of one command: the sampler is reshuffled every epoch from the global generator, the quality index is drawn per sample, the tiled training path draws a fresh random exit for every tile at every step, and cuDNN selects kernels by autotuning. The honest statement is that the recipe reproduces and the run does not, and this work has no repeated run of one configuration from which a training-side spread could be quoted. The router experiments are the exception and were built to be: every variant of the input ablation fixes seed 0 and shares architecture, parameter count, optimiser, image order and quality-index draws, so the only difference between them is which inputs the head may see, and their agreement figures carry a standard error across 1,200 frames.

quality index	mean	sd	min	median	max	worst sequence
q0	32.70	7.51	6.04	35.89	39.12	BQSquare
q16	28.81	10.60	-0.95	31.67	39.12	BQSquare
q32	24.58	12.04	-0.95	26.10	39.12	PartyScene
q48	22.40	11.75	-0.95	23.54	39.12	BlowingBubbles
q63	20.00	11.13	-0.95	20.39	39.12	RaceHorses

Table 10. The interval this work can honestly report: saving in percent against the released decoder at the 0.1 dB budget, across the 53 test sequences, on the pinned checkpoint. Take from it that the spread is content and not noise. It widens with rate, from a standard deviation of 7.5 points at q0 to 11.1 at q63, and the minimum turns negative at the high rates because a sequence whose tiles all stay at the deepest exit costs about 1% more than the released decoder rather than less. This measures the test set, not the training draw, and is not a substitute for the run-to-run interval that does not exist.

Source: results/supp_per_sequence_PAPER_b010.json, the rows_vs_release block, computed from results/supp_paper_curve_PAPER.json on runs/RECIPE512/ckpt_PAPER.pth.tar. The file carries the same spread in our own deepest-exit denominator as well; the two differ by more than rounding and are never mixed.

quality index	path 1	path 2	gap
q0	34.38	31.95	2.43
q16	29.25	26.63	2.62
q32	23.21	20.84	2.37
q48	17.45	15.67	1.78
q63	7.21	8.74	-1.52

Table 11. Two implementations of the same quantity do not agree. Saving in percent at the 0.1 dB budget, computed by the curve-sweeping path and by the signalling path, which differ in how they pool distortion across tiles and frames. Take from it a floor on protocol sensitivity: the gap reaches 2.62 points, larger than several of the differences the paper draws conclusions from, so numbers from the two paths are never mixed inside one table.

Source: results/crosscheck_paths.json. The file records no checkpoint field.

Evaluation itself is deterministic once a checkpoint is fixed: the sequence list, the leading frames and the tile grid are all fixed and no crop is random. scripts/check_paper.py re-reads 85 numerical claims out of the paper's prose and checks each against the file it is supposed to come from. As recorded in results/check_paper.json, 85 of them pass. All of them pass. One did not until recently, and the cause is worth recording because it was not what it looked like: the partially signalled configuration appeared not to reduce to the fully signalled one at its endpoint, by 0.09 to 0.21 points, which bisection noise is far too small to explain. Neither code path was wrong. The two sides of the comparison were reading different measurements of the same quantity, and the endpoint reproduces exactly once both read the same file.

A.16. Data, code and provenance

- **Training images.** Open Images [24], from the CVDF public mirror named in scripts/fetch_openimages.sh. The terms are the ones that mirror publishes and nothing here redistributes them.
- **Test sequences.** UVG [25] from ultravideo.fi, MCL-JCV [26] and HEVC classes B, C and D from the public mirrors named in scripts/fetch_mcljcv.sh and scripts/fetch_hevc_bcd.sh, and HEVC class E from media.xiph.org. Each fetch script records its source URL, and no sequence is re-encoded before it is measured.
- **The released codec.** The DCVC-UF intra checkpoint [14], identified by sha256 in results/dmc_ld_recon_audit.json and used under the licence its repository ships.
- **Our code and checkpoints.** The trainer, the ladder, the cost model, the meter, the router and every evaluation script named here are in the repository this supplement is built from; the pinned checkpoint is identified by md5 in A.14.
- **Human subjects.** None, which is why A.4 declines to claim that 0.1 dB is invisible.

A.17. What no file in results/ pins

The rule this supplement follows is that a number is quoted only with the file it came from. Applying it honestly means listing the places where the file does not exist.

- **The decoder's training recipe.** Nothing that writes to results/ runs inside the trainer, so no results file records the learning rate, the schedule, the batch size, the crop, the loss weights or the number of steps. Table 6 is read from scripts/launch_recipe512.sh, train_flexuf_image.py and the run's own log, and the dataset statistics of A.8 from a prepare_stats.json that lives outside the repository.
- **The wall clock.** Read from runs/RECIPE512/train_log.jsonl, which a live process appends to, so the position quoted in A.10 and the hours in Table 8 are a snapshot taken at 2026-08-20 12:48.
- **The warm start.** Its own report, runs/warmstart/warmstart_report.json, holds the transfer counts and both zero-difference controls, but the copy on disk is the K = 12 rebuild of 17 August and the K = 6 report the pinned run descends from was overwritten. The controls are asserted inside the script at every build, so a failure would stop the run, but the K = 6 record is gone.
- **The exit-clamp mis-billing.** The two percentages in Table 3 are a comment in flexuf/cost.py recording a measurement made once and never written out.
- **A repeated training run.** With no seed and no configuration trained twice, no run-to-run interval exists. Table 10 is the nearest substitute and measures the test set, and Table 11 measures the evaluation path; neither measures the training draw.
- **A second class of accelerator.** All eight cards on this machine are the same model, so the only second device class section B can report a wall clock on is a CPU.

- **Any resolution above 1080p.** The queued 4K latency job returned without writing a file and results/supp_footprint.json marks its 3840×2304 row oom, so nothing here is measured at 4K.
- **A second decoder.** The ladder needs only a residual trunk with a shared head, and no second codec was rearranged into one, so the generality of the construction is argued and not measured.

B. Architecture and complexity accounting

This section states what the decoder is, module by module and convolution by convolution, derives what a decode costs from those shapes, and then checks the derivation against counts taken off decodes that were run. It closes with what the saving is worth in the units a deployment measures, which are milliseconds, joules, megabytes and frames per second, and none of which is the unit the allocation is optimised in.

Some vocabulary first, because the rest of the supplement uses it. A *multiply-accumulate* (MAC) is one multiply and one add, and MAC/px is per pixel of the grid the operator runs on, which is not the same grid for every operator. The *trunk* is the released decoder's stack of $N = 12$ identical DepthConvBlocks at width $C = 384$. An *exit* is a depth at which a tile may stop; the ladder has $K = 6$ of them, one every $b = N/K = 2$ blocks. The *split depth* $j = 2$ is how many exits' worth of blocks run once for the whole frame before any tile is allowed to leave, so the first $j \cdot b = 4$ blocks are full-frame and the last 8 are per-tile. An *adapter* is a small zero-initialised residual module that stands in for the blocks an exit skips. A *tile* is 256 RGB px square, which is 32 px of the feature grid and 16 px of the latent. The *exit map* is the per-tile assignment of exits. The *released decoder* is DCVC-UF's intra decoder [14] with none of this added, and it is the unit every cost below is quoted in.

B.1. The decoder, module by module

The released intra decoder is 14,231,808 parameters in 72 convolutions and nothing else: no attention, no normalisation with parameters of its own on the critical path, and no operator that changes the spatial grid except two pixel shuffles. It has three parts. An *upsample* takes the decoded latent, widens it with a 1×1 and doubles its grid with a pixel shuffle, then runs one DepthConvBlock at $C = 384$. A *trunk* of 12 further DepthConvBlocks runs at that width and grid throughout. A *head* narrows to 192 channels with a 1×1 , runs one more DepthConvBlock at that narrower width, and pixel-shuffles by 8 straight to RGB, with no convolution after the shuffle. Table 12 is that pipeline with its shapes.

Module	Where	Grid	Out ch.	MAC/px	% dec.
Latent y, decoder input	entropy dec.	68×120	256	0	0.00
1×1, widen	upsample	68×120	1536	393,216	0.71
PixelShuffle 2	upsample	136×240	384	0	0.00
DepthConvBlock	upsample	136×240	384	1,035,648	7.45
DepthConvBlock × 12	trunk	136×240	384	1,035,648	89.44
1×1, narrow	head	136×240	192	73,728	0.53
DepthConvBlock	head	136×240	192	259,776	1.87
PixelShuffle 8	head	1088×1920	3	0	0.00
Ours, added					
Conv1x1Adapter	exit 4	32×32	384	147,456	1.06
FFNAdapter	exits 0 to 3	32×32	384	737,280	5.31
GridSeamRepair	post-stitch	136×240	384	150,912	1.09
Router head	stem	per tile	$K = 6$	n/a	0.163

Table 12. The decode, module by module, at a 1920×1080 frame padded to 1920×1088. Grids are height × width, and “Where” names the released decoder's three parts: the upsample dec_1[0], the trunk dec_1[1..12] and the head dec_2. The last column is the module's share of one released decode, and for the two adapters it is the share if every tile wears that adapter. Take from the table that the grid changes exactly twice and never inside the trunk, so an exit ladder cut into the trunk needs no resampling; that the head is cheap because it runs at 192 channels rather than 384, which is a quarter of the arithmetic per pixel; and that the FFN adapter is the most expensive thing this work adds, at 5.31% of a decode, more than half a trunk block.

Released modules and their shapes: results/mac_audit.json, forward hooks on every convolution of an IntraDecoder at the real tensor shapes. A MAC count depends on shapes and not on values, so no weights are loaded for it. Adapters and seam repair: results/adapter_cost.json, hooks on the modules of runs/RECIPE512/ckpt_PAPER.pth.tar. Router: results/router_latency.json, measured on runs/RECIPE512/ckpt_eval.pth.tar; the head is the same size in both checkpoints.

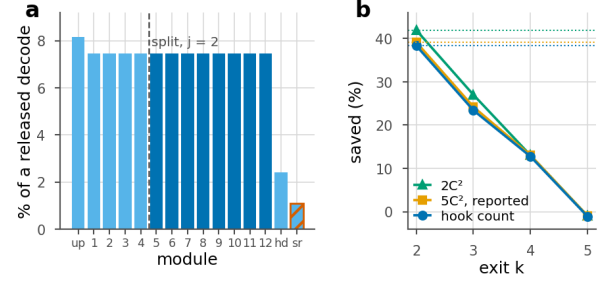


Figure 4. Where the arithmetic is, and what an exit saves. **a**, one released decode by module: the opening upsample, the twelve trunk blocks, the head, and the seam-repair pass this work adds (hatched). Light bars always run; dark bars are the eight blocks after the split that a tile can skip. **b**, the saving at each reachable exit under three prices for the same architecture, with each price's ceiling dotted. Section B.5 is the table behind it.

B.2. Every convolution in the decode

Table 13 opens up the modules of Table 12, giving every convolution in the decode in the (K, C_{in}, C_{out}, S) convention that the architecture figures of this literature use, with the group count added because two of the shapes are depthwise. The 72 convolutions of the released decoder have only 8 distinct shapes between them, because the DepthConvBlock repeats.

Convolution	K	C_{in}	C_{out}	S	Grp	MAC/px	Used
up.conv.0	1	256	1536	1	1	393,216	1
dc.0, 1×1 in	1	384	384	1	1	147,456	×13
dc.2, 3×3 depthwise	3	384	384	1	384	3,456	×13
dc.3, 1×1 out	1	384	384	1	1	147,456	×13
ffn.0, 1×1 C→4C	1	384	1536	1	1	589,824	×13
ffn.2, 1×1 4C→C	1	384	384	1	1	147,456	×13
dec_2.adapter	1	384	192	1	1	73,728	1
head dc.0	1	192	192	1	1	36,864	1
head dc.2, depthwise	3	192	192	1	192	1,728	1
head dc.3	1	192	192	1	1	36,864	1
head ffn.0	1	192	768	1	1	147,456	1
head ffn.2	1	192	192	1	1	36,864	1
Conv1x1Adapter	1	384	384	1	1	147,456	1
FFNAdapter pw_in	1	384	1536	1	1	589,824	×4
FFNAdapter pw_out	1	384	384	1	1	147,456	×4
Seam repair, 3×3 dw	3	384	384	1	384	3,456	1
Seam repair, 1×1	1	384	384	1	1	147,456	1

Table 13. Every distinct convolution in the decode. K is the kernel side, S the stride, Grp the group count; MAC/px is per pixel of that operator's own grid, which is the latent grid for the first row and the feature grid for the rest. “Used” counts the instances in one decode: the DepthConvBlock appears 13 times, once in the upsample and 12 times in the trunk, and there are $K - 1 = 5$ adapters of which four are of the FFN kind. Take from it that only two of the shapes have a kernel wider than 1×1 and both are depthwise, so the 14 instances of them in one decode carry 0.34% of the arithmetic between them.

results/mac_audit.json for the released rows, results/adapter_cost.json for ours. The two depthwise convolutions are the whole of the decoder's spatial footprint, which is why a tile boundary is cheap to repair.

That last figure is the premise of the whole method, so it is worth stating separately: 99.66% of the released decoder's arithmetic is 1×1 convolution, which has no neighbours to miss. The receptive field of the trunk plus head grows by exactly one feature pixel per block, from 1 feature pixel with no trunk blocks to 13 with all 12, which is 104 RGB px per tile edge (results/dmc_ld_rf_probe.json, measured on the released DCVC checkpoints rather than ours). A tile decoded alone is therefore wrong only in a band, and the band is narrower for a tile that exits early.

The two DepthConvBlock widths obey the same closed form. Writing the block as a 1×1 projection, a 3×3 depthwise, a second 1×1 and a feed-forward pair whose first 1×1 expands to $4C$ and whose gated activation folds $4C$ back to C for free, the closing 1×1 is $C \rightarrow C$ rather than $4C \rightarrow C$ and the block is

$$M_{\text{block}}(C) = C^2 + 9C + C^2 + 4C^2 + C^2 = 7C^2 + 9C \quad (1)$$

which is 1,035,648 MAC/px at $C = 384$ and 259,776 at $C = 192$, both matching Table 13 row by row. The cost model shipped in flexuf/cost.py writes it as $8C^2 + 9C = 1,183,104$ instead, one C^2 too many, which overstates a block by 14.24%. Sections B.5 and B.6 follow that one term to the end.

B.3. What the ladder adds

Three modules are new. An exit that stops early hands the head a feature map the remaining blocks would have refined, and an *adapter* stands in for them. The 1×1 adapter is $f + Wf$ with W a $C \times C$ matrix. The FFN adapter repeats the block's feed-forward pair, a 1×1 expanding $C \rightarrow 4C$, the same gated activation, and a 1×1 $C \rightarrow C$. Both are zero-initialised, so the ladder is the released decoder exactly at step zero and any quality the adapters add is attributable to them. Neither has a spatial footprint, so neither widens the seam. The *seam repair* is a gated depthwise 3×3 followed by a zero-initialised 1×1 , run once on the stitched canvas after unpatchify and before the head, which is the only place a 3×3 can see across a tile boundary. The *router head* reads the shared stem and predicts a per-tile exit; Section F is its architecture and this section only prices it.

Module	Operators	MAC/px	In C^2	Blocks	Of decode
1×1 adapter	1×1 $C \rightarrow C$	147,456	1.00	0.1424	1.061%
FFN adapter	1×1 $C \rightarrow 4C$, 1×1 $C \rightarrow C$	737,280	5.00	0.7119	5.306%
Seam repair	3×3 dw, 1×1 $C \rightarrow C$	150,912	1.02	0.1457	1.086%
Router head	two 1×1 , then an MLP	n/a	n/a	n/a	0.163%

Table 14. What the ladder adds, priced against one released decode. “Blocks” is the module in units of a trunk block. Take from it that the FFN adapter costs $5C^2$ and not $2C^2$: the gated activation is free, so the pair is $4C^2 + C^2$ rather than $4C^2 + 4C^2$, and the module is 0.71 of a whole block. Since the routed allocation lives at exits 2 and 3, which both wear the FFN adapter, that correction lands on precisely the exits the reported saving leans on.

results/adapter_cost.json for the two adapters, on the pinned checkpoint. The seam-repair row is $9C + C^2$ per pixel; Section B.6 confirms that figure against a hook count rather than taking it from the source. Router: results/router_latency.json.

The ladder assigns the FFN adapter to any exit skipping four or more blocks and the 1×1 adapter otherwise, which at $K = 6$ and $b = 2$ means the FFN at exits 0 to 3, the 1×1 at exit 4, and no adapter at exit 5, whose feature is the trunk's own output with nothing standing in for a skipped block. The arithmetic at exit 5 is therefore the released decoder's, plus the repair pass, which is why the deepest exit costs slightly more than the release rather than the same.

B.4. From the block to the decoder

Write s_{up} , s_{trunk} and s_{head} for the shares of a released decode taken by the upsample, the trunk and the head; a_k for the adapter at exit k ; and r for the seam-repair pass. A tile assigned an exit shallower than j still runs group j , because the decoder clamps the map, so the cost of exit k in units of one released decode is

$$c_k = s_{\text{up}} + s_{\text{head}} + r + s_{\text{trunk}} \frac{(\max(k, j) + 1)b}{N} + a_{\max(k, j)} \quad (2)$$

and the frame-level cost of an exit map a over T tiles amortises the shared part over the frame rather than over the tile,

$$C(a) = s_{\text{up}} + s_{\text{trunk}} \frac{jb}{N} + s_{\text{head}} + r + \frac{1}{T} \sum_{i=1}^T u_{a(i)} \quad (3)$$

with u_k the per-tile suffix, meaning the blocks after the split plus the adapter. Only the last term depends on the map. That is the structural reason a ladder over a decoder of this shape has a ceiling well below 100%: at the split depth the upsample, the first $j_b = 4$ blocks, the head and the repair pass are already 41.5% of a released decode, and they run whatever the map says.

None of the four shares has to be taken on trust. results/mac_audit.json gives $s_{\text{up}} = 0.0816$, $s_{\text{trunk}} = 0.8944$ and $s_{\text{head}} = 0.0240$ by hook count, so one trunk block is 0.074533 of a decode. That figure has an independent check. results/static_RECIPES12_b01.json records the frame-level saving of a map that sends every tile to exit e , for $e = 2$ to 5, on the pinned checkpoint; differencing consecutive entries gives one trunk block as 0.074533, which agrees with the audit to 7×10^{-7} of a decode. One is a shape audit of the released decoder with no weights loaded and the other is a set of measured savings on ours, which is the reason to report both.

B.5. The per-exit cost vector and the ceiling

Exit k	Blocks	Adapter	c ($2C^2$)	c ($5C^2$)	c hook	Saved %
0 (clamped)	6	FFN	0.5809	0.6088	0.6167	38.33
1 (clamped)	6	FFN	0.5809	0.6088	0.6167	38.33
2	6	FFN	0.5809	0.6088	0.6167	38.33
3	8	FFN	0.7300	0.7578	0.7658	23.42
4	10	1×1	0.8697	0.8697	0.8724	12.76
5	12	none	1.0095	1.0095	1.0109	-1.09

Table 15. The per-exit cost vector, in units of one released decode, under three prices for the same architecture: the first shipped table, which put the FFN adapter at $2C^2$; the table the paper reports, which corrected that to $5C^2$ but kept the closed-form block; and the hook count assembled in Section B.4. Take from it that the two corrections move the shallowest reachable exit by 2.79 and then 0.80 points and leave exits 4 and 5 nearly untouched, because those wear the cheap adapter or none, and that the deepest exit costs 1.09% more than the release, which is the seam-repair pass.

Exits 0 and 1 are unreachable: the decoder clamps the map at $j = 2$, so a tile nominally assigned them leaves through exit 2 and wears exit 2's cost. The $2C^2$ and $5C^2$ vectors are recorded in results/why_qp.json and results/why_qp_PAPER.json; a cost vector is a property of the model rather than of the weights, so the first being measured on runs/BEST/ckpt_eval.pth.tar does not affect it.

The ceiling is $100(1 - c_j)$, the saving when every tile takes the shallowest reachable exit. It is a property of the architecture and no allocation can pass it. Three values are in circulation and they are the three columns of Table 15: 41.91% at $2C^2$, 39.1% at $5C^2$, and 38.33% by hook count, which is the value the paper's tables use. The last of the three is the one a meter agrees with, and the next subsection is the demonstration.

B.6. Predicted against hook-measured

A cost model and the decoder it models drift apart. The check that catches it is a hook count: flexuf/measure.py attaches a forward hook to every convolution and linear layer and accumulates MACs from the output shape of each call. A group skipped by an early exit never fires its hook and so costs nothing; a group run on 28 of 40 tiles costs 28 tiles' worth, because tiles are the batch dimension and the output shape carries them. There is no bookkeeping to get wrong, and the count is the ground truth the model is judged against.

Condition	Map 2/3/4/5	Measured	5C ²	diff	Hook	diff
720p q0	11/4/0/0	34.3524	35.1494	+0.797	34.3524	+0.0000
720p q32	6/4/2/3	23.0606	23.6547	+0.594	23.0606	+0.0000
720p q63	4/3/4/4	18.0177	18.4971	+0.479	18.0177	+0.0000
1080p q0	28/10/1/1	32.9763	33.7436	+0.767	32.9763	+0.0000
1080p q32	16/10/6/8	22.8829	23.4682	+0.585	22.8829	+0.0000
1080p q63	11/7/11/11	17.8489	18.3184	+0.470	17.8489	+0.0000

Table 16. Predicted against measured, on six decodes with six different exit maps at two resolutions. “Measured” is the hook count off the decode that was executed and timed. Take from it that the assembly of Section B.4 reproduces the count exactly in every condition, to four decimal places and in fact to floating point, while the model the paper reports is optimistic by 0.47 to 0.80 points and always in the same direction.

results/supp_power.json, pinned checkpoint, 0.1 dB budget, NVIDIA RTX A6000. The map is the file’s pooled exit histogram rounded to the frame’s tile count, which is what scripts/power_profile.py decodes; its four columns are the tiles leaving at exits 2, 3, 4 and 5.

Because a hook count is exactly linear in the exit map, six mixtures spanning four reachable exits over-determine the per-exit vector and it can simply be solved for. The six equations are consistent to 3×10^{-16} , and the solution differs from the assembled vector by at most 2×10^{-15} at any exit. The identity is worth stating plainly: the per-module counts of results/mac_audit.json and results/adapter_cost.json are not a model of what the meter reports, they are what the meter reports, rearranged. Where the paper’s tables differ from the meter, the difference has a closed form and Table 17 gives it term by term.

Term	Model price	Hook price	Points of saving
Seam repair	0.009507	0.010861	0.1354
1×1 adapter	0.009289	0.010612	0.1323
FFN adapter	0.046447	0.053060	0.6613
Exit 5 residual			0.1354
Exit 4 residual			0.2678
Exits 2, 3 residual			0.7970

Table 17. The residual of Table 16, term by term. Three modules are priced as a fraction of a block, and the shipped model’s block is $8C^2 + 9C$ where the meter’s is $7C^2 + 9C$. Take from it that the residual is not noise and not a tolerance: exit 5 wears the repair mispricing alone, exit 4 wears the repair plus the 1×1 adapter, and exits 2 and 3 wear the repair plus the FFN adapter, which is why the error is largest exactly where the saving is largest.

Model prices: the module’s MAC/px divided by $8C^2 + 9C$, times one trunk block. Hook prices: the module’s MAC/px times the feature grid, divided by the decode total, both from results/mac_audit.json and results/adapter_cost.json, and they are the last column of Table 14. The three residual rows are the corresponding columns of Table 15.

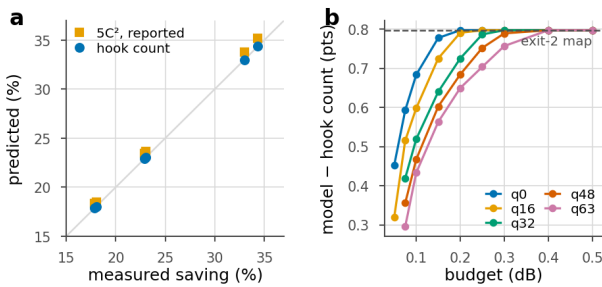


Figure 5. The model against the meter. **a**, six decodes at two resolutions: the hook-count assembly lies on the identity line and the model the paper reports lies above it. **b**, the same residual over the whole budget grid, 9 budgets by 5 quality indices on 53 sequences. It is positive everywhere, grows with the budget, and saturates at the value an all-exit-2 map implies.

Figure 5b runs the same comparison over the whole operating range rather than six points of it. The residual is positive in all 42 cells, from 0.296 to 0.797 points, and it saturates at exactly the exits-2 value of Table 17. Reading the paper’s headline figures as hook counts therefore means subtracting between 0.43 and 0.69 points: at the 0.1 dB budget the model reports 29.2% at the lowest rate against a measured 29.21%, and

15.2% at the highest against a measured 15.20%.

results/signalled_RECIPES12_grid.json and results/signalled_RECIPES12_ctc53.json, both on the pinned checkpoint. Both columns are in the files: saving_pct_vs_release is the model, saving_pct_measured is the hook count, and model_minus_measured is their difference.

B.7. How the timings were taken

The rest of this section leaves arithmetic for the clock, so the protocol comes first. Everything below follows the same rules and they are stated rather than implied.

- **Batch 1, and why.** Every wall clock here is one frame at a time, which is what a decoder does. A batch is genuinely different work for this decoder and not a repetition, because exit_map is indexed over the whole batch’s tiles, so B frames of 40 tiles form one shrinking active set of 40B and the groups run wider. Section B.8 measures what the choice costs.
- **Warm-up, then interleaving, then the median.** 20 untimed iterations, then 40 rounds of one call of each variant in turn. This is a shared card, and timing all of the released decoder and then all of ours puts any drift in a neighbouring job’s load straight into the ratio, which is the only quantity these runs exist to produce.
- **What the timer wraps.** time.perf_counter around a synchronised call, not CUDA events, because events do not exist on the CPU and the two device classes have to be timed alike. The timer starts at the decoded latent and stops at the reconstruction, so it includes patchify, every group, unpatchify, the seam-repair pass and the head.
- **What is deliberately outside it.** Entropy decoding, because the bitstream and therefore the rate do not depend on the exit map: including it would divide both sides of every ratio by the same constant and flatter the method. The map itself is charged, in bits rather than in time, at 89 bits per frame.
- **The router is charged in MACs and not in seconds.** Its arithmetic is 0.163% of a decode but 0.50% of decode time, 3.0× its arithmetic share, an extra 0.33 points bought by launching a small kernel over a large map. That is the general reason a MAC count cannot see a kernel launch, in miniature (results/router_latency.json, 40 iterations at 2048×1280 on an NVIDIA RTX A6000, measured on runs/RECIPES12/ckpt_eval.pth.tar). The router does not run at all in the signalled configuration, where the encoder chooses the map.
- **The masked path, not the sorted one.** The shipped configuration leaves sorted_tiles off, so every group is a masked gather over the full tile batch. Sorting the tiles by depth turns each group into a contiguous slice and recovers 1.2 points at 1080p for bit-identical output (results/latency_RECIPES12_sorted.json, on runs/RECIPES12/ckpt_eval.pth.tar). None of the timings below use it, so they are the slower of the two implementations.

B.8. Where the seconds go, and on what device

Stage	MAC %	1080p tiles	1080p ms	720p tiles	720p ms
Stem: upsample + groups 0, 1	37.97	frame	41.39	frame	16.30
Patchify	0.00		0.21		0.09
Group 2 (2 blocks, per tile)	14.91	40	15.98	15	6.19
Group 3 (2 blocks, per tile)	14.91	28	11.24	10	4.16
Group 4 (2 blocks, per tile)	14.91	14	5.72	5	2.23
Group 5 (2 blocks, per tile)	14.91	9	3.79	3	1.30
Unpatchify	0.00		0.21		0.09
Seam repair (full-frame)	1.09	frame	2.07	frame	0.83
Head	2.40	frame	4.36	frame	1.72

Table 18. Where a routed decode goes, in arithmetic and in milliseconds. The MAC column is each stage's share of a released decode at full occupancy and sums to 101.09%, which is the deepest exit's cost. Take from the table that patchify and unpatchify are free arithmetically and nearly free on the clock, that the repair pass is 1.09% of the arithmetic and 2.4% of this decode's clock, and that the head takes 5.1% of the clock for 2.40% of the arithmetic, being bandwidth-bound rather than arithmetic-bound.

results/supp_latency_1920x1080.json and results/supp_latency_1280x720.json, pinned checkpoint, q32, 0.1 dB budget, NVIDIA RTX A6000 taken under the evaluation lock so the card was not shared. Tile counts are that decode's own exit map, which leaves 28, 14 and 9 of 40 tiles alive in groups 3, 4 and 5.

Patchify and unpatchify are reshapes and cost no arithmetic at all. They require the feature map to divide by the 32 px feature tile, which the encoder-side replicate padding guarantees: 1920×1080 becomes 2048×1280 and 40 tiles, 1280×720 becomes 1280×768 and 15 (results/tile_definition.json).

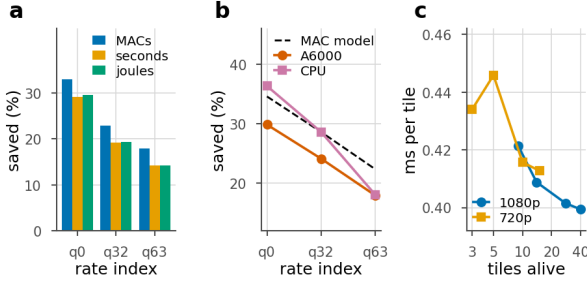


Figure 6. What the arithmetic is worth. **a**, one exit map measured three ways at 1080p. **b**, the same map on two device classes against what the MAC model predicts: the A6000 falls short of the prediction at every rate, the CPU does not. **c**, why a GPU falls short: a tile costs more to decode in a group that has fewer tiles left in it.

Two effects separate seconds from multiply-accumulates and the figure isolates both. The first is the tiled path itself, which the released decoder does not pay: patchify, unpatchify and the repair pass are 2.9% of the routed clock at 1080p. Timing our own decoder with every tile at the deepest exit isolates it: that variant does the same arithmetic as the release plus the tiling machinery, and it is 2.6 to 3.8% slower across nine measurements on the pinned checkpoint, against 3.6% for the figure the main paper quotes. The second is occupancy. A group late in the ladder launches kernels over a handful of 32×32 feature maps, and a GPU sized for the first group is partly idle inside them. Dividing each group of Table 18 by the tiles alive in it, at 1080p a tile costs 5.5% more in group 5, on 9 tiles, than in group 2 on 40. At 720p the same curve rises by 8.0% from 15 tiles to 5 and then falls back at 3, so the trend is clear at 1080p and not monotone at 720p.

Device class and batch size are the two knobs a reader would ask about next, and Table 19 sweeps both. All eight cards in this machine are RTX A6000s, so the only second device class reachable here is the CPU. It is also the unflattering comparison to make, in the sense that it is the one our own explanation predicts we will look better on: a CPU decode is closer to arithmetic-bound, so if the shortfall on the GPU is scheduling rather than arithmetic it should shrink.

Device	Rate	Batch	ms/frame rel.	ms/frame ours	fps ours	Saved %	MAC model %
A6000	q0	1	111.3	78.1	12.8	29.84	34.58
A6000	q0	2	111.1	76.7	13.0	30.97	34.58
A6000	q0	4	110.7	76.5	13.1	30.96	34.58
A6000	q32	1	111.5	84.6	11.8	24.11	28.59
A6000	q32	2	111.3	83.3	12.0	25.16	28.59
A6000	q32	4	110.8	82.9	12.1	25.17	28.59
A6000	q63	1	111.5	91.5	10.9	17.92	22.33
A6000	q63	2	111.2	89.9	11.1	19.14	22.33
A6000	q63	4	110.7	89.6	11.2	19.06	22.33
CPU, 8 thr.	q0	1	8836	5625	0.178	36.34	34.58
CPU, 8 thr.	q32	1	8153	5822	0.172	28.59	28.59
CPU, 8 thr.	q63	1	8280	6788	0.147	18.02	22.33

Table 19. Wall clock at 1080p by device class and batch size, on one exit map per rate. “rel.” is the released decoder timed in the same loop; fps is one over our per-frame time. Take from it that batch 1 costs about a point of saving against batch 4 and nothing beyond it, so the batch-1 convention this work uses is close to free; and that the A6000 falls short of the MAC model by 3.2 to 4.7 points at every rate and every batch size, while the CPU meets or beats it at the two lower rates.

results/supp_latency_batch_1920x1080.json and results/supp_latency_cpu_1920x1080.json, pinned checkpoint, 0.1 dB budget, exit maps read from results/supp_paper_curve_PAPER.json. GPU rows: 20 warm-up and 40 interleaved iterations. CPU rows: 8 threads, 1 warm-up and 5 iterations only, on a shared machine.

The CPU rows carry a noise floor and it should be read before the conclusion. The released decode does not depend on the rate, so its three timings should be equal; they range from 8.15 to 8.84 s, a spread of 8.4%, which is what 5 iterations on a shared machine buys. That is enough to support the gap at q0 and q32, where the CPU is 6.5 and 4.5 points above the A6000 on the same map, and not enough to support anything at q63, where the two agree to a tenth of a point. The direction is the one the occupancy explanation predicts, and a firmer statement needs a machine that is not shared.

B.9. What the saving is worth in joules

Board power was sampled from the driver at 100 ms, which is about the rate the sensor updates. Each condition therefore runs for a fixed wall time of 20 s rather than a fixed iteration count, so that a 30 ms decode yields two hundred power samples rather than three. Idle is measured for 4 s immediately before and after every timed run on the same card, so a neighbouring process waking up mid-measurement appears as a drift between the two rather than as a saving. Energy per frame is the mean board power over the run times the median decode time.

Condition	MACs %	Time %	Gap	Energy %	Above idle %
720p q0	34.35	30.81	3.54	30.10	29.70
720p q32	23.06	18.13	4.93	18.45	18.63
720p q63	18.02	12.99	5.03	12.96	12.94
1080p q0	32.98	29.07	3.91	29.56	29.88
1080p q32	22.88	19.14	3.75	19.34	19.48
1080p q63	17.85	14.14	3.71	14.17	14.18

Table 20. The three units side by side, all as savings of the routed decode against the released decode on the same latent. Take from it the result of this subsection: energy saved tracks time saved to within 0.5 points in every condition, while arithmetic saved runs 3.5 to 5.0 points ahead of both. Subtracting idle power changes nothing, which is the check that the agreement is not an artefact of the static draw.

results/supp_power.json, pinned checkpoint, 0.1 dB budget, NVIDIA RTX A6000 with a 300 W limit, taken under the evaluation lock. 720p is padded to 1280×768 and 15 tiles, 1080p to 2048×1280 and 40 tiles.

Condition	ms rel.	ms ours	W rel.	W ours	J rel.	J ours
720p q0	43.45	30.06	283.9	286.8	12.338	8.624
720p q32	43.50	35.61	293.7	292.6	12.774	10.418
720p q63	43.53	37.88	294.1	294.2	12.804	11.144
1080p q0	110.74	78.55	298.1	296.1	33.016	23.257
1080p q32	110.76	89.56	298.1	297.4	33.019	26.633
1080p q63	110.75	95.08	298.0	298.0	33.008	28.332

Table 21. The measurement behind Table 20, in absolute units. Take from it why energy tracks time: the routed decode draws the same board power as the released one, within 3 W in the worst case and within 1 W in four of the six conditions, and at 720p q0 it draws slightly *more*. Routing does not lower the instantaneous draw of the card. It shortens the job.

results/supp_power.json. Watts are the mean of about 200 driver samples per run; times are medians over 181 to 664 iterations. Idle over the six conditions in the order run: 74.6, 78.5, 91.2, 100.5, 106.5 and 98.6 W before, and 122.4, 120.0, 126.2, 128.4, 125.0 and 121.9 W after, on a card whose decode draw is 284 to 298 W.

The engineering result should be stated plainly: the energy saving of this method tracks its time saving, not its arithmetic saving. A partly idle GPU still draws most of its static power, so the joules follow the seconds almost exactly, and the seconds fall short of the multiply-accumulates for the occupancy reason of Figure 6c, at the board powers of Table 21. At 1080p and the lowest rate the three read 32.98%, 29.07% and 29.56%; at the highest rate they read 17.85%, 14.14% and 14.17%. Quoting the arithmetic figure as an energy figure overstates it by three and a half to five points.

Two limits. The idle baseline drifts upward as the card warms, from 74.6 W before the first condition to 128.4 W after the fourth, so the above-idle column of Table 20 is the weaker of the two energy figures, and it is only because it agrees with the unadjusted column to within 0.4 points that the conclusion survives. Power also comes from the driver counter rather than an external meter, so the absolute joules inherit whatever bias that sensor has; the savings are ratios of two measurements taken the same way on the same card minutes apart, which is what the bias mostly cancels in.

B.10. Memory, frame rate, and what the encoder pays

Frame (padded)	Tiles	MB rel.	MB ours	Change %	fps rel.	fps ours
1024x512	8	180.0	204.0	+13.33	43.89	49.88
1280x768	15	337.5	382.5	+13.33	22.94	28.03
2048x1280	40	900.0	1020.0	+13.33	9.03	11.18
3840x2304	135	out of memory				

Table 22. Peak decoder memory and frame rate, released against ours, at every resolution that fitted. Take from it that routing costs memory rather than saving it, by a constant 13.33% at every resolution, because the tiled path holds the whole tile batch and the stitched canvas alive together; and that the frame-rate gain grows with the frame, from 43.9 to 49.9 fps at 832×480 up to 9.0 to 11.2 fps at 1080p, because a larger frame has more tiles to allocate across. The 4K row is a genuine failure and is reported as one: the job ran out of memory on a 48 GB card that four other processes were sharing, so no measurement above 1080p exists anywhere in this work.

results/supp_footprint.json, pinned checkpoint, q32, 0.1 dB budget, NVIDIA RTX A6000. Frame rates here are lower than in Table 19 because that table's maps are at different rates and this one is q32 throughout. The 4K stage profile queued alongside it failed the same way (results/supp_queue.log).

The signalled configuration moves the choice of exit map to the encoder, which has to price the tiles before it can allocate them, and that cost belongs in a complexity section even though it falls on the other side. On one 1080p sequence at the highest rate, one tiled decode takes 114.9 ms; building the per-tile cost table full-frame takes 157.9 ms, or 1.373× one decode; and building it the way the deployed encoder does, per tile, takes 519.2 ms, or 4.517×. The two searches agree on 82.5% of tiles and reach 17.39% and 16.71% saving at the same budget. The asymmetry is the design rather than an accident: a decoder-side saving is bought with encoder-side work, which suits a compress-once decode-many deployment and does not suit live encoding.

results/supp_encoder_cost_PAPER.json: pinned checkpoint, one sequence (Bosphorus), q63, 40 tiles, 15 iterations on an NVIDIA RTX A6000.

B.11. What no file in results/ pins

Four gaps, listed so that they are visible rather than quietly absent.

- **The paper's tables are on the 5C² column.** Tables 15 and 17 together say by how much: between 0.14 and 0.80 points of saving, in the optimistic direction at every exit. The hook-count column is the one a meter agrees with, and the honest ceiling is 38.33% rather than 39.1%. The BD-Rate figures are unaffected, being integrals of rate against quality with no compute axis in them; the BD-saving figures would shift by the same fraction of a point as the savings they integrate.
- **Nothing above 1080p.** Both 4K jobs ran out of memory (Table 22). GPU 2 has about 5 GB free on a card four other processes are sharing, and a 3840×2304 decode does not fit, so the resolution trend in this section rests on three points of which two carry power measurements. This is a limit of the machine and not of the method.
- **One GPU model, one power sensor, and no second card class.** Every millisecond and joule on a GPU here is an NVIDIA RTX A6000, because all eight cards in this machine are; the CPU rows of Table 19 are the only second device class, and they carry the noise floor stated beside them. Nothing here separates the decoder's energy from the board's, and no external meter was available to calibrate the driver counter against.
- **One budget for the clock.** Every timing, power and memory measurement in this section is at the 0.1 dB budget, and the stage profiles are at q32 only. The arithmetic is swept across the whole grid in Figure 6b; the seconds are not, and a deeper budget moves more tiles to exit 2, where the occupancy penalty of Figure 6c is smallest.

C. Derivations

This section states the allocation problem from its objects upward and proves what the paper asserts about it. Nothing here needs a result from elsewhere in the supplement. Each statement carries the assumptions it uses, a proof, and a remark on what it leaves open.

no.	claim	in	checked in	ckpt
P1	separates over tiles	C.2	verify_theory	eval
P2	monotone in λ	C.2	verify_theory	eval
P3	blind sets are a hull	C.3	static_RECIPES12_b01	PAPER
T4	a Minkowski average	C.3	not measured	-
P5	reaches only the hull	C.3	hull_gap	eval
P6	savings on a 1/N lattice	C.3	theory_checks	eval
P7	the floor	C.4	saturation_RECIPES12	PAPER
P8	the saturation price	C.4	verify_theory	eval
P9	the ceiling	C.4	per_class_RECIPES12	PAPER
T10	adaptivity gain	C.5	theory_check	BEST
P11	plotted log-convexity	C.6	logconvexity	none
P12	Lorenz bound	C.7	hybrid_RECIPES12_b01	PAPER
P13	rank-1 threshold rule	C.8	raterank_RECIPES12_b01	PAPER
P14	rank-1 uses hull exits	C.8	raterank_RECIPES12_b01	PAPER

Table 23. The numbered results of this section, and the evidence for each. P is a proposition and T a theorem; the numbering is stable and is what the main paper cites. PAPER is runs/RECIPES12/ckpt_PAPER.pth.tar, eval is runs/RECIPES12/ckpt_eval.pth.tar, since overwritten by the per-epoch watcher, and BEST is runs/BEST/ckpt_eval.pth.tar. verify_theory is scripts/verify_theory.py, which reports seven of seven propositions passing on one sequence at one rate.

Files named without a directory are under results/ with a .json extension; the saturation and Lorenz rows are the _ctc53 and _b01_fixed variants of the names given. T4 is proved and not measured: it describes a set of 120 vertices that no experiment enumerates.

C.1. The setting, and four assumptions

Tiles. The decoder is cut at a fixed depth. Everything above the cut runs once over the whole frame; everything below runs on square patches of the trunk's feature grid, which are the tiles. The frame is padded by replication first, so the grid divides exactly and the tiling is a partition. A tile is 32 feature positions on a side, 16 latent positions and 256 pixels. Tiles are indexed by t and N is their number in a frame: 40 at 1080p and 2 at 416×240.

Exits. The per-tile part of the decoder is a stack of 12 residual blocks, and an exit is a point at which a tile may stop and be handed to the reconstruction head. There are $K = 6$, evenly spaced, so exit k runs $(k+1)b$ blocks with $b = 2$. The split depth is $j = 2$: blocks 0 to $jb-1$ run full-frame whatever exit a tile takes, so the usable ladder has $K - j = 4$ members and k ranges over $j, \dots, K-1$ throughout. Each exit below the deepest carries a small zero-initialised adapter, a tile leaving early handing the head a feature the head was not trained to read.

Distortion. $D_{t,k}$ is the mean squared error tile t incurs when it leaves at exit k , and two choices in that definition do real work. The error is measured on the path actually deployed, a tiled decode with replicate padding at tile borders, so what tiling costs sits inside $D_{t,k}$. And it is measured against the released decoder's decode of the same latent, so encoder, hyperprior and entropy model are byte-identical on both sides and synthesis is the only difference. Nothing forces $D_{t,K-1}$ to zero, and C.4 returns to that.

Decibels. Distortion is reported as $10 \log_{10}$ of the ratio to the reference, and the ratio can be formed at two levels. Pooling every tile of every frame into one mean squared error is the form the Lagrangian below is exact for; averaging a per-frame decibel is the codec convention, and it is what the budget is bisected against throughout. On an identical allocation the two read 0.023 to 0.033 dB apart, 23% to 33% of the 0.1 dB budget, so a decibel here means nothing without its convention.

Convention gap from results/db_convention.json, checkpoint runs/wdec_j2_p128_grid/ckpt_epo0.pth.tar, 40 frames: an old checkpoint for a claim about arithmetic rather than about a model, and the only file in results/ measuring both conventions on one allocation. The budget section of this supplement measures the gap on the pinned checkpoint.

Compute. Compute is counted in multiply-accumulate operations and divided by those of one released full-frame decode, 453.5 GMAC at 1080p, so $c = 1$ is one released decode and a saving of $S\%$ means the frame was decoded for $(1 - S/100)$ of one. Every operator runs at a fixed multiple of the latent grid, which makes the cost of an exit resolution-independent. One tile at exit k costs

$$c_k = s_{\text{up}} + s_{\text{trunk}} \frac{(k+1)b}{N_{\text{blk}}} + s_{\text{head}} + a_k + r \quad (4)$$

with $s_{\text{up}} = 0.0816$ the opening upsample, $s_{\text{trunk}} = 0.8944$ the trunk spread over $N_{\text{blk}}^{\text{up}} = 12$ blocks of which exit k runs $(k+1)b$, $s_{\text{head}} = 0.0240$ the head, a_k the adapter exit k wears and $r = 0.0095$ the full-frame deblocking filter. Only the trunk term depends on k , which is why a ceiling exists: however shallow the ladder gets, the stem, the head and the filter are still paid. The four shares sum to 1.0095072, the deepest-exit cost in every configuration file read here.

exit k	blocks	adapter	share	c_k	$100(1-c_k)$	earlier c_k
2	6	ffn	0.712	0.6088	39.12	0.5809
3	8	ffn	0.712	0.7578	24.22	0.7300
4	10	1×1	0.142	0.8697	13.03	0.8697
5	12	none	0.000	1.0095	-0.95	1.0095

Table 24. What each exit costs, in units of one released full-frame decode, with the adapter share in units of one residual block. The deepest exit costs more than 1 because a tiled decode still pays the deblocking filter. The last column is the earlier cost vector, built when the FFN adapter was billed at 2C² instead of its measured 5C²; the checks of Propositions 5, 6, 8 and 10 were run against it and say so where they appear.

c_k from the uniform-depth rows of results/static_RECIPES12_b01.json and adapter shares from results/adapter_cost.json, both on runs/RECIPES12/ckpt_PAPER.pth.tar;

earlier vector from results/tile_table.json. The complexity section audits the block count itself.

The gaps between those costs are what the allocation trades in, and they are unequal: 0.1491, 0.1119 and 0.1398. The first is largest because exit 3 gives up the FFN adapter's saving as well as two blocks.

Assignments. An assignment is a map a from tiles to exits. There are $(K-j)^N$ of them, 4^{40} at 1080p, so they will not be enumerated. The frame's compute and distortion are

$$C(a) = \frac{1}{N} \sum_{t=1}^N c_{a(t)}, \quad D(a) = \frac{1}{N} \sum_{t=1}^N D_{t,a(t)} \quad (5)$$

Everything after this rests on four assumptions, none of them a mathematical necessity. Each result names the ones it uses.

- **(A1) Compute is additive over tiles.** Exact when tiles decode independently, which is what a tiled decoder does, and it is why the stem and the head sit inside c_k .
- **(A2) Distortion is a tile mean**, and a tile's error does not depend on which exits its neighbours took. The first half is exact for any per-pixel loss averaged over a frame; the second is not free, the deblocking filter running on the stitched canvas and seeing the whole exit map at once.
- **(A3) The reference is fixed.** $D_{t,k}$ is measured on the deployed tiled path against the released decoder's own decode of the same latent, so rate is identical on both sides and cancels.
- **(A4) The cost vector is strictly increasing** in k over $k \geq j$, which Table 24 shows it is.

The second half of (A2) is checked rather than asserted. The per-tile table and the measured decode of the corresponding mixed map are recorded for 60 allocations of one frame, spanning savings of 1.2% to 41.9%. The largest disagreement between prediction and decode is 5.5×10^{-5} dB, four orders of magnitude below the budget. Nothing in (A1) to (A4) constrains $D_{t,k}$ itself, and nothing requires it to fall as k grows.

Additivity check computed from results/tile_table.json (Bosphorus, q32, 40 tiles), checkpoint runs/RECIPES12/ckpt_eval.pth.tar, since overwritten by the per-epoch watcher. What it checks is a property of the decode path rather than of the weights.

C.2. A price on compute

The encoder's problem is to spend as little compute as possible while staying inside a distortion budget:

$$\min_a C(a) \quad \text{subject to} \quad D(a) \leq D_{\text{budget}} \quad (6)$$

Constraint (6) couples the tiles: whether a tile can afford to leave early depends on how much of the budget the other 39 have used. The standard way out is to buy the constraint off with a price $\lambda \geq 0$ charged per unit of compute:

$$L(a, \lambda) = D(a) + \lambda C(a) = \frac{1}{N} \sum_{t=1}^N [D_{t,a(t)} + \lambda c_{a(t)}] \quad (7)$$

λ is an exchange rate, in squared error per unit of relative compute, so λc_k converts the compute an exit uses into the squared error that compute is deemed to be worth. Below it runs from about 2×10^{-7} to about 1×10^{-4} , against squared errors of order 1×10^{-4} on a 0 to 1 scale. What matters is where λ sits relative to the differences between exits.

At $\lambda = 0$ compute is free and every tile takes whichever exit has the smallest error. As λ grows the compute term grows fastest for the deepest exit, so a tile drops to a shallower one once the compute that exit saves, times the price, exceeds the error it adds. Past a large enough λ every tile has dropped as far as the ladder allows.

Proposition 1 (separation)

For every $\lambda \geq 0$, an assignment minimises $L(a, \lambda)$ over all assignments if and only if it minimises the bracket tile by tile:

$$k_t^*(\lambda) \in \arg \min_{k \geq j} [D_{t,k} + \lambda c_k] \quad (8)$$

Assumes (A1) and (A2).

Proof. Write $f_t(k) = D_{t,k} + \lambda c_k$. Then $N L(a, \lambda) = \sum_t f_t(a(t))$, a sum in which the t -th term depends on a only through $a(t)$. For any assignment a , term by term $f_t(a(t)) \geq \min_k f_t(k)$, so $N L(a, \lambda) \geq \sum_t \min_k f_t(k)$, and the right-hand side is attained by any a that picks a minimiser in every tile. Conversely if a is optimal and some tile had $f_t(a(t)) > \min_k f_t(k)$, replacing $a(t)$ by that minimiser and leaving every other tile alone would strictly lower L .

Remark. The result is about the frame mean and bounds no single tile. At the deployed 0.1 dB operating point the per-tile penalty over 1,765 tiles has a mean of 0.11 to 0.16 dB by rate, a 99th percentile of 0.54 to 1.18 dB and a worst tile of 1.97 dB, so a frame inside its budget can hold a tile an order of magnitude outside it. No result in this section repairs that.

Per-tile tail from results/supp_opquality_PAPER.json, checkpoint runs/RECIPES12/ckpt_PAPER.pth.tar, 53 sequences at five rates, at the operating point read from results/signalled_RECIPES12_ctc53.json.

The N -dimensional search has become N independent searches over 4 numbers each. Ties are broken toward the shallower exit, which makes k^* a function and costs nothing. We call $k_t^*(\lambda)$ the oracle's choice, because it uses the true $D_{t,k}$, which only the encoder has. The construction is Shoham and Gersho's [28] with compute in place of rate, made standard in coding by [29].

Proposition 2 (monotonicity)

Let $\lambda_1 < \lambda_2$ and let a_1, a_2 be the corresponding oracle assignments. Then $C(a_2) \leq C(a_1)$ and $D(a_2) \geq D(a_1)$. Assumes (A1), (A2) and Proposition 1.

Proof. Fix a tile and drop the index. Optimality at each price gives $D_1 + \lambda_1 c_1 \leq D_2 + \lambda_1 c_2$ and $D_2 + \lambda_2 c_2 \leq D_1 + \lambda_2 c_1$, where (D_i, c_i) is the pair chosen at λ_i . Adding the two and cancelling $D_1 + D_2$ leaves $(\lambda_2 - \lambda_1)(c_2 - c_1) \leq 0$, hence $c_2 \leq c_1$. Substituting that back into the first inequality gives $D_2 \geq D_1 + \lambda_1(c_1 - c_2) \geq D_1$. Averaging over tiles preserves both.

Remark. Monotone is not continuous. Cost moves in the jumps Proposition 6 measures, so bisecting λ converges on a reachable level rather than on the budget. The result also assumes exact minimisation of the true table, whereas the deployed search bisects a predicted table and corrects with a real decode, at most six passes to 5×10^{-4} dB.

Monotonicity is what makes the budget searchable: bisecting λ against the measured decibels converges on the largest price whose allocation still meets the budget, and that is also the cheapest such allocation the family contains. Every λ quoted in this project is found that way, and so is the router's β .

tile	bits	D(2)	D(3)	D(4)	D(5)	exits as λ rises
34	10,446	1.608	1.638	1.679	1.655	2 throughout
0	12,996	9.346	9.249	9.207	9.180	5, 4 at 1.9, 3 at 3.0, 2 at 6.5
18	16,989	16.249	15.795	15.638	15.606	5, 4 at 2.3, 3 at 11.2, 2 at 30.6

Table 25. Three tiles of one frame, and what the price does to them. D is in units of 10^{-5} squared error and switch points in units of 10^{-6} of λ . The cheapest tile prefers the shallowest exit at every price including $\lambda = 0$, its error not falling with depth at all, so depth is not always worth buying. The most expensive tile holds out against the shallowest exit 4.7 times longer than the middle one, which is why one uniform depth cannot suit both.

results/tile_table.json (Bosphorus, q32, 40 tiles), checkpoint runs/RECIPES12/ckpt_eval.pth.tar. The 60 prices stored in that file produce 33 distinct savings, the granularity C.3 quantifies.

C.3. The achievable set, and what one price reaches

The pairs $(C(a), D(a))$ an allocation can produce form a set whose shape decides both what a sweep can reach and what adaptivity is worth. There are two versions of it. Write

$$\bar{D}_k = \frac{1}{N} \sum_{t=1}^N D_{t,k} \quad (9)$$

for the exit mean at k , the one place tiles are averaged before anything is chosen.

Proposition 3 (fixed proportions)

Suppose tiles are assigned to exits in proportions p , one probability per exit, independently of their content. Then the achievable set of expected pairs is exactly the convex hull of the K -j points whose coordinates are the cost c_k and the exit mean at k . Assumes (A1) and (A2).

Proof. Under content-independent proportions the expected cost is $\sum_k p_k c_k$ and, by the tile mean, the expected distortion is $\sum_k p_k$ times the exit mean at k . The map from p to that pair is affine and its domain is the probability simplex, whose extreme points are the unit vectors. An affine image of a simplex is the convex hull of the images of its vertices, and those images are the cost and exit mean of each exit.

Remark. The hull is a hull of expectations. One frame with N tiles realises proportions in multiples of $1/N$, so interior points are attained only to that granularity, and at $N = 2$ the smallest test class reaches three mixtures per pair of exits.

q	exit 2	exit 3	exit 4	exit 5	best single
q0	0.1791	0.0932	0.0589	0.0361	3 at 24.2%
q16	0.2194	0.1181	0.0755	0.0452	4 at 13.0%
q32	0.2819	0.1472	0.0897	0.0543	4 at 13.0%
q48	0.3330	0.1757	0.1030	0.0622	5 at -1.0%
q63	0.3955	0.2057	0.1156	0.0716	5 at -1.0%

Table 26. The vertices of the fixed-proportion set, in dB below the released decoder, one uniform-depth decode per column. These four points and their hull are everything a content-blind allocation can reach. The last column is the shallowest uniform depth that still meets a 0.1 dB budget together with its saving; at the two highest rates that is the deepest exit, which saves nothing.

results/static_RECIPES12_b01.json, checkpoint runs/RECIPES12/ckpt_PAPER.pth.tar, 53 frames. The mean over rates of the best single depth is 9.7%, against 21.5% for the per-tile allocation at the same budget.

Theorem 4 (per-tile assignment)

The convex hull of the pairs achievable by per-tile assignment is the Minkowski average of the N per-tile hulls,

$$\mathcal{A}_{\text{tile}} = \frac{1}{N} \bigoplus_{t=1}^N \text{conv}\{(c_k, D_{t,k}) : k \geq j\} \quad (10)$$

Assumes (A1) and (A2).

Proof. By (A1) and (A2) an achievable pair is the average of one point drawn from each per-tile set S_t , the set of pairs cost c_k against error $D_{t,k}$, so the achievable set is the Minkowski average of the S_t . Taking convex hulls commutes with Minkowski addition, the hull of a sum being the sum of the hulls, and with positive scaling; applying that $N-1$ times moves the hull inside the sum.

Remark. The theorem bounds what any allocation reaches, a perfect router included, and says nothing about which of those points a price reaches, which is Proposition 5. It is also the one result here with no measurement behind it: the set has up to $N(K-j-1) = 120$ vertices at 1080p and no experiment enumerates them.

Proposition 3 is the special case in which every tile has the same row. Otherwise the per-tile set is strictly larger, and the difference shows in the vertex count: the fixed-proportion hull has at most $K-j = 4$ vertices and so 3 straight segments, while the Minkowski average is smooth at any plotted scale. Figure 7 draws both on one frame.

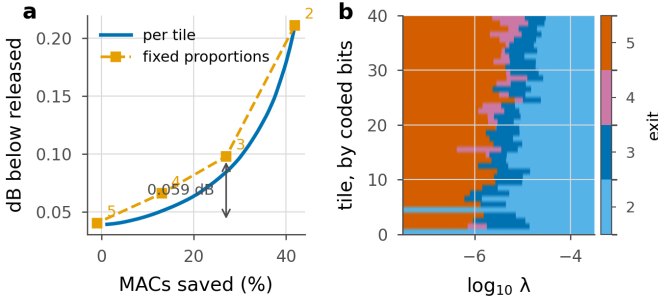


Figure 7. The two achievable sets, on one frame. **a**, the per-tile frontier of Theorem 4 traced by sweeping λ , against the fixed-proportion chain of Proposition 3 through the four uniform-depth points, labelled by exit. The per-tile curve lies below the chain everywhere between the vertices, by 0.059 dB at the exit-3 vertex. **b**, the exit each of the 40 tiles takes as λ rises, tiles ordered by the bits the entropy coder spent on them. Every row is non-increasing, which is Proposition 2 per tile, and the rows differ, which is what Theorem 4 buys over Proposition 3. The bottom row is a tile that never leaves the shallowest exit.

Both panels from results/tile_table.json (Bosphorus, q32, 40 tiles), checkpoint runs/RECIPE512/ckpt_eval.pth.tar, drawn by scripts/supp_derivations_figure.py on that file's own cost vector, the last column of Table 24. A grid of 4001 prices finds 93 distinct reachable savings on this frame, against the 95 the dynamic programme of Table 27 enumerates.

Proposition 5 (a price reaches only the lower hull)

For every $\lambda \geq 0$ the point (C, D) produced by the oracle assignment lies on the lower convex hull of the achievable set, and no λ produces a point strictly inside it. *Assumes* (A1), (A2) and Proposition 1.

Proof. By Proposition 1 the oracle assignment minimises $D(a) + \lambda C(a)$ over all assignments, so its pair minimises the linear functional $(C, D) \rightarrow D + \lambda C$ over the achievable set and hence over that set's convex hull. A minimiser of a linear functional over a compact convex set lies on its boundary, and because $\lambda \geq 0$ the supporting line has slope $-\lambda \leq 0$, which places the point on the lower boundary. Conversely a point strictly inside the hull is a strict convex combination of achievable points and is therefore strictly beaten, at that same C , by some point of the hull, so it cannot minimise any such functional.

Remark. A budget between two hull vertices is unreachable by any price, and the proposition does not say what that costs, which is Proposition 6. The measurement below enumerates the exact Pareto set on one frame at one rate and on the earlier cost vector.

Everett's generalised multiplier method (1963) supplies the reassuring half of the converse: whatever compute the returned allocation consumes, it is optimal for that level, so a sweep never returns something dominated. With four distinct exit costs the whole Pareto set can be enumerated by dynamic programming over tiles, so the other half can be priced exactly.

budget dB	by sweep	exact Pareto	gap
0.05	12.398	12.421	0.0230
0.08	25.443	25.489	0.0452
0.10	30.475	30.496	0.0209
0.12	33.782	33.803	0.0211
0.15	37.439	37.461	0.0211
0.18	40.048	40.046	-0.0018
0.20	41.166	41.165	-0.0016

Table 27. What convexity costs. Savings in per cent at seven budgets on one frame. The sweep reaches 95 allocations against the 635 on the exact Pareto set, and the largest loss over these budgets is 0.045 saving points. The two negative entries are the sweep landing on a budget the enumeration's grid straddles rather than the sweep beating the Pareto set, which Proposition 5 forbids.

results/hull_gap.json, Bosphorus q32, 40 tiles, on the earlier cost vector; the enumeration and the sweep use the same vector, so the gap is unaffected by the correction.

Proposition 6 (granularity)

If at most m tiles switch exit at any single breakpoint of the sweep, and each switch moves a tile by one rung, then consecutive reachable savings differ by at most

$$\text{spacing} \leq \frac{100 m \max_k (c_{k+1} - c_k)}{N} \% \quad (11)$$

and the loss from convexity cannot exceed the largest spacing. *Assumes* (A1), (A4) and Proposition 2.

Proof. Between breakpoints the assignment is constant, so the reachable costs are exactly the costs at the breakpoints. At one breakpoint the mean cost changes by $1/N$ times the sum of the individual changes, each at most $\max_k (c_{k+1} - c_k)$ in magnitude, and there are at most m of them; multiplying by 100 puts it in saving points. For the second part, let a budget fall strictly between two reachable levels. By Proposition 2 the bisection returns the lower level, so the loss is at most the distance between them.

Remark. Both hypotheses are read off a measured table rather than guaranteed, and the bound is vacuous if either fails on another frame. The expression falls as $1/N$, so it says nothing useful at the smallest test class, where $N = 2$ and one switch moves the frame by half the largest rung.

On the measured table $m = 2$, tiles with identical rows switching together, the largest rung is 0.1491 and $N = 40$, so the bound is 0.7453 saving points. The measured spacing is 0.7453, sitting on the bound, and the convexity loss of Table 27 is 0.0452. Nothing in the expression depends on content or on rate, and halving the tile side would put four times as many tiles on the lattice, each carrying a quarter of the step.

m , the largest rung, the bound and the measured spacing from results/theory_checks.json (Bosphorus q32, 40 tiles, runs/RECIPE512/ckpt_eval.pth.tar), written by scripts/verify_theory.py.

C.4. Floor, saturation and ceiling

The sweep has two ends, and a budget outside them buys nothing. The budget section measures where they sit at each rate; this one derives them.

Proposition 7 (the floor)

At $\lambda = 0$ the allocation attains $D_{\min} = (1/N) \sum_t \min_k D_{t,k}$, and no allocation attains less. *Assumes* (A2) and Proposition 1. *Proof.* Immediate from Proposition 1 at $\lambda = 0$, and from $D(a)$ being a mean of terms each bounded below by its own tile minimum.

Remark. The floor is a statement about the table, and the table is a product of training, which (A3) does not constrain. If the deepest exit drifts from the reference by δ then the achievable set shifts up by δ before any compute is saved, and a budget below δ is unreachable at every allocation. With the training objective's anchor term the drift is 0.005 to 0.036 dB over q0 to q63; without it, 0.110 to 0.159 dB, which alone consumes more than the headline budget.

Anchored drift from results/supp_anchor_PAPER.json, checkpoint runs/RECIPE512/ckpt_PAPER.pth.tar, 53 frames. Unanchored drift from results/anchor_VERBATIM.json, checkpoint runs/VERBATIM/ckpt_eval.pth.tar, 40 frames, a different training run at three rates. The two are not a controlled pair; the direction is what the argument needs and each interval is quoted as measured.

Because $D_{t,k}$ is measured on the deployed tiled path, $D_{\min}$ is not zero: it is what tiling costs before any tile exits early. Over the dense sweep the floor exceeds the budget in 3 of 45 budget-and-rate cells, all of them at 0.05 dB.

results/signalled_RECIPE512_grid.json, checkpoint runs/RECIPE512/ckpt_PAPER.pth.tar, 53 sequences, 2 frames per sequence, 9 budgets by 5 rates. Floor and saturation run 0.036 to 0.072 dB and 0.179 to 0.396 dB across the five reported rates (results/saturation_RECIPE512_ctc53.json, same checkpoint).

Proposition 8 (saturation)

Define

$$\lambda_{\text{sat}} = \max_t \max_{k>j} \frac{D_{t,j} - D_{t,k}}{c_k - c_j} \quad (12)$$

Then for every $\lambda > \lambda_{\text{sat}}$ the oracle assigns exit j to every tile, the frame's cost is exactly c_j , and no larger price changes anything. Assumes (A1), (A4) and Proposition 1.

Proof. Exit j beats exit k on tile t precisely when $D_{t,j} + \lambda c_j \leq D_{t,k} + \lambda c_k$, that is when $\lambda \geq (D_{t,j} - D_{t,k})/(c_k - c_j)$, the denominator being positive for $k > j$ by (A4). Taking the maximum over k and then over t gives a single price beyond which exit j wins everywhere. The cost of the constant map is c_j by (5).

Remark. λ_{sat} is a maximum over the tiles of one frame, so a price chosen for a sequence saturates some of its frames and not others, and it constrains the price rather than the budget. On the worked frame the closed form gives 3.045×10^{-5} , exactly the last switch of the most expensive tile in Table 25.

Proposition 9 (the ceiling)

The largest saving any allocation can reach is

$$S_{\text{max}} = 100(1 - c_j) \% \quad (13)$$

Assumes (A1), (A4) and Proposition 8. *Proof.* $C(a)$ is a mean of costs each at least c_j , j being the shallowest usable exit, so $C(a) \geq c_j$ with equality only for the constant map, which Proposition 8 shows is attained.

Remark. $S_{\text{max}} = 39.1\%$ depends on the split depth alone, so no amount of training raises it and the only lever is j . It is a ceiling in multiply-accumulates and not in seconds; the complexity section reports the wall-clock saving falling short of it. Over the 18 class-and-rate cells measured at a saturating budget the reported ceiling spreads by 6e-06 saving points, which is single-precision noise.

Per-class invariance from results/per_class_RECIPES12.json at budgets of 0.5 dB and above, checkpoint runs/RECIPES12/ckpt_PAPER.pth.tar; the ceiling itself is 39.1%, from results/saturation_RECIPES12_ctc53.json.

C.5. What adaptivity is worth

What content awareness is worth can be answered without building a predictor. Compare the best per-tile allocation with the best content-blind one at the same price:

$$J_{\text{ad}}(\lambda) = \frac{1}{N} \sum_t \min_k [D_{t,k} + \lambda c_k], \quad J_{\text{fix}}(\lambda) = \min_k [\bar{D}_k + \lambda c_k] \quad (14)$$

J_{ad} takes the minimum inside the average and J_{fix} takes it outside. By Propositions 1 and 3 the first is the best the per-tile family attains at price λ and the second the best the fixed-proportion family attains.

Theorem 10 (adaptivity gain)

For every $\lambda \geq 0$, $\Delta(\lambda) = J_{\text{fix}}(\lambda) - J_{\text{ad}}(\lambda) \geq 0$, with equality if and only if some single exit k^* attains the per-tile minimum for every tile. Assumes (A1) and (A2).

Proof. Fix λ and let k be any exit. Then the exit mean at k plus $\lambda c_k = (1/N) \sum_t [D_{t,k} + \lambda c_k] \geq (1/N) \sum_t \min_{k'} [D_{t,k'} + \lambda c_{k'}] = J_{\text{ad}}(\lambda)$, because each summand dominates the corresponding minimum. The right-hand side does not depend on k , so taking the minimum over k on the left preserves the inequality and gives $J_{\text{fix}} \geq J_{\text{ad}}$. For equality, suppose some k^* attains the per-tile minimum everywhere; then choosing it makes every summand equal to its minimum and the inequality is tight. Conversely suppose $\Delta = 0$ and let k^* attain J_{fix} . Then $(1/N) \sum_t (D_{t,k^*} + \lambda c_{k^*}) - \min_{k'} [D_{t,k'} + \lambda c_{k'}] = 0$. Every summand is non-negative, so every summand is zero, which says exactly that k^* attains the minimum on every tile.

Remark. Δ upper-bounds what any predictor gains over a content-blind allocation at that price and says nothing about whether one can get any of it, which is what C.7 measures. It is also a property of the ladder it is

measured on: improving the shallow exits compresses the differences between exits, more tiles agree on the argmin, and Δ falls even as the saving rises, so it is not a figure of merit for a decoder.

Two things follow. Δ depends only on the table $D_{t,k}$ and the costs, so it is computable before any router exists. And its equality condition is a condition on the data rather than on the method: when every tile agrees on the best exit there is nothing to adapt to.

q	$\lambda=10^{-5}$	3×10^{-5}	10^{-4}	3×10^{-4}
q0	0.30%	0.39%	0.45%	0.03%
q16	0.65%	0.74%	0.38%	0.01%
q32	0.62%	1.41%	0.37%	0.00%
q48	0.62%	2.00%	0.30%	0.00%
q63	0.68%	2.09%	0.27%	0.00%

Table 28. The adaptivity gain in scale-free form, $\Delta(\lambda)$ as a percentage of $J_{\text{ad}}(\lambda)$. At the two smaller prices it rises with rate, so adaptivity is worth most where the deep blocks are doing real work. At the largest price it is near zero at every rate, which is Theorem 10's equality condition arriving: the oracle has itself collapsed onto one exit and nothing is left for a router to recover.

results/theory_check.json, checkpoint runs/BEST/ckpt_eval.pth.tar, 40 sequences (UVG, MCL-JCV and HEVC class E; classes B, C and D not measured), on the earlier cost vector of Table 24. This is the one table in the section not on the pinned checkpoint, and nothing on the pinned checkpoint currently reproduces it.

q	blind %	oracle %	gap	blind dB	shuffled dB
q0	25.38	29.90	4.52	0.1262	0.1248
q16	19.45	25.63	6.18	0.1278	0.1335
q32	15.02	20.93	5.91	0.1304	0.1411
q48	12.00	18.28	6.27	0.1373	0.1451
q63	8.05	15.62	7.57	0.1367	0.1413

Table 29. The same gap in the units the paper reports. Columns 2 to 4 are savings at a 0.1 dB budget: the best content-blind mixture of uniform depths, the per-tile oracle, and the difference. Columns 5 and 6 are decibels at the oracle's own compute, what the best blind mixture would read there and what a random shuffle of the oracle's exit map actually reads. Adaptivity is worth 4.5 to 7.6 saving points, rising with rate, against an oracle reading 0.100 dB and a shuffle reading up to 0.141 dB.

Computed from results/static_RECIPES12_b01.json, checkpoint runs/RECIPES12/ckpt_PAPER.pth.tar, 53 frames. The blind columns are the lower hull of the four uniform-depth points of Table 26 under Proposition 3, evaluated at the budget and at the oracle's compute; the shuffle column is a measured decode. At q0 the shuffle reads slightly better than the hull predicts, which bounds the interaction (A2) ignores at under 0.002 dB.

C.6. Convexity in the units the frontier is plotted in

Theorem 4 gives D convex in C . The frontier is almost never plotted in those units: it is plotted as decibels against percentage saving, where convexity is a different statement, the logarithm being concave and increasing and so preserving neither convexity nor concavity. With $S = 1 - C/c_{K-1}$,

$$\text{dB}(S) = \frac{10}{\ln 10} \ln D(C), \quad C = (1 - S)c_{K-1} \quad (15)$$

Proposition 11 (plotted convexity is log-convexity)

Let D be twice differentiable along the frontier. Then $\text{dB}(S)$ is convex if and only if D is log-convex in C :

$$DD'' \geq (D')^2 \iff \frac{d}{dC} \left(\frac{1}{\lambda} \right) \geq \frac{1}{D} \quad (16)$$

Assumes twice differentiability along the frontier, and Proposition 5 for the second form.

Proof. C is affine in S and convexity is preserved under affine reparametrisation, so by (15) dB is convex in S if and only if $\ln D$ is convex in C . Differentiating twice, $(\ln D)'' = D''/D - (D'/D)^2 = [D D'' - (D')^2]/D^2$, and $D > 0$, so $(\ln D)'' \geq 0$ if and only if $D D'' \geq (D')^2$. For the second form, the slope of the frontier is $D' = -\lambda$ by the supporting-line argument of Proposition 5, so $D'' = -\lambda'$; substituting gives $-\lambda' D \geq \lambda^2$, and

dividing by $\lambda^2 > 0$ gives $-\lambda'/\lambda^2 \geq 1/D$, whose left-hand side is $d(1/\lambda)/dC$. The second form reads as a rate condition: distortion has to fall at least exponentially in compute for the plotted curve to be convex.

Remark. This is a property of the decoder rather than a consequence of Theorem 4, and a convex D can fail it. Differentiability also fails in one of the two cases: under fixed proportions D is piecewise linear with K -j vertices, so $\ln D$ is piecewise concave with upward jumps at the vertices and the shape one sees depends on where one samples, whereas the per-tile hull is smooth at any plotted scale. Treating one curve as the other is how a confident claim about frontier shape becomes an artefact of vertex spacing.

q	points	condition holds	margin	quartic fit
q0	16	100.0%	0.0024	84.0%
q16	14	100.0%	0.0050	76.3%
q32	13	100.0%	0.0088	75.3%
q48	13	100.0%	0.0145	73.7%
q63	13	100.0%	0.0230	74.7%

Table 30. Testing log-convexity on the measured frontier. The direct test asks whether the secant slopes of $\ln D$ against S are non-decreasing, and the margin is the smallest increase attained. The condition holds at every point of every frontier and the margin grows tenfold from the lowest rate to the highest. Fitting a quartic and differentiating it twice instead reports 73.7% to 84.0%, and the difference is the fit rather than the data: near the ceiling the frontier is close to vertical and a polynomial overshoots, whereas the secant test has no fitted degree to choose.

results/logconvexity.json. That file records no checkpoint, so it cannot be attributed from disk; its five frontiers have 13 to 16 points each. The quartic column is the file's own quartic_frac_ok field.

C.7. How much of the oracle a partial signal recovers

The decoder cannot run the argmin of Proposition 1, $D_{t,k}$ being an error against a source it never receives. A predictor can guess it. Writing $p_{t,k}$ for a head's probability that tile t belongs at exit k , its rule is

$$\hat{k}_t = \arg \max_k [\log p_{t,k} - \beta c_k] = \arg \min_k \left[\frac{-\log p_{t,k}}{\kappa} + \lambda c_k \right] \quad (17)$$

The second form of (17) is the same rule with $\beta = \lambda \kappa$, κ being a fixed calibration turning nats into squared error. Written that way it says that a router is the oracle of (8) with a surrogate in place of $D_{t,k}$, and that one price governs both. Define the per-tile regret at a fixed price:

$$r_t(\lambda) = [D_{t,\hat{k}_t} + \lambda c_{\hat{k}_t}] - [D_{t,k^*} + \lambda c_{k^*}] \geq 0 \quad (18)$$

Non-negativity is immediate from (8). The regret is what the frame's Lagrangian loses on tile t by trusting the prediction, and by Proposition 1 those losses add. That gives the encoder an option the two extremes do not have: signal the exits of a subset S of the tiles and let the head decide the rest, which is deferral in the sense of [39, 40] with the cost of deferring made explicit. Signalling S removes exactly $\sum_{t \in S} r_t$ from the objective.

Proposition 12 (Lorenz bound)

Fix λ and let $r_{(1)} \geq r_{(2)} \geq \dots \geq r_{(N)}$ be the regrets in decreasing order with total R . For any subset S of size s , the fraction of the total regret that signalling S removes is at most

$$L(\rho) = \frac{1}{R} \sum_{i=1}^{\lceil \rho N \rceil} r_{(i)}, \quad \rho = s/N \quad (19)$$

with equality if and only if S is a set of s largest regrets. L is the Lorenz curve of the regret distribution: non-decreasing, concave, running from 0 to 1, and lying above the diagonal by an amount the Gini coefficient summarises. Assumes (A1), (A2) and a fixed λ .

Proof. The sum of any s of the r_t is at most the sum of the s largest, since replacing an element of S by a larger one outside S cannot decrease the sum, and repeating that exchange terminates at a set of s largest values.

Dividing by R gives the bound. Concavity holds because the increments $r_{(i)}$ are non-increasing by construction.

Remark. The bound holds at a fixed price, and the deployed system re-bisects λ at every ρ so the frame lands back on its budget. Re-bisecting enlarges the feasible set: the overridden tiles and the predicted ones then carry two multipliers, and pairs of multipliers reach allocations no single one does. The measured recovery is therefore not bounded by L , and Proposition 12 is the wrong thing to call a bound on the deployed system.

The practical content is that an unequal regret distribution can be repaired cheaply. If a tenth of the tiles carry half the regret then signalling a tenth of them removes half of what the predictor gives up, at a tenth of the bits.

q	Gini	L(.1)	rec(.1)	L(.2)	rec(.2)	L(.5)	rec(.5)
q0	0.53	35%	29%	51%	53%	87%	103%
q16	0.49	27%	33%	46%	59%	87%	102%
q32	0.61	37%	40%	59%	68%	95%	106%
q48	0.67	45%	39%	65%	63%	96%	100%
q63	0.73	53%	39%	72%	62%	99%	94%

Table 31. The Lorenz curve of the regret, and what signalling recovers. $L(\rho)$ is the share of the total regret carried by the ρ worst tiles, which Proposition 12 says is the most any signal of that size can remove at a fixed price; $\text{rec}(\rho)$ is the share of the predictor-to-oracle gap the measured sweep recovers. Regret is concentrated at every rate, Gini 0.49 to 0.73: a tenth of the tiles carries 27% to 53% of the total regret and recovers 29% to 40% of the gap. Over every reachable cell the ratio rec/L runs from 46% to 127%, for the reason the remark gives.

results/hybrid_RECIPES12_b01_fixed.json, checkpoint runs/RECIPES12/ckpt_PAPER.pth.tar, 53 sequences at a 0.1 dB budget; the head it is measured against was trained on runs/RECIPES12/ckpt_eval.pth.tar. $\text{rec}(\rho)$ is computed here from the same file as the saving at ρ , relative to the $\rho = 0$ and $\rho = 1$ endpoints. Signalling half the tiles beats signalling all of them at 4 of 5 rates, which is the same re-bisection effect.

The signal is an entropy-coded mask over tiles plus an index per override. A fifth of the tiles costs 44 bits per frame and recovers over half the gap. The $\rho = 1$ endpoint, which reproduces the oracle's allocation, costs 100 bits over 33.3 tiles, 3.0 bits a tile against the 2 a fixed-length code over the K -j = 4 reachable exits would need, because at $\rho = 1$ the mask names every tile and is pure overhead. Configuration A signals its own map instead and pays 79 to 95 bits per frame, 0.021% of a typical bitrate.

Map costs from results/hybrid_RECIPES12_b01_fixed.json and results/signalled_RECIPES12_ctc53.json, both on runs/RECIPES12/ckpt_PAPER.pth.tar.

C.8. The rate-rank rule, and the model it comes from

There is a cheaper way to guess the oracle than a trained head, and it is worth deriving rather than presenting as a heuristic, because the derivation says when it must work and how it fails. The decoder holds, before the trunk runs, the bits the entropy coder spent on each tile's latents; write b_t for that count over the frame's mean. The model is separability in $\log$ space: one scalar difficulty per tile, and one ladder profile applying to every tile in proportion.

$$\log D_{t,k} \approx u_t + \log \varphi_k, \quad u_t = \alpha \log b_t + c \quad (20)$$

Equivalently $D_{t,k} = g_t \phi_k$ with $g_t = \exp(u_t) > 0$. The profile ϕ is K -j numbers fitted once per rate and α and c are two more, all three fitted leave-one-sequence-out. Nothing is learned per frame and nothing is transmitted, which is why the rule costs the decoder zero.

Proposition 13 (a rank-1 table gives a threshold rule)

If $D_{t,k} = g_t \phi_k$ with $g_t > 0$ then

$$\arg \min_k [g_t \varphi_k + \lambda c_k] = \arg \min_k \left[\varphi_k + \frac{\lambda}{g_t} c_k \right] \quad (21)$$

so a tile's exit depends on the tile only through the scalar λ/g_t , and by Proposition 2 the chosen exit is non-increasing in it. Sorting tiles by g_t and cutting the sorted list at $K-j-1$ thresholds therefore reproduces the oracle exactly. Assumes exact rank-1 structure, and Proposition 2.

Proof. Divide the bracket by $g_t > 0$, which does not move the argmin. The reduced problem is the one-tile problem of (8) with distortion ϕ and price λ/g_t , so Proposition 2 applies and its solution is non-increasing in that price. Since λ/g_t is decreasing in g_t , the exit is non-decreasing in g_t : harder tiles go deeper. The thresholds are the breakpoints of that one common problem and there are at most $K-j-1$ of them.

Remark. Exactness needs an exactly rank-1 table, which no real table is, and the shortfall of the deployed rule measures the departure. The proposition also says the exit is monotone in g_t and not that a bit count estimates g_t , which is a separate and weaker claim.

Proposition 14 (a rank-1 rule uses only the profile's hull)

Under the same model, an exit k is chosen for some tile at some price only if (c_k, ϕ_k) lies on the lower convex hull of the profile. Assumes exact rank-1 structure, and Propositions 5 and 13. *Proof.* By Proposition 13 the reduced problem is a single Lagrangian sweep over the points (c_k, ϕ_k) , and by Proposition 5 such a sweep reaches only hull points.

Remark. The statement is about the fitted profile rather than the true table, so an exit off the hull is dropped by the rule and may still be the oracle's choice. Hull membership is not automatic: on the single-sequence table of results/tile_table.json the same construction drops exit 4.

q	α	$\phi(2)$	$\phi(3)$	$\phi(4)$	$\phi(5)$	on hull	agree
q0	9.91	1.0242	1.0009	0.9912	0.9841	2,3,4,5	0.453
q16	4.58	1.0251	1.0021	0.9911	0.9822	2,3,4,5	0.320
q32	2.73	1.0297	1.0013	0.9893	0.9804	2,3,4,5	0.283
q48	1.94	1.0350	1.0004	0.9884	0.9771	2,3,4,5	0.353
q63	1.43	1.0494	0.9963	0.9829	0.9731	2,3,4,5	0.392

Table 32. The fitted separable model. α is the exponent relating a tile's coded bits to its difficulty and ϕ is the exit profile, both fitted leave-one-sequence-out; agree is the fraction of tiles on which the resulting rule and the oracle choose the same exit. α is positive at every rate, so a tile the entropy coder spent bits on is a tile every exit reconstructs worse, and it falls 7-fold from the lowest rate to the highest. The profile spans 4% to 8% from its shallowest entry to its deepest, which is the margin the rule works in. All four exits survive on the lower hull at every rate, so Proposition 14 costs nothing here.

results/raterank_RECIPES12_b01.json, checkpoint runs/RECIPES12/ckpt_PAPER.pth.tar, 53 sequences at a 0.1 dB budget. Hull membership computed against the shipped cost vector of Table 24. The router section reports what the rule saves against the trained head.

The rule gives up 3.4 saving points to the trained head, and the loss is not in the separability. On the measured tile table the separable model absorbs 99.96% of the variance of $\log D$. The quantity it has to resolve is small: the exit profile spans 0.0449 in log units while the non-separable residual has a root mean square of 0.0144, 32% of it. A model can explain almost all of a table and still be unable to order the differences that decide the argmin, which reads only the differences. Predicting a tile's difficulty from its bit count alone then leaves $R^2 = 0.84$, and by Proposition 13 an error in $\log g_t$ is a proportional error in the price that tile faces. Coded bits correlate with the mean level of a tile's row at Spearman 0.83 to 0.88, with its spread at 0.61 to 0.72 and with the oracle's exit index at 0.39 to 0.54, the ordering the model predicts.

Residual analysis computed from results/tile_table.json (Bosphorus, q32, 40 tiles), checkpoint runs/RECIPES12/ckpt_eval.pth.tar; α fitted there is 4.69 against 2.73 for the 53-sequence fit at the same rate, the difference being one sequence against 53. Spearman correlations from results/raterank_RECIPES12_b01.json, pinned checkpoint; the file stores the last against the negated exit index and it is reported here with the sign flipped, so a positive value means more bits and a deeper exit.

The rule is the exact oracle for a decoder whose distortion table happens to be rank-1, so its shortfall measures how far from rank-1 the real table is, and Proposition 12 applies to it as it does to any predictor whose regret the encoder can compute.

C.9. What is proved, what is checked, and what neither covers

Table 23 lists every result with its evidence, and four gaps in it are worth naming. Theorem 10, the result the paper leans on hardest, is measured only on runs/BEST/ckpt_eval.pth.tar over 40 sequences and on the earlier cost vector; the pinned version in Table 29 gives it in saving points but not as a ratio. The convexity check of Table 30 records no checkpoint at all. The granularity and hull results of C.3 rest on one sequence at one rate, enough to check arithmetic and not enough to characterise a codec. And every proposition here concerns a frame mean under (A1) and (A2), so none bounds what a single tile gives up; the tail after Proposition 1 fills that in as a measurement rather than a guarantee.

D. Complete results

This section prints in full what the main paper samples: every class at every quality index and at three budgets, the frontier drawn class by class, and the distribution behind each mean, over sequences and over tiles. It also carries three measurements the main paper has no room for: a perceptual metric beside the decibel, the composition with weight quantisation, and the raw numbers behind the curves. It ends with the cases where the method does badly.

D.1. Every class at every rate

Class	q0	q16	q32	q48	q63
MCL-JCV	33.7	30.4	26.2	23.0	20.2
UVG	30.6	28.9	23.1	18.7	15.5
HEVC B	31.1	25.8	18.2	12.4	9.7
HEVC E	29.1	25.2	18.4	15.1	15.5
HEVC C	18.0	6.3	1.7	2.8	2.5
HEVC D	8.8	2.5	2.5	2.5	2.5
All 53, pooled	29.7	25.5	20.9	17.9	15.6
Quality given up (dB)	0.099	0.099	0.100	0.098	0.100

Table 33. Compute saved against the released decoder, by class and by quality index, at the 0.1 dB budget; the main paper prints the q0, q32 and q63 columns of the first six rows. Read it in two directions. Along a row the saving falls with rate, because the residual a shallow exit fails to reconstruct grows relative to the quality being protected. Down a column it falls with resolution, from 33.7% on MCL-JCV to 8.8% on HEVC D at q0, because a 256 px tile is 40 tiles on a 1080p frame and two at 416x240. The last row is the quality the pooled allocation gave up, which is the budget to within a thousandth of a decibel at every rate.

results/supp_per_class_budgets.json, on runs/RECIPES12/ckpt_PAPER.pth.tar, 53 sequences at one frame each, per-frame convention. The pooled row reads 29.7% at q0 where the main paper's 29.2% comes from results/signalled_RECIPES12_ctc53.json; the two bisect the same budget with different code and settle on $\lambda = 5.126 \times 10^{-5}$ and 5.150×10^{-5} .

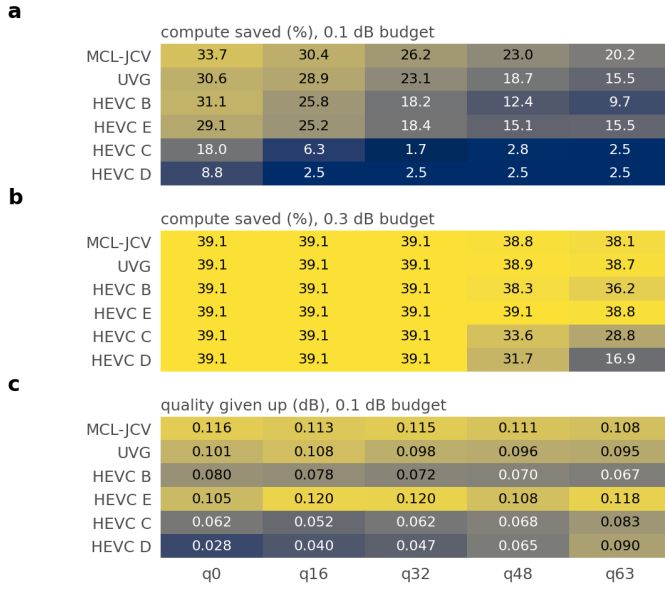


Figure 8. The same measurement as a grid, with the looser budget under it. **a**, compute saved at the 0.1 dB budget. **b**, the same at 0.3 dB, where every class at the three lowest rates has reached the architectural ceiling of 39.1% and only q48 and q63 are still making a choice. **c**, the quality actually given up at the 0.1 dB budget, which the set meets and no class does: at q0 the allocation spends 0.116 dB on MCL-JCV and 0.028 dB on HEVC D. The small classes are cheap in quality for the same reason they are poor in saving, that with two or eight tiles there is no fine-grained way to spend a budget. The 0.5 dB grid is not drawn because every one of its thirty cells is at the ceiling.

Drawn by scripts/supp_results_figs.py from results/supp_per_class_budgets.json, on runs/RECIPE512/ckpt_PAPER.pth.tar.

The resolution effect is a property of the tiling and not of the content, which is the argument against reading a single mean over a test set that mixes resolutions: MCL-JCV is 30 of the 53 sequences and all of them 1080p, so it carries most of the pooled row.

D.2. Where the tiles exit

Rate	e2	e3	e4	e5	Blocks skipped	Saved
q0	67.4	20.7	8.3	3.6	5.04	29.7
q16	58.2	18.6	13.1	10.0	4.50	25.5
q32	43.5	19.5	18.8	18.2	3.77	20.9
q48	34.4	16.8	24.3	24.5	3.22	17.9
q63	22.3	23.1	28.1	26.6	2.82	15.6

Table 34. The exit histogram at the 0.1 dB budget, as a percentage of the 1765 tiles in one pass over the test set, with the mean number of trunk blocks skipped and the saving that buys. At q0 two thirds of tiles take the shallowest selectable exit and the ladder above it is barely used. At q63 the mass is spread almost evenly over the four exits and the mean depth skipped has fallen from 5.04 blocks to 2.82. Exits e0 and e1 are omitted because the exit clamp cannot select them: the split depth is 2, so a map naming a shallower exit runs to e2 and wears e2's adapter.

Histograms summed over the six classes of results/supp_per_class_budgets.json, on runs/RECIPE512/ckpt_PAPER.pth.tar. Blocks skipped per exit from results/adaptor_cost.json, same checkpoint.

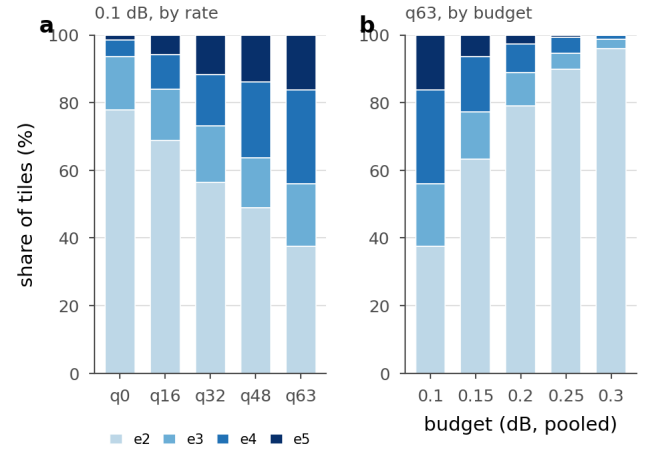


Figure 9. How the ladder fills up. **a**, exit shares at the 0.1 dB budget as the rate rises. **b**, exit shares at q63 as the budget loosens, until at 0.3 dB 96% of tiles are already at the shallowest exit and there is almost nothing left to allocate. Both panels are pooled over the test set.

Drawn by scripts/supp_results_figs.py from results/supp_paper_curve_PAPER.json, on runs/RECIPE512/ckpt_PAPER.pth.tar, pooled convention, so the budgets on the horizontal axis of **b** are looser than the identically labelled ones in the table above.

Class	e2	e3	e4	e5	e2	e3	e4	e5
MCL-JCV	73.8	17.2	5.7	3.4	29.8	23.4	23.5	23.3
UVG	54.3	31.1	14.3	0.4	5.7	31.4	44.6	18.2
HEVC B	64.0	18.5	12.5	5.0	6.5	11.0	37.5	45.0
HEVC E	51.1	35.6	4.4	8.9	15.6	35.6	8.9	40.0
HEVC C	3.1	53.1	31.2	12.5	0.0	0.0	25.0	75.0
HEVC D	0.0	25.0	25.0	50.0	0.0	0.0	25.0	75.0

Table 35. The same histogram split by class at the two ends of the rate range: the first four columns are q0 and the second four are q63, each as a percentage of that class's tiles. HEVC C and HEVC D reach neither of the two shallowest exits at q63, and three quarters of their tiles run to full depth, which is the 2.5% saving of the first table seen tile by tile. UVG behaves differently from MCL-JCV at q63 despite being the same resolution, which is content and not geometry.

results/supp_per_class_budgets.json, on runs/RECIPE512/ckpt_PAPER.pth.tar.

D.3. The frontier, class by class

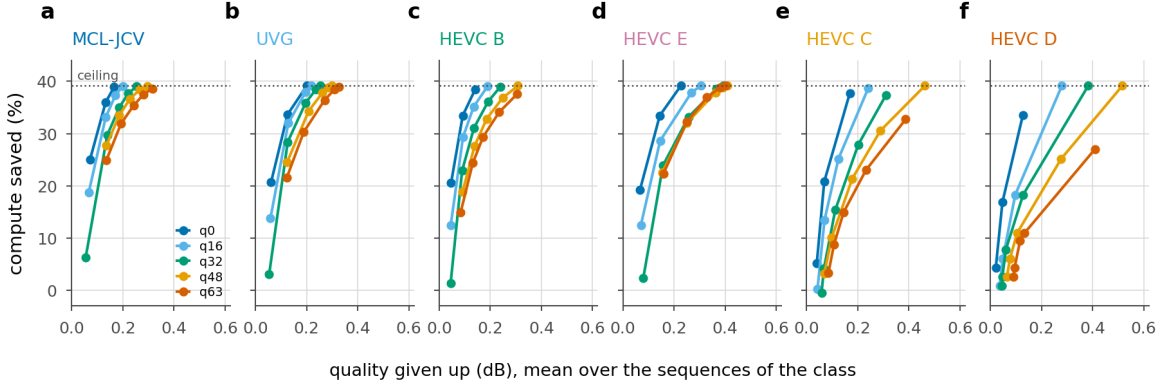


Figure 10. Compute saved against quality given up, one panel per class and one curve per quality index, from the seven budgets at which the frontier was measured. The dotted line is the architectural ceiling of 39.1%. Three things are visible that a table of operating points hides. The curves are steep and then flat, so most of the saving is bought in the first tenth of a decibel and the rest of the band buys little. The rate ordering is preserved at every budget, so a curve never crosses another within a panel. And the panels move to the right as resolution falls: MCL-JCV reaches the ceiling by 0.3 dB at every rate, while HEVC D needs 0.5 dB at q48 and does not reach it at q63 within the measured range.

Drawn by scripts/supp_results_figs.py from results/supp_paper_curve_PAPER.json, on runs/RECIPE512/ckpt_PAPER.pth.tar. A point is the mean over the sequences of that class at a global operating point, so the multiplier is the one the whole set was bisected to and not a per-class retuning. Class membership per sequence from results/supp_opquality_PAPER.json, same checkpoint.

Two limits bound every panel. A budget below the floor cannot be met at all, because our deepest exit already differs from the released decoder by 0.036 dB at q0 and 0.072 dB at q63 in the per-frame convention; a budget above 0.179 dB at q0 or 0.396 dB at q63 changes nothing, because every tile is already at the shallowest selectable exit. Between those two the multiplier is doing work, and the 0.1 dB budget is inside the window at every rate.

D.4. Per-sequence spread

A set mean is not what any single clip receives. One multiplier prices compute for the whole set, so each sequence lands wherever its own content puts it, and the spread of those landings is what a deployment would have to plan against.

Rate	n	Mean	SD	Min	p25	Median	p75	Max
q0	53	32.7	7.5	6.0	28.8	35.9	38.8	39.1
q16	53	28.8	10.6	-1.0	23.6	31.7	37.6	39.1
q32	53	24.6	12.0	-1.0	18.2	26.1	34.3	39.1
q48	53	22.4	11.7	-1.0	13.8	23.5	31.4	39.1
q63	53	20.0	11.1	-1.0	12.7	20.4	27.7	39.1

Table 36. Compute saved per sequence at the 0.1 dB budget: the distribution behind the mean, over all 53 sequences. The interquartile range widens from 9.9 points at q0 to 15.0 at q63, and the gap between best and worst sequence widens from 33.1 to 40.1. The minimum is negative at four of the five rates, because a sequence whose tiles all run to full depth costs 0.95% more than the released decoder: our deepest exit is the more expensive of the two. Counting over rates as well as sequences, 33 of the 265 pairs save less than a tenth of a decode at this budget.

results/supp_per_sequence_PAPER_b010.json, on runs/RECIPE512/ckpt_PAPER.pth.tar, read from its rows_vs_release block so that the denominator is the released decoder. Pooled convention, because the operating points come from results/supp_paper_curve_PAPER.json.

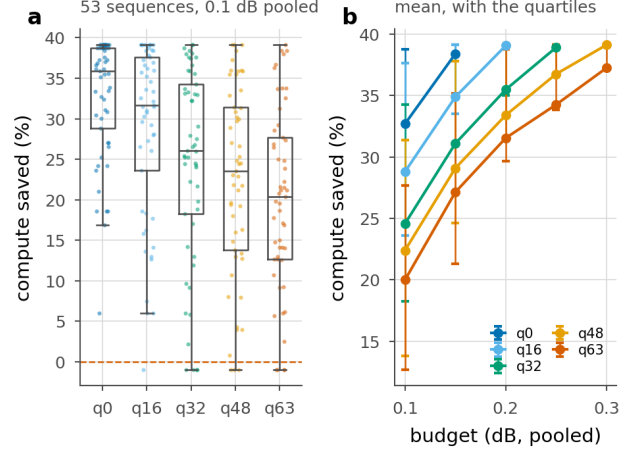


Figure 11. The same distribution drawn. **a**, every sequence at the 0.1 dB budget, one point per sequence, with the quartiles and whiskers over them; the dashed line is the released decoder. **b**, the mean with its quartiles as the budget loosens. A rate disappears from **b** once its frontier saturates, because a saturated operating point has no allocation left to break down.

Drawn by scripts/supp_results_figs.py from the five results/supp_per_sequence_PAPER_b0*.json files, all on runs/RECIPE512/ckpt_PAPER.pth.tar.

Sequence	q0	q63	q63 dB
RaceHorses_832x480	18.6	-1.0	0.094
BlowingBubbles_416x240	18.6	-1.0	0.075
BQSquare_416x240	6.0	-1.0	0.185
RaceHorses_416x240	18.6	-1.0	0.086
PartyScene_832x480	16.9	2.5	0.110
BQMall_832x480	26.5	5.7	0.107
videoSRC27	39.1	38.4	0.210
videoSRC29	39.1	39.1	0.194

Table 37. The six hardest sequences at q63 and the two easiest, with their q0 value beside them and the decibel each actually received. All six of the hard ones are 832x480 or 416x240. The delivered decibel is not the budget on any of them: across the 53 sequences at q63 it runs from 0.013 dB to 0.237 dB against a 0.1 dB target, so a single multiplier for the set means some clips overshoot the budget by more than a factor of two while others use an eighth of it.

results/supp_per_sequence_PAPER_b010.json, on runs/RECIPE512/ckpt_PAPER.pth.tar. Savings are against the released decoder; the decibel is the sequence's own, in the pooled convention.

D.5. A second metric, and the tiles that pay for the mean

The whole mechanism is denominated in PSNR, and PSNR is known not to track visual quality closely. Two things follow that are worth measuring rather than asserting: whether a perceptual metric agrees that

the loss is small, and how the loss is distributed over tiles, since a mean of 0.1 dB is compatible with a few tiles losing a great deal.

Rate	bpp	PSNR	ours	lost	MS-SSIM	ours	lost
q0	0.2313	32.536	32.435	0.1018	0.89934	0.89813	0.0520
q16	0.2671	35.335	35.235	0.1001	0.94904	0.94850	0.0455
q32	0.3352	38.135	38.035	0.1004	0.97565	0.97542	0.0399
q48	0.4561	40.777	40.674	0.1026	0.98790	0.98779	0.0376
q63	0.6717	43.155	43.037	0.1181	0.99419	0.99413	0.0464

Table 38. PSNR and MS-SSIM at the 0.1 dB operating point, over the whole test set. MS-SSIM agrees with PSNR about the direction and is kinder about the size: expressed as $-10\log_{10}(1 \text{ minus MS-SSIM})$, the loss is 0.052 dB at q0 against 0.102 dB of PSNR, and 0.046 against 0.118 at q63, so on this metric the budget costs about half what the decibel it is written in suggests. Each metric is given as the released decoder, ours, and what was lost, with the MS-SSIM loss expressed as $-10\log_{10}(1 \text{ minus MS-SSIM})$ so that the two loss columns are in the same unit. The bpp column is the released bitstream, which early exiting does not change beyond the exit map.

results/supp_opquality_PAPER.json, on runs/RECIPE512/ckpt_PAPER.pth.tar, 53 sequences at one frame each. MS-SSIM is Wang et al. 2003 at five scales with an 11-tap Gaussian of $\sigma = 1.5$ on the valid region, on the luma plane in 4:2:0 on 0 to 255, the domain the PSNR column uses; the implementation self-checks at start-up and the file records that identical inputs score 1.000000 and a 3x3 box blur 0.7797. λ is read from results/signalled_RECIPE512_ctc53.json so the allocation is the deployed one, and the delivered decibel recomputed here matches the stored value to four decimals.

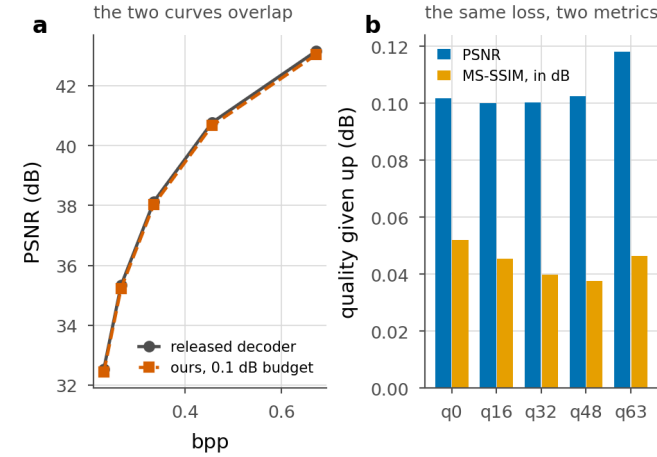


Figure 12. a, the rate-quality curve of the released decoder and of ours at the 0.1 dB budget, over the five quality indices. At plot scale they are one curve, which is what a 0.1 dB budget means. b, the gap that panel a cannot show, in PSNR and in MS-SSIM expressed as a decibel.

Drawn by scripts/supp_results_figs.py from results/supp_opquality_PAPER.json, on runs/RECIPE512/ckpt_PAPER.pth.tar.

Rate	Tiles	Mean	Median	p95	p99	Max	Over 0.25	Over 0.5	Over 1.0
q0	1765	0.164	0.109	0.463	1.184	1.971	302	79	26
q16	1765	0.153	0.106	0.480	0.852	1.495	267	80	7
q32	1765	0.142	0.104	0.400	0.671	1.587	250	50	2
q48	1765	0.151	0.100	0.441	0.934	1.636	237	74	17
q63	1765	0.111	0.098	0.288	0.539	1.264	136	21	4

Table 39. How the 0.1 dB is distributed over the 1765 tiles of one pass over the test set. The last three columns count tiles losing more than a quarter, a half and a whole decibel. The median tile is at or below the budget at every rate, and the tail is long: at q0, 302 tiles lose more than 0.25 dB, 79 lose more than 0.5 dB, and the worst loses 1.97 dB. A budget stated as a mean says nothing about the last three columns, which is why they are printed.

results/supp_opquality_PAPER.json, on runs/RECIPE512/ckpt_PAPER.pth.tar, at the same operating points as the metric table above. A tile penalty is the decibel between the released decode of that tile and ours.

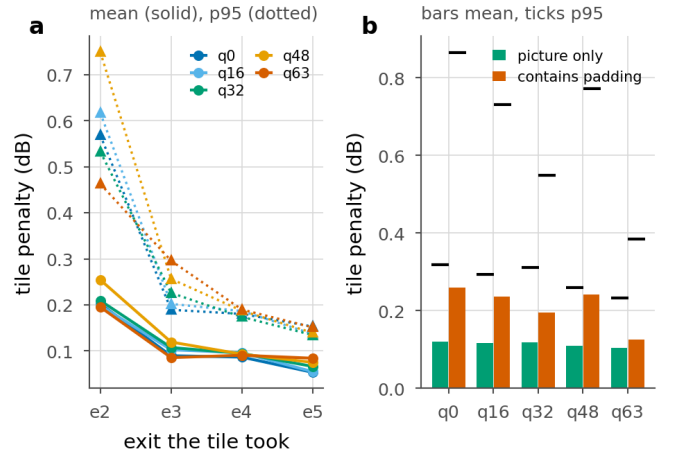


Figure 13. Where the tail comes from. a, mean and 95th percentile tile penalty against the exit the tile took. The tail sits on e2, the shallowest exit the clamp allows: at q0 its mean is 0.202 dB against 0.054 dB at e5, and all 79 tiles above 0.5 dB took it. b, the same penalty split by whether the tile contains replicated padding. A 1080p frame is padded to 2048x1280, so 200 of the bottom tile row's 256 rows are invented by replication, and those tiles carry roughly double the mean penalty at every rate: 0.261 dB against 0.121 dB at q0, and 65 of the 79 tiles above half a decibel are padded ones.

Drawn by scripts/supp_results_figs.py from the tail_by_exit and tail_by_padding blocks of results/supp_opquality_PAPER.json, on runs/RECIPE512/ckpt_PAPER.pth.tar. A tile counts as padded if any of its pixels came from the replicate pad, recorded per tile as pad_fraction. Padding geometry from results/tile_definition.json, same checkpoint.

D.6. Composition with weight quantisation

Early exiting removes work and quantisation makes the remaining work cheaper, so the two ought to compose. Whether they do is a question about the same weights, and it is measured here rather than assumed.

Weights	Rate	e2	e3	e4	e5	Layers
float32	q0	0.172	0.083	0.052	0.027	0
float32	q32	0.381	0.212	0.133	0.048	0
float32	q63	0.890	0.545	0.344	0.108	0
int8	q0	0.178	0.088	0.057	0.032	86
int8	q32	0.407	0.237	0.159	0.072	86
int8	q63	1.001	0.660	0.463	0.223	86
int6	q0	0.300	0.206	0.166	0.135	86
int6	q32	1.021	0.845	0.735	0.633	86
int6	q63	2.830	2.568	2.407	2.183	86
int4	q0	1.651	1.525	1.484	1.400	86
int4	q32	4.228	4.026	4.008	3.892	86
int4	q63	8.177	7.850	7.929	7.830	86

Table 40. Quality given up against the released decoder with the decoder weights quantised, at four exits and three rates. The float32 rows are the ladder itself and are the baseline each integer row should be read against. At q0 int8 is nearly free, moving the deepest exit from 0.027 dB to 0.032 dB, so the two levers compose. At q63 they compete: the deepest exit moves from 0.108 dB to 0.223 dB, which is more than a whole 0.1 dB budget spent before any tile has exited early. Six-bit costs 0.13 dB at the deepest exit at q0 and 2.18 dB at q63, and four-bit is unusable at every rate. The last column is the number of layers quantised, the same 86 in every integer row.

results/supp_quant_PAPER.json, on runs/RECIPE512/ckpt_PAPER.pth.tar, 32 held-out images at 512 px. Weight-only, symmetric, per output channel, no calibration, on dec.* and router_head.* with the encoder asserted unchanged; it supersedes results/quant_BEST.json, the same experiment on runs/BEST/ckpt_eval.pth.tar. Nothing here measures an integer kernel: the file records the bit width and the implied bit-operation ratio, and the arithmetic ran in floating point on quantised weights.

D.7. Integrated figures, and what they depend on

A budget is one sample of a curve. Video coding answers this for rate against quality with Bjontegaard's construction [4], integrating one axis over a stated interval of the other, and the same construction applies with compute in place of rate. BD-saving is the mean compute saved over an

interval of decibels and BD-quality is the mean decibel paid over an interval of saving. Neither means anything without its interval and its convention, and both are given with theirs.

Rate	BD-saving	BD-quality	BD-saving	BD-quality	Points
q0	n/a	0.0544	n/a	0.0453	15
q16	n/a	0.0686	n/a	0.0576	14
q32	n/a	0.0861	n/a	0.0721	13
q48	31.1%	0.0981	32.8%	0.0816	13
q63	28.7%	0.1121	30.8%	0.0923	13
Mean where defined	29.9%	0.1051	31.8%	0.0870	

Table 41. BD-saving and BD-quality per rate under both decibel conventions: the first pair of columns is per frame and the second is pooled. BD-saving is defined at only two of the five rates, because at q0, q16 and q32 the frontier saturates below the upper limit of the interval and there is no curve left to integrate. The convention moves the answer by 1.9 points of BD-saving and 0.018 dB of BD-quality on identical allocations, and it moves it consistently, since per-frame averaging raises every floor and pooling does not. Points is the number of measured frontier points inside the interval.

results/supp_bd_PAPER_per_frame.json and results/supp_bd_PAPER_pooled.json, both on runs/RECIPE512/ckpt_PAPER.pth.tar over 53 sequences, integrating results/supp_paper_curve_PAPER.json. BD-saving is over 0.066 to 0.300 dB per frame and 0.059 to 0.300 dB pooled, the lower limit being the highest floor any rate has under that convention. BD-quality is over 10% to 30% saving. The floors that set those lower limits are recorded per rate in both files as floor_db, and appear directly in the frontier table below at the two rates where the 0.05 dB budget is unreachable.

Two sensitivities sit around those numbers and are worth stating beside them. The first is the anchor. Our deepest exit is not bit-exact with the released decoder, since it carries seam repair and a fine-tuned trunk, and it starts every budget slightly behind: the drift runs from 0.005 dB at q0 to 0.036 dB at q63, which at q63 is a third of the whole budget. Setting it to zero would raise BD-saving at q63 from 28.7% to 34.9%, a gain of 6.2 points. The second is the interval. Over six defensible intervals the mean BD-saving moves from 22.8% to 29.2%, a range of 6.3 points, while the gap between the lowest and highest rate stays inside 15.3 to 16.8 points on every one of them; the level depends on the integration and the rate effect does not.

Anchor drift from results/supp_anchor_PAPER.json, on runs/RECIPE512/ckpt_PAPER.pth.tar, 53 frames, per-frame convention; the zero-drift figure is the bd_saving_pct_zero_drift field of results/supp_bd_PAPER_per_frame.json. Interval sensitivity from results/bd_sensitivity.json, which is not pinned: scripts/bd_sensitivity.py names its curve as results/paper_curve_grid128.json, on runs/wdec_j2_p128_grid/ckpt_epo0.pth.tar at 128 px tiles over 40 sequences, so its levels are not the levels above and only its robustness conclusion is quoted.

Budget	A BD-Rate	B BD-Rate	Difference	A saved	B saved
0.1 dB	1.034%	1.006%	0.028 pt	22.1%	17.6%
0.3 dB	2.813%	2.795%	0.018 pt	38.2%	35.6%
0.5 dB	3.075%	3.055%	0.021 pt	39.1%	39.0%

Table 42. BD-Rate cost of the two configurations: the rate a reader would have to spend to buy back the quality the compute saving costs, integrated over the five rate points, which is the number that makes this trade-off comparable with a published codec result. Configuration A signals the exit map in the bitstream and pays 79 to 95 bits a frame for it at the 0.1 dB budget, depending on the rate, configuration B infers it from what the decoder already holds and pays none. Their BD-Rate costs differ by at most 0.028 of a point, so the map is close to free. What A buys with it is compute: the gap in the last two columns is 4.5 points at the tight budget and 0.2 at the loose one, where both configurations are at the ceiling.

results/bd_rate.json, every row on runs/RECIPE512/ckpt_PAPER.pth.tar, which its sources block records file by file. The file it replaced is kept as results/bd_rate_before_pinning.json, whose saving column reads 41.9% at 0.5 dB where the pinned one reaches 39.1%.

D.8. Raw values behind the curves

Everything above is an integral, a mean or a picture. What follows is the measured numbers themselves, so that a reader can compare against them without digitising a plot or rerunning a decode.

Rate	Budget	lambda	dB	Model	Meter	Map bits
q0	0.1	5.150e-05	0.0998	29.9	29.2	79
q0	0.2	1.000e+00	0.1791	39.1	38.3	48
q0	0.3	1.000e+00	0.1791	39.1	38.3	48
q0	0.5	1.000e+00	0.1791	39.1	38.3	48
q16	0.1	2.280e-05	0.1000	25.6	25.0	86
q16	0.2	1.284e-04	0.1999	37.8	37.0	52
q16	0.3	1.000e+00	0.2194	39.1	38.3	48
q16	0.5	1.000e+00	0.2194	39.1	38.3	48
q32	0.1	9.080e-06	0.1000	20.9	20.4	93
q32	0.2	4.135e-05	0.1999	33.7	33.0	64
q32	0.3	1.000e+00	0.2819	39.1	38.3	48
q32	0.5	1.000e+00	0.2819	39.1	38.3	48
q48	0.1	4.731e-06	0.1000	18.3	17.9	94
q48	0.2	2.061e-05	0.1997	31.7	31.0	71
q48	0.3	6.974e-05	0.2993	37.9	37.1	52
q48	0.5	1.000e+00	0.3330	39.1	38.3	48
q63	0.1	2.874e-06	0.1000	15.6	15.2	95
q63	0.2	1.103e-05	0.1999	29.6	28.9	78
q63	0.3	3.221e-05	0.2999	35.8	35.0	57
q63	0.5	1.000e+00	0.3955	39.1	38.3	48

Table 43. The deployed operating points: four budgets at five rates. Model is the saving the cost model predicts for the chosen map and Meter is what a forward hook on every convolution and linear layer counted on the same decode, so the two columns are the cost model checked against itself; the model reads high by 0.4 to 0.8 points throughout. Map bits is the entropy of the per-frame exit histogram, charged to the bitstream before any saving is computed, and it falls as the budget loosens because the histogram concentrates.

results/signalled_RECIPE512_ctc53.json, on runs/RECIPE512/ckpt_PAPER.pth.tar, 53 sequences, per-frame convention. λ is the multiplier the two-level bisection settled on, and a λ of exactly 1 is the top of its bracket, which marks a budget the ladder saturated below rather than a price that was chosen. Saving is against the released decoder.

Rate	Budget	lambda	dB pooled	dB per frame	Saved	e2/e3/e4/e5
q0	0.05	2.065e-05	0.0500	0.0610	23.2	36/27/20/17
q0	0.1	7.286e-05	0.0999	0.1171	34.9	78/16/5/2
q0	0.15	2.371e-04	0.1499	0.1686	38.8	98/2/0/0
q0	0.2	n/a	0.1597	0.1791	39.1	100/0/0/0
q0	0.25	n/a	0.1597	0.1791	39.1	100/0/0/0
q0	0.3	n/a	0.1597	0.1791	39.1	100/0/0/0
q0	0.5	n/a	0.1597	0.1791	39.1	100/0/0/0
q16	0.05	6.631e-06	0.0500	0.0594	16.6	24/20/20/35
q16	0.1	3.199e-05	0.1000	0.1179	31.9	69/15/10/6
q16	0.15	7.234e-05	0.1499	0.1673	36.9	88/9/2/1
q16	0.2	2.483e-04	0.1998	0.2167	39.1	100/0/0/0
q16	0.25	n/a	0.2022	0.2194	39.1	100/0/0/0
q16	0.3	n/a	0.2022	0.2194	39.1	100/0/0/0
q16	0.5	n/a	0.2022	0.2194	39.1	100/0/0/0
q32	0.05	9.305e-07	0.0500	0.0544	5.0	6/8/11/75
q32	0.1	1.398e-05	0.1000	0.1212	28.0	56/17/15/12
q32	0.15	2.862e-05	0.1500	0.1716	34.2	78/12/8/3
q32	0.2	5.497e-05	0.1999	0.2199	37.4	91/7/2/0
q32	0.25	1.445e-04	0.2498	0.2741	39.0	99/1/0/0
q32	0.3	n/a	0.2581	0.2819	39.1	100/0/0/0
q32	0.5	n/a	0.2581	0.2819	39.1	100/0/0/0
q48	0.05	n/a	0.0544	0.0599	n/a	none
q48	0.1	7.190e-06	0.1000	0.1204	25.5	49/15/22/14
q48	0.15	1.508e-05	0.1500	0.1728	32.3	71/12/11/5
q48	0.2	2.682e-05	0.1998	0.2228	36.0	85/8/6/1
q48	0.25	4.950e-05	0.2500	0.2768	38.1	95/4/1/0
q48	0.3	1.548e-04	0.2996	0.3322	39.1	100/0/0/0
q48	0.5	n/a	0.3005	0.3330	39.1	100/0/0/0

Rate	Budget	lambda	dB pooled	dB per frame	Saved	e2/e3/e4/e5
q63	0.05	n/a	0.0584	0.0659	n/a	none
q63	0.1	4.293e-06	0.1000	0.1225	22.6	38/18/28/16
q63	0.15	8.642e-06	0.1500	0.1757	30.2	63/14/16/6
q63	0.2	1.500e-05	0.1999	0.2274	34.4	79/10/9/3
q63	0.25	2.537e-05	0.2499	0.2709	36.9	90/5/5/1
q63	0.3	4.557e-05	0.2998	0.3333	38.4	96/3/1/0
q63	0.5	n/a	0.3534	0.3955	39.1	100/0/0/0

Table 44. The frontier itself: every operating point behind the per-class figure, at seven budgets and five rates, with the exit shares that produced each one. Both decibel conventions are printed for the same allocation, and they differ by 0.004 to 0.042 dB over the table and by 0.017 to 0.022 dB at the 0.1 dB budget, which is a fifth of that budget. A row reading n/a in the saving column is a budget below the floor at that rate, where no allocation meets the target. A row reading 100/0/0/0 is past saturation: the file stores neither a multiplier nor a histogram there, and the shares are filled in from what saturation means, every tile at the shallowest selectable exit.

results/supp_paper_curve_PAPER.json, on runs/RECIPE512/ckpt_PAPER.pth.tar. Saving is converted to the released-decoder denominator by charging the retained cost the price of our deepest exit, 1.0095 released decodes, which is 1.0 at the precision the paper prints and is derived here from the ceiling in results/saturation_RECIPE512_ctc53.json and the saturated saving in this file. One caution about that denominator: this file bills the shared stem inside every tile's cost, while results/supp_latency_batch_1920x1080.json bills it once per frame through flexuf/cost.py frame_relative_cost, and the two accountings differ by about a third of a point on an identical exit map at q0.

D.9. Failure cases

Five cases are worth naming, all of them measured in the files above rather than inferred.

- **Sequences that cost more than they save.** At q63 four sequences finish at minus 0.95%, which is every tile at full depth and our deepest exit costing that much more than a released decode: RaceHorses at 832x480 and at 416x240, BlowingBubbles and BQSquare. One more sits under 3% and ten in all save less than a tenth of a decode. On this content the method should be switched off rather than run at a budget, and the exit histogram is what tells the encoder so.
- **The bottom tile row of a padded frame.** The six worst tiles in the test set are all in the last row of a 1080p frame at 78% padding, each losing between 1.72 and 1.97 dB. The allocation is behaving correctly given its table, and the table is measuring a region that will be cropped away before anyone looks at it, so the budget is being spent on invented pixels.
- **The two smallest classes at high rate.** HEVC C and HEVC D save 2.5% at q63, which is one tile in four moved one exit up and nothing else. With two tiles on a 416x240 frame there is no allocation worth making, and the honest reading is that this method needs a frame large enough to hold a useful number of tiles.
- **Quantisation and early exit compete at high rate.** Int8 weights cost 0.115 dB at q63 at the deepest exit, more than the whole budget the allocation is then asked to work inside, while at q0 they cost 0.005 dB. A deployment cannot assume the two savings multiply.
- **A 0.5 dB budget has no per-sequence breakdown to report.** The queue that produced this section asked for one and the job failed. The cause is the result: at 0.5 dB every rate is past saturation, the frontier stops at the ceiling of 39.1%, and a saturated operating point carries no allocation to break down. The sweep now runs from 0.10 dB to 0.30 dB.

Worst tiles from the worst_tiles block of results/supp_opquality_PAPER.json and negative sequences from results/supp_per_sequence_PAPER_b010.json, both on runs/RECIPE512/ckpt_PAPER.pth.tar. The failed job is recorded in results/supp_queue2.log at 12:06:04, and the saturation it ran into in results/saturation_RECIPE512_ctc53.json.

D.10. What this section does not measure

- No interval on any headline comes from a repeated training run. No configuration in this work was trained twice and no seed is set in the

decoder trainer, so the spread reported above is across sequences at a fixed checkpoint and not across runs. Evaluation is deterministic once a checkpoint is fixed and training is not, which matters when reading any comparison of a few tenths of a point.

- The 0.1 dB budget is a convention this work adopts and not a perceptual threshold. Subjective work measures the smallest noticeable change in a quantisation parameter or in a video quality metric, not in tenths of a decibel of PSNR, so no published result licenses a claim that 0.1 dB is invisible. The MS-SSIM table is evidence that the loss is small on a second metric, and the 0.3 dB and 0.5 dB columns are there so that nothing rests on the choice.
- One integrated figure is still not pinned. The interval study is on a 128 px run over 40 sequences and is quoted only for its robustness conclusion; every other table in this section is on runs/RECIPE512/ckpt_PAPER.pth.tar.
- Every measurement here is on intra frames of video sequences at one frame per sequence. Nothing here measures the inter-frame path, a second decoder, a resolution above 1080p, or a traditional-codec anchor. The first three are out of reach on the evaluation card, which has about 5 GB free; the last is a scope decision, since the axis this work measures is decoder-side compute against a fixed learned decoder whose own position against VTM is published.

E. The operating window

A budget D is the peak-signal-to-noise ratio the decode is allowed to give up, and an allocation is admissible if the loss it incurs stays at or below it. Between a floor, below which no allocation is admissible, and a saturation point, above which every allocation has reached the same ceiling, lies the window of budgets in which the decoder has anything to trade. Section C derives the two ends and Section D tabulates them; this section is about the window itself: how the two ends are measured and how far the measurements agree, what has to be stated with every decibel before any of it means anything, the sweep across the whole window, and how much of the rescaling that collapses the five rates onto one curve survives a change of sweep, of convention or of checkpoint.

E.1. Floor, saturation point and ceiling

The floor $D_{\min}$ is the loss of a tiled decode with every tile at full depth. No early exiting has happened, so it is the tiling penalty alone: the border of every tile has been convolved against padding rather than against its neighbour. The saturation point D_{sat} is the loss when every tile takes exit j, the shallowest the split depth allows. Neither is a quantity to search for. Each is one forward pass per rate with the exit map held constant, which is what makes the rescaling in Section E.4 cheap enough to be worth doing in deployment.

Between them the saving is fixed by the ladder rather than by the budget. With c_k the cost of exit k in units of one released full-frame decode, j the split depth and K the number of exits, the ceiling is closed form under either compute denominator:

$$S_{\max}^{\text{rel}} = 100(1 - c_j), \quad S_{\max}^{\text{self}} = 100(1 - c_j/c_{K-1}) \quad (22)$$

With $c_j = 0.6088$, recorded in results/saturation_RECIPE512_ctc53.json, and $c_{K-1} = 1.0095$, recorded as deepest_exit_cost in results/router_RECIPE512_b01.json, the first reads 39.1% and the second 39.7%. The deepest exit costs more than one released decode because it carries the seam repair, which the released decoder does not run, and that difference is the whole of the gap between the two ceilings.

Section D tabulates both ends at all nine measured rates, and that table is not repeated here. The one property of it this section needs is that a budget in decibels means something different at each rate: both ends widen as the rate falls, the band widens faster than the floor rises, and the 0.1 dB budget that sits 45% of the way up the band at q0 sits 9% of the

way up at q63. The rescaling of Section E.4 turns out to need that position rather than the decibel figure.

A budget below the floor admits no allocation at all, which is a different failure from a budget falling in a gap of the Lagrangian hull, where the sweep returns the nearest reachable point and Everett's argument still makes it optimal for the compute it consumes [28, 29]. Below the floor the feasible set is empty and there is nothing to return. Three cells of the grid in Section E.3 are in that state and are recorded as such rather than dropped: at a 0.05 dB budget the three highest rates, q32, q48 and q63, have floors of 0.053, 0.060 and 0.066 dB.

Three estimates of the floor are in play and they are not interchangeable. Table 45 gives all three, because the rescaling below divides by the distance between the floor and the saturation point, and near the floor that is a small difference divided by a small difference.

source	what it measures	q0	q63
the saturation file	a deployed decode with every tile at full depth, over 53 frames	0.036	0.072
the dense grid	the same quantity, over 106 frames	0.033	0.066
the second sweep	the per-tile table at a zero multiplier, where a tile takes its own lowest-error exit	-	0.066

Table 45. Three measurements of the floor, in decibels. The first two differ by three to six thousandths of a decibel because they are the same quantity over a different number of frames. The third is a different quantity: a tile whose shallower exit happens to have the lower error takes it, so the table's floor sits below the deployed one. Everything downstream of here takes both ends of the window from the first row, so that the floor and the saturation point are on one measurement.

The three rows are results/saturation_RECIPES12_ctc53.json, results/signalled_RECIPES12_grid.json and results/supp_paper_curve_PAPER.json, all on the pinned checkpoint and all with decibels averaged per frame. The third records a floor only where it found the 0.05 dB budget unreachable, which is why q0 is empty.

E.2. Three conventions, and the conversions between them

Three separate choices have to be pinned before a figure in this section means anything, and the project makes all three in two ways in different files. The loss can be measured against the released decoder's full-frame decode of the same latent, which puts the cost of cutting the frame into tiles inside the budget, or against this model's own deepest exit decoded tile by tile, which cancels it. Decibels can be averaged per frame and then over frames, which is what the released codec's own evaluation does, or pooled into one mean squared error first. And the saving can be divided by the released decode or by our own deepest exit. Two of the three conversions are exact:

$$D^{\text{rel}} = D^{\text{self}} + D_{\min}, \quad S^{\text{rel}} = 100 - c_{K-1}(100 - S^{\text{self}}) \quad (23)$$

choice	the two options	what separates them
loss measured against	the released full-frame decode, or our own tiled deepest exit	the floor: 0.036 dB at q0, 0.072 dB at q63
decibels averaged	per frame and then over frames, or pooled over every tile first	measured, not closed form: a pooled 0.1 dB is 0.117 dB per frame at q0 and 0.122 at q63
saving divided by	the released decode, or our deepest exit at 1.0095 of it	a self-referenced 30% is 29.33% release-referenced, and a self-referenced 0% is -0.95%

Table 46. What has to be stated with every decibel and every saving in this section. Take from it that a number quoted without its convention is uncertain by about a third of a 0.1 dB budget on the quality axis and by about two thirds of a point on the compute axis. The first and third rows convert exactly, by the identity above; the second does not, because a mean of logarithms is not the logarithm of a mean.

The floor is results/saturation_RECIPES12_ctc53.json; the two averaging figures are the 0.1 dB operating points of results/supp_paper_curve_PAPER.json, which bisects on the pooled decibel and records the per-frame decibel of the same allocation beside it; the deepest-exit cost is deepest_exit_cost in results/router_RECIPES12_b01.json. The averaging convention itself is defined in Section A.

E.3. The grid

Table 47 is the sweep the main paper quotes: nine budgets across five rates, measured against the released decoder on both axes, with decibels averaged per frame, two frames from each of the 53 test sequences. Allocation is by the exact per-tile losses, so every entry is an upper bound on what a decoder-side router can reach.

budget dB	q0	q16	q32	q48	q63
0.050	16.4	9.3	none	none	none
0.075	24.3	19.7	14.6	11.1	7.6
0.100	29.8	25.5	20.7	18.2	15.6
0.150	36.6	33.0	28.6	26.5	24.1
0.200	39.1	37.8	33.7	31.6	29.6
0.250	39.1	39.1	37.4	35.2	33.0
0.300	39.1	39.1	39.1	37.9	35.8
0.400	39.1	39.1	39.1	39.1	39.1
0.500	39.1	39.1	39.1	39.1	39.1

Table 47. Compute saved against budget, release-referenced. *none* marks a budget below that rate's floor, where no allocation is admissible. Reading down a column, the return on a looser budget falls away long before the ceiling is reached. Reading across a row, the rate dependence is largest at the tightest budget and vanishes once every rate has saturated. The 0.1 dB row reproduces the headline of the main paper, 29.2% at q0 and 15.2% at q63, to within a tenth of a point on a different frame count.

results/signalled_RECIPES12_grid.json, pinned checkpoint, two frames per sequence, budget bisected on the delivered per-frame decibel of a real decode of the mixed map. The headline file results/signalled_RECIPES12_ctc53.json uses one frame per sequence and reports 29.2% and 15.2% at the same budget.

A second sweep of the same checkpoint prices the same window differently, and Section D prints its operating points in full. It bisects on the pooled decibel rather than the per-frame one and reads 53 sequences at one frame each rather than two, and at a nominal 0.1 dB it saves several points more at every rate than the grid above. The conversions of Section E.2 account for part of that and not for all of it, which is the first sign of what Section E.5 measures. Its own saturation points are the useful thing here: the budget at which a rate runs out of allocation is 0.20 dB at q0, 0.25 dB at q16, 0.30 dB at q32, 0.50 dB at q48, 0.50 dB at q63. The three budgets the paper reports are therefore not three points on one curve. At 0.1 dB every rate still has an allocation to make, at 0.3 dB only q48 and q63 do, and at 0.5 dB none does.

results/supp_paper_curve_PAPER.json, pinned checkpoint, 53 sequences at one frame each, 35 operating points at seven budgets. A saturated point is flagged in the file rather than inferred here, and carries no exit histogram, because the allocation has stopped changing.

Budgets in [0.179, 0.194] dB saturate the lowest rate and no other, a window 15 thousandths of a decibel wide. Above 0.396 dB every rate has saturated, so the binding constraint is the ladder rather than the budget. Both figures are read off the nine-rate table in Section D rather than off either sweep.

E.4. Rescaling onto the band

With both ends of the window measured, a budget can be quoted as a position in the window rather than as a decibel figure. Write

$$u = \frac{D - D_{\min}}{D_{\text{sat}} - D_{\min}} \quad (24)$$

so that $u = 0$ is the floor and $u = 1$ the saturation point. At a matched decibel the five rates are 15.5 points apart in saving. At a matched u they are 1.7 points apart on average and 2.2 at worst, so the window accounts for most of the rate dependence of the trade-off and a couple of points are left over.

u	q0	q16	q32	q48	q63	spread
0.1	16.5	14.6	15.1	15.2	16.3	1.91
0.2	21.0	20.9	20.7	20.9	21.8	1.13
0.3	25.2	24.9	24.3	25.5	26.2	1.91
0.4	28.3	27.7	27.9	28.6	29.7	1.96
0.5	30.8	30.4	30.4	31.4	31.9	1.54
0.6	32.8	33.0	32.8	33.4	33.9	1.13
0.7	34.7	34.7	34.7	35.3	35.7	1.00

Table 48. The five rates at matched positions in their own windows. Read across a row: five rates whose raw savings at a matched decibel differ by 15.5 points agree here to about a point and a half. The spread is largest at the bottom of the window, where the floor subtraction is a large fraction of a small number and the two files disagree about the floor by a few thousandths of a decibel.

Computed in the build from results/signalled_RECIPES12_grid.json and results/saturation_RECIPES12_ctc53.json by the rescaling in scripts/tradeoff_figure.py, linearly interpolated between measured points. Every rate has measured points spanning $u = 0.10$ to 0.70 , so no entry is an extrapolation. Mean spread over these seven positions 1.51 points, worst 1.96, against the 1.7 and 2.2 recorded in results/band_collapse.json, which interpolates on a wider grid and holds the last measured value beyond each rate's last point.

The collapsed curve is then described by one free number. With C the architectural ceiling, which is not fitted,

$$S(u) \approx Cu^\beta \quad (25)$$

and fitting $\ln(S/C)$ against $\ln u$ by least squares through the origin gives 0.38 on the pinned checkpoint, with $R^2 = 0.989$ and a worst residual of 2.0 points. The same procedure on the second training run gives 0.33 with $R^2 = 0.984$, which is the replication Figure 14 draws and Section E.5 takes apart.

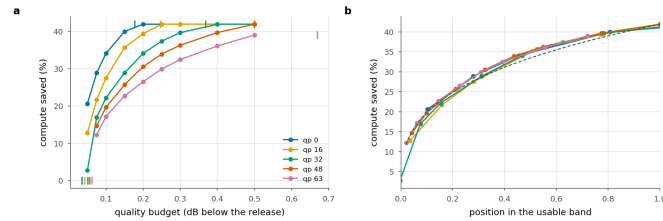


Figure 14. The collapse on the second training run. **a** Saving against budget, one line per rate, with each rate's floor and saturation point ticked. **b** The same with the budget axis rescaled onto each rate's own window; dashed is the one-parameter power law. The main paper shows this pair for the pinned checkpoint.

docs/figures/budget_band_BEST.png, drawn by scripts/tradeoff_figure.py from results/signalled_BEST_grid.json and results/saturation_BEST_ctc53.json, both on runs/BEST/ckpt_eval.pth.tar rather than on the pinned checkpoint.

E.5. How weak the replication is

The main paper says the collapse replicates and the exponent does not. Two checkpoints is a thin basis for either half of that sentence, and Table 49 is meant to show how thin. Each row refits the same one-parameter law after changing exactly one thing, over the window of positions where every sweep has points.

what is changed	β before	β after	Δ
the checkpoint, pinned to BEST, each with its own ceiling	0.360	0.310	-0.050
the sweep, 106 frames to 53, one checkpoint	0.360	0.261	-0.098
the budget axis, per-frame decibels to pooled, one sweep	0.272	0.180	-0.093
the assumed ceiling C , 39.1 to 41.91	0.360	0.421	0.061
the floor, the saturation file to the grid's own	0.360	0.369	0.010
the rate, fitted one at a time, q63 against q32	0.321	0.385	0.064
the fit form, $1/b$ from the inverse fit, per rate	0.326	0.459	0.133

Table 49. What moves the exponent. Take from it that the exponent describes a sweep rather than a decoder. Every row except the floor moves it by as much as changing the checkpoint does, re-sweeping the same checkpoint moves it twice as far, and reading the budget axis in the other averaging convention moves it almost as far again. The comparison the main paper draws between 0.38 and 0.33 is inside the noise of every other choice on this list.

Computed in the build. Rows 4, 5 and 6, and the before column of rows 1 and 2, refit results/signalled_RECIPES12_grid.json against results/saturation_RECIPES12_ctc53.json; the after column of row 1 uses results/signalled_BEST_grid.json and results/saturation_BEST_ctc53.json on runs/BEST/ckpt_eval.pth.tar, whose ceiling is its own; the after column of row 2 uses the operating points of results/supp_paper_curve_PAPER.json and row 3 uses its raw sweep; row 7 is results/frontier_low.json, fitted to results/curve_BEST.json on the same second run over 40 sequences. Every fit is over u in $[0.15, 0.85]$ and through the origin in log space, on 16 to 22 points. R^2 stays between 0.911 and 0.980 across the rows, so no row is a bad fit to its own data.

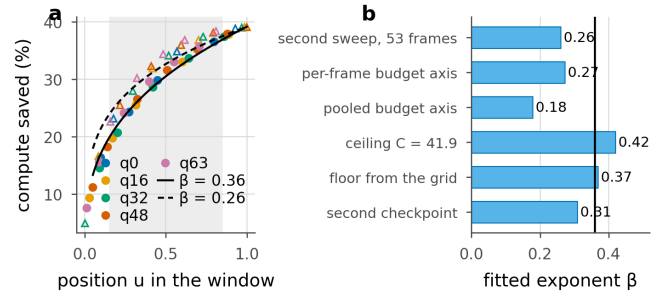


Figure 15. The collapse, and what the exponent is sensitive to. **a** The rescaled points from two sweeps of the pinned checkpoint, filled circles for the 106-frame grid and open triangles for the 53-frame frontier, with the power law fitted to each over the shaded window. The two sweeps of one checkpoint do not lie on one curve. **b** The exponent under one change at a time, with the vertical line at the exponent of the first row of Table 49. The change of checkpoint, which the main paper reads as a finding, is the smallest of the six.

docs/figures/band_sensitivity_PAPER.png, drawn by scripts/supp_fig_band_sensitivity.py, which imports the fit from this section's module so that the figure and the table cannot disagree. Sources as for the table above.

The floor is the one input the exponent is insensitive to over this window, and only over this window. Refitting over the whole range instead, where the points below $u = 0.15$ sit, moves it from 0.379 to 0.417 when the floor is taken from the grid rather than from the saturation file. A rescaling is only as good as the two numbers it divides by, and near the floor it is dividing a small difference by a small difference.

One stored file needs regenerating before submission. Re-running the fit on the inputs results/band_collapse.json names, as they stand now and over the full range it used, gives $\beta = 0.3792$ with $R^2 = 0.9895$, against the 0.3767 and 0.9890 the file records; both of its inputs were rewritten after it was written, and it still carries the pre-correction ceiling of 41.91. The two digits the paper quotes, 0.38 and 0.989, survive the regeneration unchanged, so nothing in the main text moves. results/band_collapse_BEST.json reproduces exactly from its inputs: 0.3253 and 0.9843 against the 0.3253 and 0.9843 recorded.

E.6. What the operating window does not pin

- The floor and the saturation point are measured, cheap and unambiguous. Each is one forward pass per rate, and the ceiling between them is closed form in the per-exit costs.
- The collapse is a strong empirical regularity on both checkpoints. Rescaling takes the spread across rates from 15.5 points to under two at matched positions.
- The exponent is a description of one sweep. It moves further when the same checkpoint is swept again, or when the budget axis is read in the other averaging convention, than it moves between the two training runs the main paper compares.
- Everything here is the oracle allocation, obtained with the per-tile losses known. It bounds any router and says nothing about how much of the window a decoder-side predictor can reach.
- Every entry is a mean over the test set at one global multiplier. The spread behind those means is in Section D; no configuration was trained twice, so neither section bounds the variation between two runs of the same recipe.
- The window is measured with the exit map chosen by an exact multiplier sweep. Nothing here says how wide the window looks to a decoder that has to guess the map, which is what Section F measures.
- The grids are intra frames at 1080p and below, on the CTC set. Nothing here is measured on inter frames, and the window has not been measured above 1080p, where the evaluation card does not have the memory for a decode.

F. The router and the parameter-free rule

An exit map assigns one exit index to every tile of a frame, and this work produces one in three ways. Configuration A searches all K exits for every tile at the encoder, which holds the source, and signals the answer. Configuration B predicts the map at the decoder from data the decoder already holds, and signals nothing. The third way computes it at the decoder from one number the entropy coder has already produced, with nothing learned. The head architecture, the inputs it reads and the rule by which it turns logits into an exit are given in the main paper's allocation subsection, and none of that is repeated here. What is here is what those inputs are worth, what the head costs to run, and how it fares against the third way.

Write $D(t,k)$ for the mean squared error of tile t decoded at exit k and c_k for the cost of exit k in units of one released decode. At a price λ on compute, the Lagrangian oracle gives each tile the exit that minimises

$$\ell(t,k) = D(t,k) + \lambda c_k, \quad k^*(t) = \arg \min_{k \geq j} \ell(t,k) \quad (26)$$

The split depth j is the number of trunk groups every tile runs before any tile may leave, so exits below j do not exist and the minimisation starts there. Throughout this section, *agreement* means the fraction of tiles on which a predictor picks the same exit as this oracle at the same λ . It is the quantity the head is trained on. Part of what F.4 reports is that it is not the quantity that decides which allocation is cheaper.

F.1. What each input is worth

The head reads four per-tile signals and one per-frame signal. Since F.4 finds that a rule reading a single number beats it, it is fair to ask which of the five carries anything. The ablation trains the same head once per input group, with every other group zeroed after its projection rather than deleted. Architecture, parameter count, optimiser, seed, image order and quality draws are then identical across the six runs, and the only thing that differs is how much the head is allowed to know. All six carry 148,176 parameters, which is the deployed head's 144,024 plus the projection for the bit count that the deployed head does not use.

The quality index q stays live in every single-group run. It is one number per frame, identical for every tile in that frame, so it can say which operating point the decoder is at and cannot, even in principle, tell two

tiles of one frame apart. Holding it live keeps the runs comparable on per-tile information alone. The run with q on its own is then the floor that choice implies, which is the agreement reachable with no per-tile information whatever.

live inputs	agreement	$\pm$ s.e.	last 250	over floor
stem,qp	0.7750	0.0074	0.7607	+0.2744
stem,latent,scales,bits,qp	0.7721	0.0075	0.7543	+0.2715
latent,qp	0.7448	0.0080	0.7433	+0.2442
scales,qp	0.7156	0.0083	0.7170	+0.2150
bits,qp	0.6202	0.0091	0.6137	+0.1196
qp	0.4923	0.0114	0.4900	-0.0083

Table 50. What each router input is worth. Read the first column against the last: only the stem moves agreement far from what a single constant exit already achieves. Agreement with the oracle's exit choice on 1,200 frames and 4,800 tiles the head never trained on, with one standard error taken across frames rather than across tiles, since tiles of one image are not independent draws. *last 250* is the running held-out agreement over the final 250 training steps, on 12 tiles a step, and confirms that the end-of-run measurement is not a fluctuation. *over floor* is the margin over the best single constant exit, which scores 0.5006 on these tiles.

results/router_ablation.json, and the six per-variant training records results/router_ablation_stem.json and its five siblings. All on the pinned checkpoint, all at $\lambda = 1.3 \times 10^{-5}$, all 3,000 steps from seed 0.

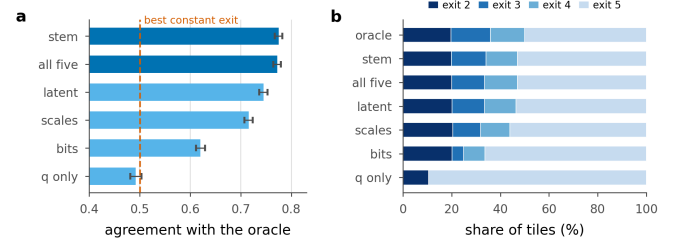


Figure 16. The input ablation, both halves of it. **a**, agreement with the oracle per variant, with one standard error across frames; the dashed line is the best single constant exit. The two dark bars are the two heads that see the stem. **b**, where each variant sends its tiles, against the oracle's own distribution. Every variant reproduces the oracle's use of the shallowest exit to within a point, and what degrades as inputs are removed is the separation of the middle of the ladder: the bit count alone nearly empties exit 3, and q alone uses two of the four exits available to it.

Drawn by scripts/supp_router_figs.py from results/router_ablation.json, fields agree, stderr, pred_hist and oracle_hist.

The stem alone. Reading the stem and q gives 0.7750; reading the stem, the latent, the scales, the bit count and q gives 0.7721. The difference is 0.0029 against a standard error of 0.0105 on the difference, so it is under a third of one standard error and its sign is not determined. Four of the five inputs add nothing this measurement can resolve. That result is about those four inputs in this head, and it does not show that the latent, the scales or the bit count are uninformative about a tile. What it shows is that whatever they carry is already carried by the stem, which is computed downstream of all of them.

The single-group ordering. The stem at 0.7750 is 0.0302 above the latent, against a standard error of 0.0109 on the difference; the latent is 0.0292 above the scales; and the scales are 0.0954 above the bit count, which is more than seven standard errors. The bit count is the weakest per-tile input the head has, and F.4 shows a rule that reads the bit count and nothing else beating the head at every rate. The input the head learns least from is the input that wins when it is used differently.

The floor. The quality index on its own reaches 0.4923 ± 0.0114 where the best single constant exit scores 0.5006. A head with no per-tile information cannot beat the best constant in expectation, and this one does not. The floor belongs beside every other row, because an agreement of 0.62 cannot be read at all until it is known that 0.50 is free.

The ablation's own cost. The six trainings took 6.5 hours of one NVIDIA RTX A6000 between them, from 62 to 66 minutes each

(results/router_ablation_*.json, field wall_s). The decoder is frozen for all of them, so none of that is decoder training. It is the price of the question, and it is worth stating beside a negative answer.

F.2. What the head costs

q	decode (ms)	stem (ms)	router (ms)	share (%)
0	114.29	41.25	0.571	0.500
32	114.41	41.32	0.568	0.497
63	114.44	41.31	0.568	0.496

Table 51. The head against the decode it decides for. The share column is the number to take away, and it is three times what the operation count predicts. Three timings on one latent, interleaved so that a co-tenant's load drift lands on all of them equally. *decode* is the tiled decoder at the deepest exit, which is what the head's share is a share of; *stem* is the first j groups, which the head does not pay for because the decoder runs them anyway before the split. The head takes 0.50% of the decode where its operation count predicts 0.163%, a factor of 3.0.

results/router_latency.json: 1920×1080 padded to 2048×1280, 40 iterations after warm-up, one NVIDIA RTX A6000, the 144,024 parameter head runs/RECIPE512/routers2/v2_lam1.3e-5.pth. Measured on runs/RECIPE512/ckpt_eval.pth.tar rather than on the pinned checkpoint; the quantity is a ratio of two timings of the same weights, so it does not depend on which epoch they came from.

The extra 0.33 points are launch overhead. A head of 144,024 parameters evaluated on 40 vectors is small enough that its wall-clock is dominated by the cost of starting its kernels, and an operation count does not contain that. Every configuration-B saving in this work charges the head at its operation share, which is the smaller of the two numbers and therefore the more flattering to us; the honest reading is that the head costs about half a percent of the decode in time. It stays small against what it decides about, since one trunk block is 7.5% of the decode by the cost vector below.

Two heads, two shares. The pinned checkpoint carries a head that was trained jointly with the decoder, and a second head was trained afterwards against that decoder once it was frozen. They are different objects and they are priced differently: the jointly trained one is charged 0.044% of the decode and the frozen-decoder one 0.163%, because the second reads the decoded latent and the entropy model's scales as well as the stem. Both shares are measured by hooks on the head's own convolutions and linear layers rather than assumed, in the same run that produced the saving. F.4 reports both heads.

results/router_RECIPE512_b01_joinhead.json and results/router_RECIPE512_b01.json, field router_compute_share_pct in each.

For scale, what the search costs instead. Configuration A does not run a head; it runs the argmin itself, which needs the true error of every exit. On the pinned checkpoint at q63 one tiled 1080p decode takes 115 ms, the deployed table costs 4.52 of those and the full-frame table 1.37. A decoder-side head at half a percent of one decode, and a rule at nothing, are both several orders below that, which is the whole reason for wanting one.

results/supp_encoder_cost_PAPER.json, pinned checkpoint, 15 iterations on one NVIDIA RTX A6000.

F.3. The calibrated bit rule

The entropy model produces one number per tile before the trunk runs, at no cost, and then discards it: how many bits that tile's latents took. Per-block bit allocation is a standard quantity in learned compression, where it is something to choose, and block-level rate control sets it so that complex regions get more bits [42]. Here it is read in the other direction, after the fact and at the decoder, as a statement about how hard the region was. The results files call the rule *rate-rank*; the paper calls it the calibrated bit rule. What follows is all of it.

The statistic. Sum the entropy coder's estimated bits r_i over the latent positions i falling inside tile t , and divide by the mean over the N tiles of that frame. Normalising per frame rather than globally is deliberate: the absolute rate level is a property of the frame, and what decides a tile is

how it compares with the rest of its own frame.

$$b(t) = N \frac{\sum_i r_i}{\sum_i r_i} \quad (27)$$

The surrogate. Assume every tile has the same shape of decay across the ladder, up to a scale that depends only on $b(t)$.

$$\log D(t, k) \approx \alpha \log b(t) + c + \log \varphi_k \quad (28)$$

ϕ has one entry per reachable exit and says what that exit costs on an average tile; α is an exponent fitted rather than assumed. Fitting it is what lets the rule find the direction of the relation instead of asserting one, and an earlier version of this surrogate that fixed the exponent at 1 was wrong about that direction. The fit is offline and leave-one-sequence-out, so no sequence contributes to the profile that routes it, and its whole output is six numbers per rate.

The decision. Run the oracle's own Lagrangian on the surrogate table in place of the true one, and bisect λ against the budget as configuration A does.

$$\hat{k}(t) = \arg \min_{k \geq j} [e^c b(t)^\alpha \varphi_k + \lambda c_k] \quad (29)$$

the decoder, once per frame

- 1 *input* r , the entropy coder's estimated bits per latent position; the cost vector c ; the price λ ; the calibration (α, c_0, ϕ) for this quality index
- 2 *for each* tile t of the N tiles of the frame
- 3 $b \leftarrow N \cdot (\text{sum of } r_i \text{ inside } t) / (\text{sum of } r_i \text{ over the frame})$
- 4 *for* $k = j \dots K-1$
- 5 $L_k \leftarrow e^{c_0} \cdot b^\alpha \cdot \phi_k + \lambda \cdot c_k$
- 6 $\text{exit}(t) \leftarrow$ the k that minimises L_k
- 7 *return* exit

offline, once per quality index, leave-one-sequence-out

- 8 $d(t) \leftarrow$ mean over $k \geq j$ of $\log D(t, k)$
- 9 $(\alpha, c_0) \leftarrow$ least squares of $d(t)$ on $\log b(t)$
- 10 $\log \phi_k \leftarrow$ mean over t of $[\log D(t, k) - \alpha \log b(t) - c_0]$

Table 52. The calibrated bit rule, in full. Lines 1 to 7 are what the decoder runs: one power and K products per tile, no parameters, nothing signalled. Lines 8 to 10 are the offline calibration, whose entire output is the six numbers per rate printed in the next table. λ is bisected in two levels, on the cheap per-tile table first and then against a real decode of the resulting map, because the table and the decode differ slightly at the tile seams. Sweeping λ traces the lower convex hull of the achievable set in the sense of [28], so the point returned is the best one on that hull at the budget it consumes.

q	delivered dB	α	ϕ_2	ϕ_3	ϕ_4	ϕ_5
0	0.1000	9.91	1.0242	1.0009	0.9912	0.9841
16	0.1000	4.58	1.0251	1.0021	0.9911	0.9822
32	0.0998	2.73	1.0297	1.0013	0.9893	0.9804
48	0.0999	1.94	1.0350	1.0004	0.9884	0.9771
63	0.0997	1.43	1.0494	0.9963	0.9829	0.9731

Table 53. The rule's entire state, at the 0.1 dB budget. There is nothing else to store, and it is printed here so that the rule can be reproduced without refitting it. α is positive everywhere, so a tile whose latents cost more bits is modelled as harder at every exit and is sent deeper, which is what the correlations in F.4 confirm. ϕ is nearly flat, spanning 4.1% at q0 and 7.8% at q63 between its shallowest and deepest entry, against costs that span 0.61 to 1.01, so what moves a tile along the ladder is its own $b(t)$ rather than the shape of the profile. *delivered dB* is what the two-level bisection actually landed on.

results/raterank_RECIPE512_b01.json, pinned checkpoint, 53 CTC sequences at one frame each.

The costs c_k on the pinned checkpoint are 0.6088, 0.7578, 0.8697, 1.0095 for exits 2 to 5, recovered from the frame-level saving of maps that send every tile to one fixed exit. The shallowest of them, $c_j = 0.6088$, sets the architectural ceiling of 39.1%. The deepest is above 1 because our ladder carries seam repair and an adapter that the released

decoder does not.

results/static_RECIPES12_b01.json, field uniform, and
results/saturation_RECIPES12_ctc53.json, fields cost_j and ceiling_pct.

F.4. The rule against the trained head

Two trained heads exist on this checkpoint and both are reported, since averaging them would hide the only spread this work has. *joint* is the head the checkpoint carries, trained alongside the decoder and read straight out of it. *frozen* is the 144 K head trained afterwards, against the decoder once it had stopped moving. Both were evaluated on the pinned checkpoint over 53 sequences at one frame each, by one script, at the same budgets, and both are charged for their own compute.

q	0.1 rule	0.1 joint	0.1 frozen	0.3 rule	0.3 joint	0.3 frozen
0	27.76	26.84	24.41	39.12	39.08	38.96
16	22.61	18.36	20.95	39.12	39.08	38.96
32	18.20	13.21	17.43	39.12	39.08	38.96
48	14.66	8.26	14.15	37.73	37.03	33.40
63	11.97	5.13	11.14	34.34	34.07	27.73

Table 54. The parameter-free rule against two trained heads, as the percentage of the released decoder's operations saved, at two budgets and five rates. The rule is ahead of both heads in every cell of this table. Every column is charged for what it costs: each head pays its own compute share and the rule pays nothing. The two heads are further apart from each other than either is from the rule at several rates, which F.5 takes up.

Rule: results/raterank_RECIPES12_b01.json and b03. Joint head: results/router_RECIPES12_b01_jointhead.json and its b03 sibling. Frozen head: results/router_RECIPES12_b01.json and b03. All on runs/RECIPES12/ckpt_PAPER.pth.tar.

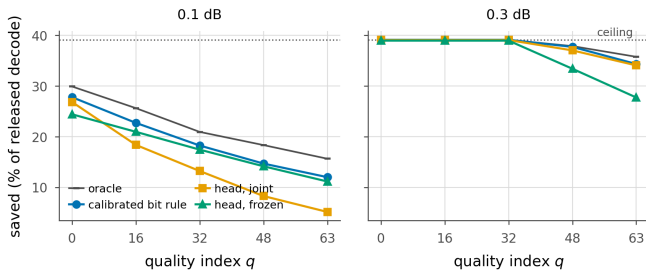


Figure 17. The rule sits between the two heads and the oracle. Saving at each rate, against the Lagrangian oracle measured inside the same run and the architectural ceiling (dotted). At 0.1 dB the rule is above both heads at every rate, and the gap between the two heads is wider than the gap between the rule and either of them. At 0.3 dB the three lowest rates saturate and the three curves separate only at q48 and q63.

Drawn by scripts/supp_router_figs.py from the six files named in the table above and results/saturation_RECIPES12_ctc53.json.

At the working budget. Against the joint head the rule is ahead by +0.92 points at its narrowest and +6.85 at its widest, the latter at q63. Against the frozen head it is ahead by +0.52 to +3.35 points. The rule reaches 27.1% at q0 and 11.6% at q63, and it is ahead at all 5 measured rates whichever head it is measured against. A head trained on this decoder against this oracle returns nothing over a rule with no parameters, while carrying a compute share that the rule does not.

At 0.3 dB the low rates saturate and the arithmetic is exact. At q0, q16 and q32 all three configurations send every tile to the shallowest exit, so the maps are identical and the only difference left is the price of deciding. The joint head falls 0.044 points below the 39.1% ceiling and the frozen head 0.163, which are their compute shares of 0.044% and 0.163% to three decimals. At q48 and q63 the budget still binds and the allocations separate: the rule reaches 37.7% and 34.3% against the joint head's 37.0% and 34.1% and the frozen head's 33.4% and 27.7%. A looser budget gives an allocation more room to be wrong in as well as more room to be right, and the frozen head uses it to be wrong.

Why 0.5 dB is not in that table. At 0.5 dB every rate saturates for every configuration, so all three maps are identical and the only thing a comparison could measure is the cost vector each file was written with. The rule's 0.5 dB file is on runs/RECIPES12/ckpt_eval.pth.tar, where the shallowest exit was priced at 0.5809 against the pinned 0.6088, a difference worth 2.79 points of apparent saving. The file is results/raterank_RECIPES12_b05.json and it records that the rule matches the oracle map exactly at all five rates there, which is the only thing read from it.

q	rule agrees	p depth	p level	p spread	oracle
0	0.453	+0.54	+0.83	+0.61	29.90
16	0.320	+0.52	+0.84	+0.65	25.62
32	0.283	+0.53	+0.86	+0.71	20.93
48	0.353	+0.53	+0.84	+0.72	18.33
63	0.392	+0.39	+0.88	+0.68	15.64

Table 55. Why the rule works, and it is not by agreeing, at 0.1 dB. Compare the second column with the fifth: the rule picks the oracle's exit on a minority of tiles and still tracks the oracle's saving. The three Spearman correlations are between a tile's bit count and, respectively, the depth the oracle assigns it, its mean distortion across the ladder, and the spread between its shallowest and its deepest exit. *oracle* is the Lagrangian oracle measured inside the same run, which is the ceiling the rule is chasing.

results/raterank_RECIPES12_b01.json. The file stores spearman_bits_vs_exit against the negated exit index, as scripts/raterank_curve.py computes it, so the correlation with depth quoted here is its negative. The oracle column is measured at one frame per sequence and is not charged for the exit map; deployed configuration A, which carries the map, reads 29.9, 25.6, 20.9, 18.3, 15.6% over the same five rates at two frames per sequence (results/signalled_RECIPES12_ctc53.json).

The rule agrees with the oracle on 0.28 to 0.45 of tiles, well under the frozen head's held-out 0.718, and saves more at every rate. Agreement counts a disagreement between two exits that are within a hair of each other exactly as heavily as one that costs most of the frame's error, and most disagreements are of the first kind. What the rule gets right is the ordering. A tile's bit count correlates with the spread across the ladder, which is how much that tile stands to gain from depth, at 0.61 to 0.72 at every rate, and with the depth the oracle actually assigns at +0.39 to +0.54.

What the rule cannot do is see past that ordering. A rank-1 model gives every tile the same relative profile over exits, so b(t) decides where on the ladder a tile falls and never the shape of its trade-off. That is the ceiling this baseline sits at, and it is the part a learned head would have to earn its parameters on. Neither of our two heads does.

F.5. How much a trained head varies

No head in this work was trained twice at two seeds, so there is no seed variance to report and none was measured; results/router_ablation.json records seed 0 for all six of its runs and one_checkpoint true. A second training run is a training run, and the evaluation queue that produced the rest of this supplement was restricted to inference. What can be reported is the spread between heads that were trained independently of each other, which is a looser quantity than seed variance and a larger one.

Two heads on one decoder. The joint and frozen heads differ by +2.43 points at q0 and by -6.01 at q63, a range of 8.44 points across the five rates at 0.1 dB, and they cross: the joint head is the better of the two at q0 and the worse at every other rate. At 5 of the five rates the two heads differ from each other by more than the rule's margin over whichever of them is nearer, so a comparison against one trained head says less than it appears to. That is the reason both are printed in F.4 rather than the better of them.

q	oracle	head	rule	margin	rule agrees
0	34.02	31.56	30.92	-0.64	0.479
16	27.67	25.07	27.14	+2.07	0.370
32	22.29	16.47	22.39	+5.92	0.399
48	19.75	13.27	19.10	+5.83	0.431
63	17.19	8.99	16.16	+7.17	0.479

Table 56. The same comparison on a second training run. The margin column is the point: it is positive at four of the five rates and wider than on the pinned run, so the pinned numbers are the conservative ones. BEST is a separate recipe at a different epoch with its own head, trained the same way. The rule is ahead at 4 of 5 rates, by up to 7.2 points, and stays within 3.1 points of the oracle everywhere. The one rate it concedes is the only rate on either run where a trained head finishes ahead, and it does so by under a point.

results/raterank_BEST_compare.json: the BEST run, 53 sequences, 0.1 dB, not the pinned checkpoint.

q	before	after	change	β before	β after
0	27.18	30.27	+3.10	136.6	19.0
16	23.27	25.82	+2.54	88.8	7.7
32	19.40	19.43	+0.03	9.1	-19.1
48	16.06	15.54	-0.51	-39.7	-35.9
63	12.94	9.58	-3.36	-50.3	-42.2

Table 57. Agreement rises and the saving does not follow. The change column has both signs. The head was retrained after the exit mask was fixed, on the same recipe at the same λ ; held-out agreement went from 0.718 to 0.808 while the deployed saving moved by +3.1 points at q0 and -3.4 at q63. β is the cost multiplier the bisection applies to move a head trained at one λ onto this operating point, and the retrained head is ahead where that multiplier is small and behind where it is large. This is the same lesson as the rule's low agreement, from the other direction.

results/router_retrain_compare.json. The file records no checkpoint, and its before column reads 27.18% at q0 where the pinned measurement in results/router_RECIPES12_b01.json reads 24.41%, so the two are not on one basis. Only the comparison of the two heads within the file is read here.

F.6. What this section does not measure

- No seed variance, for either head or for any ablation variant. Every run in results/router_ablation.json is seed 0 on one checkpoint, and nothing here says the ordering of the input groups would survive a second seed or a second decoder.
- The ablation reports agreement and nothing else. No variant was carried through a budget bisection to a deployed saving, so this section cannot say what a stem-only head would save, and the argument that agreement and saving come apart applies to the ablation as much as to the head.
- The frozen-decoder head was trained against one checkpoint and evaluated on another. It was trained against runs/RECIPES12/ckpt_eval.pth.tar, which the watchers overwrite, and then run on the pinned checkpoint (results/router_RECIPES12_b01.json, field router2_meta), while the ablation heads were trained against the pinned checkpoint itself. Its 0.718 and the ablation's 0.7721 are two measurements of different things.
- One head covers all five rates. A head per rate is the obvious remedy for the cost multiplier in the retrain table and is not measured here.
- The head is timed at batch 1 on one GPU class, and its share of the decode in seconds is three times its share in operations. What that factor would be on a device where kernel launches are cheaper is not measured.
- The rule's 0.5 dB file is off the pinned checkpoint, which is why that budget is described rather than tabulated.

G. Algorithms, and how each step is implemented

G.1. What this section covers

Sections A to F state what the system is, what it costs and what it delivers. This one states how it is done: the five procedures the work rests on, each as an algorithm box whose every line names the function that carries it out and the shape it returns, followed by the tensor operations underneath them and the failure each one is written to avoid. A reader who wants to reproduce the system reads A for the recipe and this section for the moves.

The geometry is fixed throughout and is the pinned checkpoint's. The ladder has $K = 6$ exits over the released decoder's 12 trunk blocks, so $b = 2$ blocks per exit, and the split depth is $j = 2$, which leaves exits 2 to 5 selectable and the first 4 blocks full-frame. One tile is 256 px of RGB, 32 px of the feature grid the per-tile trunk runs on and 16 px of the latent. A 1920×1080 frame is replicate-padded on its bottom and right to 2048×1280, which is a feature grid of 160×256 and 40 tiles in 5 rows of 8. Write T for the tile count of a frame, c for the per-exit cost vector in units of one released decode, and M for the table of per-tile errors with one row per tile and one column per exit. Those three objects are what every procedure below manipulates.

Procedure	File	Entry point
the deployed decode	decoder.py	forward
the same, tiles ordered by depth	decoder.py	_suffix_sorted
per-exit price of a tile	cost.py	exit_costs
frame-level price of a map	cost.py	frame_relative_cost
MACs a decode performed	measure.py	MacMeter
saving, as the meter counts it	measure.py	measured_saving_pct
per-tile error at every exit	eval.py	tiled_exit_msos
error a mixed map delivers	eval.py	true_frame_mse
the encoder-side search	signalled_curve.py	at_lam, bisect_to, true_db
cost of the exit map, in bits	signalled_curve.py	map_bits
the tilt and its bisection	beta.py	exit_map, bisect_beta
per-tile logits	head2.py	StemRouterHeadV2.forward
which tiles to override	hybrid_curve.py	hybrid_map, h2
one training step	train_flexuf_image.py	train_one_epoch
the training forwards	model.py	forward_random_depth
the objective	losses.py	multi_exit_rd_loss
adapters and the seam gate	decoder.py	build_adapter, GridSeamRepair
tile-border padding	padding.py	wrap_tile_padding

Table 58. Where each procedure lives. decoder.py and padding.py are under flexuf/backbone/ and head2.py under flexuf/router/; the two curve scripts are under scripts/; train_flexuf_image.py is at the repository root and the rest are flexuf/ itself. Take from it the division the rest of the section relies on: everything that decides an allocation is in the scripts and in beta.py, everything that executes one is under backbone/, and the two meet only through the two objects in the middle of the table, a table of per-tile errors and a vector of per-exit prices.

G.2. The encoder-side search

The encoder holds the source, so it can evaluate the allocation rule exactly rather than predict it. Doing so means building the table M once per rate and then running an argmin over its columns for every candidate multiplier, which is why the expensive part is hoisted out of the search entirely: the per-tile errors, the released decoder's reference error and the bitrate are functions of the latent alone, and only the argmin and the aggregates move with the multiplier. Both loops of the bisection then cost tensor operations on a table of 40 rows by 6 columns.

input frames F, budget t in dB, cost vector c of length K

```

1 for each frame in F: once per rate
2   xp = replicate-pad to whole tiles, bottom and right only
3   y, q, aux = net._encode_to_latent(xp, qp)
4   M = tiled_exit_mses(net.dec, y, q, xp, cfg) [T, K]
5   R = reference_frame_mse(ref.dec, y, q, xp) scalar
6   floor = trueDb(0) one decode per frame
7   if floor > t: record budget_reachable = false, stop
8   inner = t
9   repeat at most 6 times:
10    lam = bisectionTo(inner) no decode
11    d = trueDb(lam) one decode per frame
12    if |d - t| < 5e-4: leave
13    inner = min(1, max(1e-4, inner + (t - d)))
14  for each frame, at that lam:
15    k = (M + lam · c).argmin(1) [T], the map
16    bits = map_bits(k, K); bpp = bpp + bits / pixels
17    saving = measured_saving_pct(net.dec, ref.dec, y, q, clamp(k, j))
18  report d, and the mean over frames of saving and bits
where
  k(lam) = (M + lam · c).argmin(1)
  tableDb(lam) = 10 log10( meant [M[t, k(lam)]] / R )
  trueDb(lam) = 10 log10( true_frame_mse(dec, y, q, xp, clamp(k(lam), j)) / R )
  bisectionTo(u): lo, hi = 0, 1; 60 times: mid = (lo + hi) / 2;
  if tableDb(mid) ≤ u then lo = mid else hi = mid; return lo

```

Table 59. The encoder-side search as scripts/signalled_curve.py runs it, one line per call. A.12 gives the same search in words and explains why there are two levels; what this adds is the shape at every step and the two places the map is clamped. Lines 4 and 5 are the whole expense and they are outside both loops. Line 15 is the only arithmetic the multiplier touches.

The inner bisection of Table 59 needs a bracket, and the bracket is the unit interval while the answers sit far down inside it: at the 0.1 dB budget the multiplier runs from 5.1×10^{-5} at q0 to 2.9×10^{-6} at q63. The first 18 halvings at the highest rate, and 14 at the lowest, are spent bringing the candidate down to the right order of magnitude before the search resolves anything inside it; the forty or so that remain are what locate the multiplier. The test at the top of the bracket comes first and returns it unchanged when the budget is still not met there, so a multiplier of exactly 1 in the results file is a flag rather than a measurement, and D.8 reads it that way.

Both levels rely on the same monotonicity. Raising the multiplier makes compute expensive relative to error, so tiles fall to shallower exits, the saving rises and the distortion rises with it; neither turns back, so a bisection is valid on either quantity and the code bisections on distortion because that is what the budget is written in. The outer loop exists because the two distortions are not the same quantity. The table measures a tile with every other tile at the same exit, a routed frame is mixed, and a tile's border sees whatever depth its neighbour chose, so the delivered figure sits a little away from the table's. Shifting the inner target by the observed difference and re-bisecting closes it in two or three passes at one decode per frame per pass, against one decode per bisection step if the search were run on the delivered figure directly.

What the table costs is settled twice over. Building it is one tiled decode per exit, and a shallow decode is cheaper than a deep one, so the arithmetic is the sum of the per-exit costs: 4.50 released decodes, from the hook counts in results/ceiling_measured.json. Timed on a frame rather than added up, it is 4.52 decodes. Of that, 1.23 decodes are two duplicate columns: the clamp makes exits 0, 1 and 2 the same decode, and tiled_exit_mses runs the loop over all K exits so that an argmin over all K columns lands on a choice the decoder can carry out. The alternative table, taken from a single full-frame pass with a tap at each exit, costs 1.37 decodes and must not be used to report quality, because it

runs the trunk full-frame and the tiling penalty then cancels against a full-frame reference. It can still be used to rank: on that frame the two searches agree on 82.5% of tiles and land 0.68 points of saving apart.

Sources: results/ceiling_measured.json for the per-exit hook counts (pinned checkpoint, q32, 2048x1280, 40 tiles); results/supp_encoder_cost_PAPER.json for the timings (pinned checkpoint, Bosphorus at q63, 15 iterations on an NVIDIA RTX A6000, one tiled decode at 114.9 ms); results/signalled_RECIPES12_ctc53.json for the multipliers, pinned checkpoint, 53 sequences. The main paper quotes the same ranking comparison from results/encoder_cost.json, which is the identical measurement on an earlier checkpoint and reads a little differently; the pinned file is the one used here.

Two lines of the box decide what the reported numbers mean. Line 15 runs the argmin against the arithmetic cost model, because a bisection evaluates it for every tile at every candidate multiplier and a decode per candidate is not affordable. Line 17 counts what the chosen decode actually performed, with forward hooks, and that is the figure every table in this paper quotes; B.6 prices the difference between the two and shows it is a closed form rather than a tolerance. A mistake inside the model therefore costs a slightly worse allocation and cannot inflate a headline.

The clamp appears twice and is absent once, which matters where the map is coded in G.5. Line 17 clamps before measuring and trueDb clamps before decoding, so both are priced and measured at the exit that runs. Line 15 does not clamp, and argmin returns the first minimal index, so a tile whose cheapest choice is the shallowest reachable exit is recorded as exit 0. The price is still right, because the cost model gives exits 0, 1 and 2 the same value; the picture is still right, because the decoder clamps; the only quantity that sees six symbols where four exist is the entropy of the map.

G.3. Routing at the decoder

Configuration B adds no bits, so the decision has to be made from what the decoder already holds, and everything it reads is already in hand at the moment the decision is needed. The stem is the feature after group 1, which the decode has to run whatever the map says; the decoded latent and the entropy model's scales fall out of the pass that produced them; the quality index is one number per frame. Only the head's own convolutions are new arithmetic, and F.2 prices them at 0.163% of a decode.

input latent y, quant step q, scales, quality index qp, tilt beta

```

1 stem = dec.upsample(y)
2 for g = 0 ... j-1: stem = dec.groups[g](stem) already needed
3 z = head2(stem, y, scales, qp, 32, 16) [T, K]
4 z = z[:, j:] drop the exits that cannot run
5 lp = log_softmax(z, 1) [T, K-j]
6 optionally lp = lp / mean(lp) per frame
7 k = (lp - beta · c[j:]).argmax(1) + j [T], the map
8 saving = measured_saving_pct(..., k) - 100 · rshare
and the budget is met by bisecting the tilt
  bisectionTo(u): lo, hi = -2e4, 2e4; 60 times: mid = (lo + hi) / 2;
  if tableDb(mid) ≤ u then lo = mid else hi = mid Table 59's, with k from line 7
  then the same outer correction as Table 59

```

Table 60. Routing at the decoder, as flexuf/beta.py and scripts/router_curve.py run it. Take from it lines 4 and 7, which are one operation split in two: the logits are sliced down to the exits that exist, and the column index is put back on the exit axis afterwards. Line 8 charges the decoder for the head it ran, which configuration A never runs.

Line 4 of Table 60 is a slice where the natural spelling is a mask, and the reason is a defect that reached the numbers. The head suppresses the exits below the split by assigning them a large negative constant, which is a suppression only while the head's own logits stay well above that constant. Nothing in a cross-entropy or a regret objective penalises a common offset, one drifted in, and the raw outputs settled at the scale of the mask itself, at which point the masked entries were the largest in every row and the argmax selected an exit that does not exist. The clamp

in the decoder then turned that into the shallowest real exit, for reasons having nothing to do with the tile. Every decision site now reads the sliced logits, the head files were left alone because live training runs import them, and A.6 lists the four places the same bound has to be applied.

The tilt needs a wider bracket than the multiplier does. A confident head has log-probability gaps of hundreds, so a tilt of a few tens moves nothing at all, and the bracket runs to plus and minus 2×10^4 ; at the 0.1 dB budget and q0 the bisection settles at 146. The outer correction against a real decode is the same as in the search above and lives in the same two functions, because the calibration experiment has to run the identical bisection on held-out images and then apply its answer unchanged to the test frames; a bisection copied into a second caller is a second bisection, and this project has twice paid for a selection rule that was fixed in one copy and not the other.

Line 6 is available to a decoder and is worth stating for that reason. Dividing a frame's log-probabilities by their own mean magnitude is a statistic of the decoder's own output, so it adds no bits and reads nothing from the source; what it buys is that one global tilt means the same thing on a frame the head is confident about and on a frame it is not. Line 8 subtracts the head's own arithmetic from the saving, measured by the same hook count applied to the head alone rather than assumed.

G.4. Choosing what to override

Configuration C signals the exit of a fraction of the tiles and lets the decoder predict the rest, so it needs a rule for which tiles are worth the bits. The rule is the one the objective itself supplies. At a fixed multiplier the frame's Lagrangian cost is a sum of independent one-tile terms, so replacing the router's choice on a set of tiles removes exactly the sum of those tiles' regrets, and the best set of a given size is the set of largest regrets. The selection is therefore a topk, and the recovered fraction as a function of the signalled fraction is the Lorenz curve of the regret distribution, which is the object C.7 works with.

```

input M [T, K], lp [T, K-], tilt beta, multiplier lam, fraction rho
1 kr = (lp - beta * c[j:]).argmax(1) + j what the decoder would do
2 if rho ≤ 0: return kr, 0 exactly configuration B
3 L = M + lam * c [T, K]
4 ko = L.argmax(1) what the encoder would do
5 if rho ≥ 1: return ko, T exactly configuration A
6 d = L.gather(1, kr) - L.gather(1, ko) regret, [T], never negative
7 S = d.topk(round(rho * T)).indices
8 k = kr.clone(); k[S] = ko[S]
9 return k, |S|

```

outside the rule

```

beta is bisected once, to what configuration B needs for the budget
lam is re-bisected at every rho, over the signalled tiles
bits = T * H2(rho) + 3 * |S|
the router's own cost is charged unless rho = 1

```

Table 61. The override rule, as scripts/hybrid_curve.py runs it. Take from it that the two endpoints are reached exactly rather than approached: at rho = 0 nothing but the router's own argmax is evaluated, and at rho = 1 the router's output is not consulted at all and its compute is refunded, so the columns between them measure an interpolation between two systems rather than three unrelated ones.

One multiplier for both branches was tried first and does not work, for a reason the two bisected values make plain. The router's term is a log-probability and the encoder's is a squared error, so tying them together needs a conversion constant, and the head is confident enough that the constant is of order 10^{-6} : a shared multiplier would have to run to 10^8 before the router responded at all, and at that multiplier the oracle branch ignores distortion entirely and sends every signalled tile to the cheapest exit. Measured at the 0.1 dB budget and q0, the tilt lands at 146 and the multiplier at 5.3×10^{-4} . Keeping them separate also matches what each one is: the router is a fixed component whose operating point does

not depend on how many tiles are signalled, and the multiplier is the only knob left to absorb the quality the overrides buy back.

results/hybrid_RECIPES12_b01_fixed.json, pinned checkpoint, 53 sequences at one frame each, with the router head runs/RECIPES12/routers2/v2_lam1.3e-5.pth, itself trained against runs/RECIPES12/ckpt_eval.pth.tar and not against the pinned checkpoint. F.2 carries that provenance in full.

Two properties of the rule are asserted on synthetic tables in tests/test_hybrid_map.py rather than argued: that the overridden set is exactly the tiles of largest regret, with the smallest overridden regret at least the largest untouched one, and that the objective is monotone in the signalled fraction. The endpoint checks are there too, at the level of the returned map rather than of the summary statistics, so a rho of 0 that merely happened to score like configuration B would not pass.

G.5. What the map costs to send

The exit map is the one thing configuration A adds to the bitstream, and it is charged before any saving is computed. What is charged is an entropy and a fixed side cost, in four lines.

```

input exit map k of length T, symbol count K
1 p = bincount(k, minlength = K) / T the frame's own histogram
2 H = - sum over symbols of p log2 p bits per tile
3 bits = H * T + 8K payload, then the histogram
4 bpp = bpp + bits / pixels before any saving is computed

and for configuration C, which sends a different object
bits = T * H2(rho) + 3 * |S| the mask, then one symbol per override

```

Table 62. The map coder, in scripts/signalled_curve.py and scripts/hybrid_curve.py. Take from it that line 3 charges the histogram at a flat byte a symbol rather than coding it, and that no arithmetic coder is run at any point: the payload is the entropy, which is what an ideal coder would spend and a real one a little more of.

At the 0.1 dB budget that comes to 79 bits a frame at q0 and 95 at q63, of which 48 bits are the histogram in both cases; the payload itself is 31 and 47 bits. Against a mean of 33.3 tiles a frame over the same test set, that is 0.94 and 1.41 bits a tile, where a fixed-length code over the 4 exits that exist would spend 2. The map grows with the rate because the allocation spreads out: at low rate most tiles take the same exit and the histogram is concentrated, at high rate the mass is spread over the ladder and the entropy rises. As a share of the bitstream it runs to at most 9.1×10^{-5} bpp, and every row of the results file records both the bits and the bpp they became.

results/signalled_RECIPES12_ctc53.json, pinned checkpoint, 53 sequences, rows at the 0.1 dB budget. The tile count is the tiles_per_frame field of results/hybrid_RECIPES12_b01_fixed.json, measured on the same frames.

Two further things make the reported cost an upper bound, and they are worth naming, because understating our own overhead would be the same class of error as overstating our saving. Line 1 counts the unclamped argmin, so the cheapest allocation is spread over the three indices that share one cost and counted as three symbols where a codec would signal one; the comment in signalled_curve.py records the effect as at most 7 bits a frame at q0 and 0 at q63. And the three raw bits a configuration C override pays would code one of 4 exits in two.

Configuration C sends a mask saying which tiles were overridden and a symbol for each override, so its bill is the binary entropy of the overridden fraction over the tiles, plus the overrides themselves. At q0 and a signalled fraction of 5% that is 14 bits a frame for 1.6 overrides, against nothing at all at a signalled fraction of zero, which is what makes that column exactly configuration B. The binary entropy is checked against its closed form in the test file rather than trusted.

G.6. One training step

The training step is where the pinned checkpoint's behaviour is decided, and the shape of it is not the shape the objective is usually written in. A step draws one exit per tile uniformly from the reachable ones, decodes the batch once at that mixed depth, and measures the reconstruction against the source. The per-exit auxiliary losses are not evaluated within

a step at all.

```

input crop x [B, 3, 512, 512], quality index qp [B], multipliers lam [B]
1 lr = schedule[epoch + 99]; new modules at 20× per epoch, A.11
2 y, q, aux = encode(x, qp) analysis, pass 1
3 m ~ uniform over j ... K-1, shape [B · T] fresh every step
4 xhat = dec(y, q, exit_map = m) ONE mixed-depth decode
5 L = mean( lam · mse(x, xhat) ) + bpp(aux)
6 y2, q2 = encode(x, qp); ref = frozen.dec.forward_full(y2, q2) pass 2, no grad
7 y3, q3 = encode(x, qp); deep = dec.forward_full(y3, q3) pass 3
8 L = L + wa · mean(lam) · mse(deep, ref)
9 y4 = encode(x, qp); f = dec.exit_features(y4) pass 4
10 L = L + wd · distill(f)
11 (L / accum).backward()
12 if the window closes: norm = clip_grad_norm_(0.1);
    if norm is not finite: skip the step else optimizer.step()
13 every 200 steps: one full-frame forward_all_exits, for the per-exit log

```

Table 63. One training step of the pinned run, as `train_flexuf_image.py` executes it. A.10 and A.11 give the recipe and the value of every constant; what this adds is the order, the four analysis passes and the two lines that are diagnostics rather than objective. Line 3 draws from the reachable exits only, which is the same bound the decoder's clamp enforces.

Line 4 of Table 63 is the choice that separates this recipe from a plain multi-exit one. Decoding every exit separately trains a condition that never occurs, because a deployed frame is mixed: a tile at exit 2 sits next to one at exit 5 and the head has to stitch across that discontinuity. One decode at a random mixed depth trains the condition that always occurs, and it costs one decode a step instead of K . The relation to the weighted objective is exact enough to state. With the auxiliary weights all equal, which is what the pinned run sets, the random draw makes the step's distortion term an unbiased single-sample estimate of the equally weighted mean of the per-exit errors. The difference from evaluating that mean directly is what the neighbours are doing: the estimator measures each exit with its neighbours at random depths, and the direct form measures it with every neighbour at the same depth. That difference is the reason for the design, and the main paper's training subsection measures what closing it is worth.

Lines 2, 6, 7 and 9 are four analysis passes over the same batch. The quantiser is a rounding with a straight-through gradient and the noise the rate estimate uses is drawn separately from it, so the four passes compute the same latent, and with the encoder frozen none of them can diverge from the others. The model carries a single entry point that produces every term from one pass and exists so that `DistributedDataParallel` sees each parameter inside one forward call; the runs here are one GPU each, the trainer calls the pieces directly, and the redundancy is what that leaves behind. It costs wall clock and nothing else, and no file in results/ measures how much.

Line 12 is the released recipe's guard, applied to the parameters that are actually training rather than to all of them: the norm is clipped at 0.1 and a batch whose norm is not finite is dropped instead of being allowed to reach the weights. Gradients accumulate over a window and the loss is divided by the window length, so the accumulated gradient equals what a single large batch would produce; the pinned run uses a window of one. Line 13 exists because line 4 returns a single mixed-depth reconstruction and therefore no per-exit breakdown, and that breakdown is the health signal for the failure this literature reports most often, a ladder whose exits converge on each other until only two branches are alive. One extra full-frame pass every 200 steps recovers it for the log at a fraction of a percent of training time.

The joint path exists in the same file and is not what the pinned checkpoint trained under. It replaces the random draw with the router's own sample, taken through a Gumbel-softmax with a hard argmax, and multiplies the chosen tile's output by the selected probability divided by

its own detached copy. That factor is numerically 1, so the reconstruction is bit-identical to the deployed path, while the gradient of the rate-distortion loss reaches the logits that an argmax would have blocked. A.7 describes the gate as it appears in the decode; the launcher for the pinned run does not pass the flag that switches it on.

G.7. The operations underneath

Underneath the five procedures are perhaps a dozen operations that have to be exactly right and are silent when they are not. Table 64 writes each one out with the file and the function it lives in. Shapes are given for the pinned geometry: T tiles, $C = 384$ channels, a feature tile of 32×32 and an RGB tile of 256×256 .

The operation	File and function
tiles and their order	
patchify: <code>x.view(B, C, nh, P, nw, P).permute(0, 2, 4, 1, 3, 5).reshape(B·nh·nw, C, P, P)</code> , with P the feature tile. No arithmetic, and it asserts divisibility rather than padding, so a ragged last row stops the run	decoder.py, patchify
unpatchify: <code>x.view(B, nh, nw, C, P, P).permute(0, 3, 1, 4, 2, 5).reshape(B, C, nh·P, nw·P)</code> , the exact inverse	decoder.py, unpatchify
with a halo, in <code>patchify_with_halo</code> : replicate-pad by h , then <code>unfold(2, P+2h, P).unfold(3, P+2h, P).permute(0, 2, 3, 1, 4, 5)</code> , reshape, contiguous	decoder.py
per-tile error: $e = ((xhat - x)^2).mean(1)$, then the same permutation at the RGB tile size, then mean over the tile's pixels	eval.py, per_tile_mse
per-tile router features: the same permutation at the feature tile size, then mean and standard deviation over the tile	head2.py, _pool
per-tile bits: the same permutation at the latent tile size, then a sum over the tile	hybrid_curve.py, main
the exit map	
the clamp: <code>exit_map.to(device).long().clamp(min = j, max = K-1)</code> , before the per-tile loop and after the shape assertion. A.6 lists the four places that have to repeat it	decoder.py, forward
sorted execution: <code>order = argsort(m, descending, stable)</code> ; <code>bounds = (m_sorted[None, :] > arange(j, K))[:, None]</code> .sum(1).tolist(); each group is then a slice, and the canvas is restored with <code>out[inv]</code>	decoder.py, _suffix_sorted
the modules the ladder adds	
adapter choice: the FFN when the exit skips $(K-1-k) \cdot b \geq 4$ blocks, the 1×1 otherwise, and none at the deepest exit, whose feature goes to the head raw	decoder.py, build_adapter
adapter form: $f + Wf$ for the 1×1 ; $f + PW(\text{act}(PW(f)))$ for the FFN, $C \rightarrow 4C$ then $C \rightarrow C$ because the gated activation folds 4:1	decoder.py, the two adapter classes
adapter initialisation: the LAST convolution's weight and bias are zeroed, and zeroed again by <code>_zero_init_adapters</code> after the released model's own initialisation pass has run over the whole network	decoder.py; model.py
seam repair: $x + G[r \bmod P, c \bmod P] \cdot PW(WSiLU(DW3 \times 3(x)))$, with G a 32×32 gate initialised to $\exp(-d/\tau)$, d the distance in feature pixels to the nearest tile edge, and PW zeroed	decoder.py, GridSeamRepair
the gate laid on the canvas: <code>gate.repeat(1, 1, ceil(H/P), ceil(W/P))[:, :, :H, :W]</code>	decoder.py, the gate in forward
tile padding: the 3×3 depthwise convolutions of groups j and deeper have their padding mode reassigned, or are wrapped when the scheme is not one of PyTorch's, and are restored afterwards	decoder.py, _set_tile_padding; padding.py, wrap_tile_padding
the two ways of counting	
the model: <code>c_k</code> assembled from the measured shares with <code>k_run = max(k, j)</code> applied to both the block count and the adapter	cost.py, exit_costs
the meter: a forward hook on every <code>Conv2d</code> and <code>Linear</code> , MACs from the output shape, output positions taken as <code>numel</code> divided by channels	measure.py, MacMeter
the ratio: routed MACs on our decoder over released MACs on the reference, both on the same latent, so no calibration constant enters	measure.py, measured_cost

Table 64. The operations, with the file and the function for each, under the directories Table 58 gives. Take from it the six rows in the first two groups: the same row-major tile ordering is written out four times, at four different grid resolutions,

and the exit map is indexed by all four. Only the first pair has an inverse and a test asserting round-trip exactness; the other two agree with it by construction and by inspection.

Shapes and module forms are read from the files named in the second column. The adapter selection and the two adapter costs are confirmed against `results/adapter_cost.json`, which measures the modules of the pinned checkpoint with hooks: the 1×1 costs 147,456 MAC per feature pixel and the FFN 737,280, against a trunk block's 1,035,648. B.3 prices them as shares of a decode.

G.8. Where a natural implementation is wrong

Six operations, most of them in Table 64, are written the way they are because the obvious alternative fails, and each failure is silent in the sense that matters: it produces a decode that runs, returns a plausible picture and reports a plausible number.

The four tile orderings. A tile's exit, its measured error, its router features and its bit count are computed on four different grids, at the RGB, feature and latent resolutions, and every one of them is indexed by the same integer. A permutation that disagreed with the others would deliver a perfectly valid decode of the wrong allocation, and the symptom would be a router that predicts poorly or an oracle that underperforms its own bound, neither of which points at a reshape. The pair that has an inverse is pinned by a round-trip test with zero tolerance. The other two are pinned only by the permutation being written identically, which is why the table above writes it out three times rather than saying that they agree.

The quantisation step and its batch. The decoder multiplies the trunk output by a per-quality-index scale of shape $[B, C, 1, 1]$ before the head. Handing a batch of those to a decode of one image broadcasts a $[1, C, H, W]$ tensor back up to batch B , without an error and without a warning, and the wrong-shaped answer surfaces somewhere unrelated. It happened twice, in two files, because nothing tied the call sites together, and forward now raises when the scale's batch is neither 1 nor the latent's. It is also why one of the two training forwards decodes image by image with the scale sliced per image, while the other can decode the whole batch at once with a single exit map of length $B\cdot T$.

Installing a padding scheme. Two of the four schemes are not PyTorch padding modes, so they are installed by wrapping the convolution: pre-pad by hand, then convolve with no padding. Three things about that wrapping are deliberate. The convolutions to be replaced are collected before any of them is replaced, because assigning a wrapper during a walk of the module tree inserts it into the tree being walked and the walk descends into it without end. A second wrapping is refused rather than allowed, because a wrapper whose inner convolution is another wrapper fails inside the convolution call with a message about a missing weight, far from the line that caused it. And the trained per-channel coefficient is one tensor held on the decoder and attached to each wrapper without being registered, since registering it would hand the optimiser one copy of the same parameter per wrapped convolution. The swap is applied only to the groups that run per tile and undone afterwards, which is what keeps the full-frame path bit-exact against the released decoder and the reference honest.

The phase of the seam gate. The repair pass is told where the seams are instead of having to infer them, by a gate indexed by position within a tile. That indexing is a modulo, and a modulo is only correct if the tile lattice starts at the origin. It does, because the frame is padded on its bottom and right only: 1920×1080 becomes 2048×1280 with 128 columns and 200 rows added at the far edges. Centring the padding, which is the natural thing to do to an image, would put every seam out of phase with the gate that was trained on it. The gate is also the one module in the ladder that is tied to the tile size: it has one scalar per position in the tile, so a checkpoint trained at one tile size cannot be evaluated at another without turning it off, and the evaluation script that allows a tile-size override says so at the point of the override.

The meter's batch dimension. A convolution's MAC count is its output positions times its output channels times its taps times its input channels

over its groups, and output positions here are the output's element count divided by its channel count rather than its height times its width. The per-tile trunk runs T tiles as T batch elements, so height times width would divide that part of the decode by the tile count; the comment in `flexuf/measure.py` records the deepest exit measuring 0.43 of a released decode under that reading, against the 1.011 in `results/ceiling_measured.json`. The hooks are registered when the context is entered and removed when it is left, so nothing leaks into the next measurement, and a module an early exit skipped costs nothing because its hook never fires. There is no bookkeeping to get wrong, which is the property the arithmetic model does not have.

The clamp inside the cost model. The decoder clamps the map, so a tile assigned an exit shallower than the split still runs the group at the split and wears that exit's adapter, and the model has to bill both the same way. A.6 gives the failure when it did not. What is left after the clamp is a residual with a closed form, measured per exit on the pinned checkpoint: the model under-bills by 0.0080 of a released decode at exits 0 to 3, 0.0027 at exit 4 and 0.0014 at the deepest, which is why the reported saving is the meter's and the model's part is confined to the inside of an argmin. B.6 takes the residual apart term by term.

`results/ceiling_measured.json`, pinned checkpoint, q32, 2048×1280 , 40 tiles. Its modelled column is `flexuf/cost.py` and its measured column is `flexuf/measure.py`, on the same latent. The reading attributed to the meter's earlier form is the comment in that file and is in no results file, which G.10 repeats.

G.9. The controls that hold it in place

Every property above that could fail silently has a test that fails loudly, under tests/. They are asserted at zero tolerance wherever the arithmetic permits it, on the principle that a control which cannot fail is not a control, and the one tolerance that appears is a property of a library rather than of the algorithm.

- **test_reference_is_the_release.py** decodes with Microsoft's own class and forward function and requires our deepest exit to match it with a largest absolute difference of zero, so the claim rests on the two models producing the same pixels rather than on the key remap having been audited correctly.
- **test_equivalence.py** holds six properties at zero tolerance: the warm-started ladder at its deepest exit is the released decoder; every untrained exit is that decoder truncated at its own depth; patchify and unpatchify round-trip losslessly; with $j = K$ the tiled path reproduces the full decode, which exercises the halo, the loop, the canvas, the crop and the head at once; every seam-repair variant is the identity at initialisation and the grid gate is concentrated on the border; and a mixed-depth map costs less than an all-deepest one.
- **test_cost_matches_reality.py** runs the decode, counts its MACs with hooks and requires the arithmetic model to agree, which is the control that was missing when the model and the decoder drifted apart three times.
- **test_sorted_tiles.py** requires sorted execution to give the same picture as the masked loop, on a mixed exit map and on random content, because a uniform map makes the permutation the identity and a constant image makes any permutation exact.
- **test_hybrid_map.py** requires the override endpoints to be exactly configurations B and A at the level of the returned map, the overridden set to be the tiles of largest regret, the objective to be monotone in the signalled fraction, and the mask cost to be the binary entropy.
- **test_router_mask.py** requires the masked exits to be minus infinity, never the largest logit, and never selected by the tilted argmax at any tilt.
- **test_router_inputs.py** requires a zeroed input group to be unable to move a logit at all, and every ablation variant to have one architecture and one parameter count, so that the variants differ in information and in nothing else.

What they are chosen to catch is never a quality regression. Each pins a property whose violation would leave every downstream number internally consistent and wrong, which is the class of defect this project has actually shipped. Two of them are careful about their fixtures for the same reason: the equivalence tests initialise the adapters randomly where a zero would make the property trivial, and the sorted-execution test decodes content rather than zeros, where every tile is identical and any permutation is exact.

The sorted-execution comparison is exact on CUDA and carries a tolerance of 1×10^{-6} on CPU, where oneDNN selects a different blocking at some batch sizes and a plain DepthConvBlock is already order-dependent at 13 tiles while being exact at 4, 9, 16 and 40. That tolerance is the library's and is recorded in tests/test_sorted_tiles.py; no results file carries it.

G.10. What this section does not settle

- **Nothing here is a quality measurement.** The numbers in this section price procedures and describe shapes. No quality figure is established here: the savings and the decibels are measured under the protocol A.4 fixes and reported in D and E.
- **The search timings are one sequence.** The cost of building the per-tile table is timed on Bosphorus at q63, 15 iterations on an NVIDIA RTX A6000, and it is a ratio to a decode on the same card rather than a distribution over the test set.
- **The step's time breakdown is not measured.** The four analysis passes are counted from the source. No file in results/ records what each term of the objective costs in wall clock, and A.10 records only the total for a step.
- **Two implemented paths are not the pinned ones.** The jointly trained router and canvas coupling are both implemented and both measured elsewhere in this document; the pinned checkpoint's configuration block records coupling off, and the launcher does not pass the joint-router flag. Where a number in this paper comes from either, it says so.
- **No test asserts that the four tile orderings agree.** Only the pair with an inverse is checked. The other two are checked by reading them.
- **The map cost is an entropy, not a coder.** No arithmetic coder is run over the exit map; the reported bits are the entropy of the per-frame histogram plus a flat byte a symbol for the histogram itself, which bounds what a real coder would spend from below on the payload and from above on the side information.
- **Three numbers here are typed from source comments.** The meter's earlier reading of the deepest exit, the CPU tolerance of the sorted-execution test, and the bits that merging the three equal-cost indices would save. Each is attributed where it is used, and no file in results/ carries any of them.

H. Failure cases, limitations and open problems

The method removes decoder compute by letting easy tiles leave the trunk early, and there are frames where no tile is easy, frames with too few tiles to allocate over, and content where the quality a budget buys is spent in a place a viewer would look. This section measures those cases rather than describing them, then lists the mechanisms that were built for this work and dropped, each with the measurement that ended it, and closes on the problems that are open at submission. Section D names five failure cases read off the operating-point files and is not repeated; what follows is the content axis, the abandoned work, and the state of the checkpoints every table in this document rests on.

H.1. The frames it does worst on

A frame is cut into tiles of 256 px, so the number of decisions the allocation has to make is fixed by the resolution. At 1920×1080 a padded frame carries 40 tiles and at 416×240 it carries two. Table 65 takes one frame from each of the five test classes at the same rate and the same budget and prints what the allocation did with it. Reading down the tile

column, the exit map goes from a mixture over three exits to a constant, and the saving falls with it.

frame	tiles	exits taken	saved %	delivered dB	worst tile dB
Bosphorus, 1920×1080	40	e2:15 e3:19 e4:6	28.1	0.100	0.310
BasketballDrive, 1920×1080	40	e2:14 e3:13 e4:11 e5:2	25.1	0.098	0.232
FourPeople, 1280×720	15	e2:3 e3:4 e5:8	13.8	0.100	0.338
PartyScene, 832×480	8	e3:2 e4:4 e5:2	12.3	0.098	0.238
RaceHorses, 416×240	2	e4:2	13.0	0.097	0.113

Table 65. One frame from each class at q32 and a 0.1 dB budget. The exits taken column is the histogram over the ladder, so e4:2 is two tiles at exit 4. Take from it that the allocation stops being an allocation at the bottom of the table: at 416×240 both tiles take the same exit, and the frame receives whatever that one rung happens to cost. The worst tile column is the largest per-tile loss in the frame and is between two and three times the frame's own delivered decibel on the three largest frames, which is the tail section D measures over the whole set.

The five results/supp_exitmap_*_q32_b01.json files, all on runs/RECIPE512/ckpt_PAPER.pth.tar, one frame each. These probes carry the modelled saving and no hook count, so each is optimistic by up to 0.8 points, the constant recorded in results/ceiling_measured.json; they are printed for the allocation and the per-tile loss rather than for the headline, which section D reports hook-counted over the whole set.

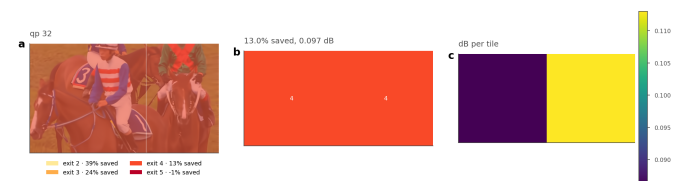


Figure 18. The smallest frame in the test set, and the whole of its allocation. a the decoded frame with the single tile boundary drawn, tinted by the exit each tile took. b the exit map, two cells. c the loss each tile pays. Both tiles take exit 4 and the map has nothing to trade; the legend gives what each rung would have saved had a tile taken it.

paper/figures/exitmap_PAPER_racehorses_416x240_q32_b01.png, drawn from results/supp_exitmap_racehorses_416x240_q32_b01.json on runs/RECIPE512/ckpt_PAPER.pth.tar. RaceHorses at 416×240, q32, 0.1 dB.

The two frames at either end of that table are the same decoder at the same budget. Bosphorus at 1080p spreads 40 tiles over exits 2, 3, 4 and saves 28.1% of the modelled decode; RaceHorses at 416×240 puts both of its tiles on exit 4 and saves 13.0%. Nothing about the content separates them. What separates them is that a two-cell map has three admissible states above the split depth and a 40-cell map has more states than the sweep could enumerate, so the Lagrangian has somewhere to put the budget in one case and not in the other.

sequence	q	tiles	PSNR lost dB	MS-SSIM lost dB	saved %	its rank
videoSRC02	q0	40	0.231	0.164	38.8	66th
videoSRC01	q0	40	0.143	0.154	38.8	66th
videoSRC26	q0	40	0.136	0.123	35.8	45th

Table 66. The three sequences that lose most on the perceptual metric. All three are 1080p frames at the lowest rate and carry the full 40 tiles. The MS-SSIM column is $-10 \log_{10}(1 - \text{MS-SSIM})$, the unit section D puts the two metrics in, and on these sequences it is two to three times the set mean of 0.052 dB at this rate. The PSNR column is the same file's own difference in PSNR and runs from 0.136 to 0.231 dB, over the nominal budget on all three, which is the per-sequence overshoot the next subsection measures. The last column is where each sits in the set by compute saved, from the 45th percentile to the 66th: these are not the sequences that save least.

The worst_sequences_by_ms_ssim block of results/supp_opquality_PAPER.json, on runs/RECIPE512/ckpt_PAPER.pth.tar, 53 sequences at one frame each, at the 0.1 dB operating point of results/supp_paper_curve_PAPER.json. That block is the file's own shortlist of the fifteen largest MS-SSIM losses over the whole sweep, reordered here by the decibel rather than by the raw difference. The saving column is the same file's per-sequence saving against our own deepest exit and the rank is its percentile among the 53 sequences of results/supp_per_sequence_PAPER_b010.json at the same rate.

The ordering there is the one to take away. A budget in PSNR does not price the second metric evenly across content, and the sequences where

the two disagree most are ordinary 1080p clips from the middle and upper part of the set by saving rather than the hard cases further down this section. On a clip with 40 tiles the multiplier can move a third of the frame two rungs down the ladder and stay inside 0.1 dB of PSNR, and the tiles it moves are the flat, low-texture ones a structural metric weighs differently from a mean squared error. The remedy is a budget written in the metric a deployment cares about, and no allocation in this work was bisected on anything but PSNR.

H.2. What a set-level budget hides

q	over budget	worst dB	on	least saved %	on
q0	34 of 53	0.245	videoSRC02	6.0	BQSquare
q16	33 of 53	0.311	videoSRC02	-1.0	BQSquare
q32	32 of 53	0.354	videoSRC29	-1.0	PartyScene
q48	29 of 53	0.290	videoSRC21	-1.0	BlowingBubbles
q63	33 of 53	0.237	videoSRC02	-1.0	RaceHorses

Table 67. The 0.1 dB budget, sequence by sequence. The multiplier is bisected once for the whole set, so the budget binds on the set mean and on nothing else. Rather more than half the sequences receive a decode worse than the budget they were nominally given, and the worst of them receives between two and three and a half times it. Section D.4 gives the spread on the compute axis; this is the same allocation seen on the quality axis.

results/supp_per_sequence_PAPER_b010.json, the rows block for the delivered decibel and rows_vs_release arithmetic for the saving, on runs/RECIPE512/ckpt_PAPER.pth.tar. Decibels are pooled, the convention results/supp_paper_curve_PAPER.json bisects in.

The other thing a set mean hides is how much of the budget is gone before any tile exits early. With every tile at full depth the decode already differs from the released decoder by 0.033 dB at q0 and 0.066 dB at q63, which is two thirds of the headline budget at the high rate. Two things make it up, and they are measured separately rather than subtracted from one another: the deepest exit's own departure from the released decoder, 0.036 dB at q63 on the same checkpoint, and the seam left by cutting the frame into tiles. Both are charged to the method, because the saving is measured against the released decoder rather than against our own full-depth decode, and both grow with rate while the budget does not.

Floors from results/signalled_RECIPE512_ctc53.json and drift from results/supp_anchor_PAPER.json, both on runs/RECIPE512/ckpt_PAPER.pth.tar over 53 sequences. The two files do not pool their decibels alike, which is why the seam is described rather than given as the difference. Section C derives why a floor above the budget makes the feasible set empty and section E tabulates the three ways the floor itself can be measured.



Figure 19. The seam, at frame scale. **a** the decoded frame. **b** the absolute error against a full-frame decode of the same latent, amplified 30 $\times$, with every tile at the deepest exit so that nothing here is early exiting. **c** a zoom on one junction. The tile grid is the error: 0.124 dB at q63, paid before the allocation begins.

paper/figures/seam_PAPER_bosphorus_q63.png, drawn from results/supp_seam_problem_bosphorus_q63.json on runs/RECIPE512/ckpt_PAPER.pth.tar. The panel is the worst case rather than the shipped one: the file records tile_pad_mode zeros with the deblocking filter removed, which is how the artefact is made visible. The floors quoted above are the shipped configuration, replicate padding with the filter on.

H.3. Built, measured and abandoned

Seven mechanisms were built, measured and then left out of the reported system. Two of them replace the border a tile is convolved against, one repairs the seam afterwards, one removes the seam by letting neighbouring tiles exchange features, one freezes the trunk so that the deepest exit cannot move, one gives the small resolutions a smaller tile, and the last is an explanation of the tiling penalty rather than a

mechanism. Table 68 gives what settled each, and the file it was settled in. The cost of building them is not otherwise recoverable from the paper, and two of them are the first thing a reader would propose.

what was built	the measurement that ended it
First-order border extrapolation	worse than replication at all three rates: 0.198, 0.286 and 0.384 dB against 0.071, 0.123 and 0.218
Fitted AR(1) border padding	ahead of replication by 0.021 dB at q63, and bought with wall clock that no file in results/ records
A learned deblocking filter	0.0008 dB even with a perfect gate, against 1.09% of a decode
Halo exchange between tiles	exact at uniform depth, and the saving falls from 25.9% to 4.2% at q32 and from 19.2% to 0.4% at q63
Freezing the trunk	drift exactly 0.000 dB, and the oracle ceiling collapses with it to 11.5% at q0, 0.4% at q32 and -1.0% at q63
A tile size chosen per resolution	worth -0.06 points on the set mean over three rates, with the training confound running in its favour
The affected-area fraction	wrong by 137 to 241%, against 13 to 22% for a power law in the per-tile block count

Table 68. Seven mechanisms and what settled each. Five were built and dropped, one is a proposed extension that measurement did not support, and the last is an explanation the paper used to give and no longer does. None of the seven is in the reported system, whose configuration is replicate padding at 256 px with the deblocking filter left on, as the ablation table of section A records.

Rows in order: results/ctc_seam_p256.json (40 sequences, three rates); the same file with results/shrink_ablation.log; results/seam_spatial_published.json with the module's share of a decode from results/supp_module_shapes.json; results/coupling_ablation.json; results/first_ckpt_headonly.log; results/tilesize_adaptive.json; results/supp_seam_vs_split.json, refitted in the build. The coupling and heads-only files are on runs/RECIPE512/ckpt_eval.pth.tar and runs/heads_only_j2_p256/ckpt_epo0.pth.tar; the seam-repair spatial file is on a runs/BEST checkpoint whose epoch was not recorded and which has since been overwritten, which the file itself says. The rest are on runs/RECIPE512/ckpt_PAPER.pth.tar.

Three of those rows need more than a line. The halo exchange was the largest remaining gain this work claimed, and two thirds of the claim survived: it is exact at uniform depth, and it removes 91% of the floor at q0 and 47% at q63. The third part, that closing the floor is worth several points of saving, is wrong in sign at every rate above the lowest. A tile whose neighbour ran six more blocks is reading an activation the trained weights have never seen, and the error that introduces is far larger than the seam it removed. Exactness holds when every tile is at the same depth, and routing is the deliberate violation of that condition, so the two are in tension by construction. The one caveat that keeps this from being final is that the model was trained with replicate padding, so switching the exchange on at inference is a distribution shift; a run trained with it could reverse the sign, and no such run exists.

The deblocking filter is the reverse case, where the mechanism does what it was built to do and the amount is too small to pay for. Error falls monotonically with distance from the nearest tile boundary, 4.7e-05 in the innermost 4 px against 4.1e-05 more than 64 px away, so the seam is real and it is local. The filter's gate leaks: it gains in the boundary band and loses slightly everywhere else. Giving it a perfect gate, with its boundary gain kept and its interior loss set to zero, bounds it at 0.0008 dB for 1.09% of a decode. Tightening the gate cannot rescue the module because there is not enough in the band for it to win. It is still switched on in the reported system, because it was trained jointly with the decoder and switching it off at inference is not the same experiment as training without it, and no run has been trained without it at 256 px.

The affected-area fraction is the one that was an explanation rather than a mechanism. It is a correct statement about which pixels a tile border can reach and a poor predictor of what the border costs, because it saturates: at 12 blocks per tile it is already 0.94 and cannot rise further, while the penalty still climbs by a factor of 3.3 from 8 blocks to 12. Fitted with one free scale it is wrong by 137 to 241% on average. A power law in the block count is wrong by 13 to 22%, with exponents 2.31, 2.08 and 1.80 at q0, q32 and q63 and log-log correlations from

0.973 to 0.994. An exponent near two has a reading: the number of affected pixels grows with the border's reach and the damage in each grows with how many convolutions reached it, and the two are the same quantity. The drift from 2.31 to 1.80 with rate is not explained here. The control is the row that licenses reading any of it as seam: at zero blocks per tile the penalty is 0.0000 dB at every rate.

The tile-size row is the one a reader is most likely to propose, since the exit maps of Table 65 say the method is governed by tile count. Splicing 128 px tiles into the two smallest classes and leaving 256 px elsewhere is worth -0.06 points on the set mean, averaged over three rates. The direction differs by rate rather than being uniformly flat: at q0 the smaller tile is ahead in five of six classes and the splice gains +0.33 points, while at q63 the smaller tile is behind on every class and the splice loses -0.33. Halving the tile side doubles the seam at fixed depth, and above the lowest rate the extra seam costs more than the extra granularity returns. The comparison is not matched for training time: the 128 px column carries 24,000 more steps than the pinned checkpoint, an advantage that runs in the smaller tile's favour, which is why the conclusion is that the extension is not supported rather than that it is harmful.

results/tilesize_adaptive.json, which splices results/per_class_RECIPES12.json on runs/RECIPES12/ckpt_PAPER.pth.tar against results/per_class_BEST128.json on runs/BEST128/ckpt_step.pth.tar. The file brackets the training confound with a third measurement on runs/RECIPES12/ckpt_PIN_e1.pth.tar and records the verdict per class and per rate; it also records that the per-class figures a previous version of this comparison used are not reproducible from anything now in results/, so the two are never quoted together.

The AR(1) padding row is the one this section cannot close from results/. It is the best border estimator measured, 0.197 dB at q63 against 0.218 for replication, and it was rejected on wall clock, which no file in results/ holds. What results/ does hold is why nothing cheaper works. Its coefficient is fitted per channel and per tile, and replacing the fit with a fixed shrinkage recovers none of the gain: at a coefficient of 0.95 the seam is 0.2138 dB against replication's 0.2160 and the fit's 0.1797, and every coefficient below that is worse than replication at every rate. The gain is in the adaptivity, and the adaptivity is what costs.

results/shrink_ablation.log, 10 CTC frames at $j = 2$ and 128 px, typed into this module because it is a log rather than JSON; the seam figures it is compared against are results/ctc_seam_p256.json, 40 sequences at 256 px, so the two sets of absolute values are not on one protocol and only the ordering within each is read. The wall-clock measurement that decided the estimator is recorded in the project's decision log and in no file in results/.

H.4. The checkpoints, and what a later one measures

Every table in this supplement is measured on runs/RECIPES12/ckpt_PAPER.pth.tar, which is the first epoch boundary of a run that has since passed three. The per-checkpoint watcher measured the epochs in between on the same 53 sequences at the same budget through the same code path, and Table 69 is what they say.

checkpoint	q0	q16	q32	q48	q63	mean
RECIPES12, epoch 0, pinned	29.2	25.0	20.4	17.9	15.2	21.5
RECIPES12, epoch 1	33.5	29.3	23.8	21.4	18.4	25.3
RECIPES12, epoch 2	32.8	28.1	22.5	19.9	16.9	24.0
BEST, epoch 1	34.0	27.7	22.3	19.8	17.2	24.2
BEST, epoch 2	29.4	23.2	18.0	14.5	11.5	19.3
BEST, epoch 3	30.8	24.7	19.2	15.6	12.2	20.5

Table 69. Compute saved at the 0.1 dB budget as training continues. RECIPES12 and BEST are two runs of one configuration, same architecture and same recipe, differing only in the draw. The pinned row is what the paper reports. Epoch 2 of the same run is hook-counted like the pinned row and is 2.5 points ahead of it; epoch 1 reads 3.7 ahead in a file that carries no hook count and is therefore about 0.8 points optimistic against the other rows. Either way the paper is quoting a checkpoint behind the run that produced it. Two measurements are not enough to call epoch 1 a peak, and moving the headline there would be choosing the better of two rather than reporting the latest.

results/signalled_RECIPES12_ctc53.json on runs/RECIPES12/ckpt_PAPER.pth.tar, and results/signalled_RECIPES12_0819_0556.json, results/signalled_RECIPES12_0820_0140.json, results/signalled_BEST_ctc53.json, results/signalled_BEST_0819_1050.json and results/signalled_BEST_0820_0841.json on the ckpt_eval and ckpt_step files of their runs, which the watchers overwrite; each records its own epoch. Savings are hook-counted where the file carries a hook count and modelled where it does not, which applies to the two epoch-1 rows and makes them optimistic by about 0.8 points against the rest. A difference between two rows that agree about the convention carries none of that offset, and the two comparisons this section draws conclusions from, epoch 0 against epoch 2 and RECIPES12 against BEST at epoch 2, are both of that kind.

One reading of these files was nearly published and was wrong, and it is worth recording because the mistake is available to anyone reading a directory of watcher output. Sorted by time rather than by test set, the same run appears to collapse: the earliest RECIPES12 measurement reads 30.3 and the pinned one 22.1 in the same convention, at the same epoch of the same run. The test set grew in between, from 40 sequences to 53, when MCL-JCV and the two smallest HEVC classes landed. Eight tiles at 832×480 and two at 416×240 save nothing like a 1080p frame does, so the whole apparent drop is the denominator. Four runs showed the same drop in the same direction at the same time, which is what made it look like a finding.

H.5. How far two runs of one recipe are apart

RECIPES12 and BEST are the same K, the same split depth, the same tile size, the same adapters and the same 45,445,398 parameters. At epoch 2 they are 4.7 points apart on the set mean. The ladder table of the main paper reports a 256 px run at 24.2 against a 128 px run at 19.3 and a twelve-exit run at 19.0, the last over the four rates it reaches rather than five, which are differences of about five points. The gap between two runs of one configuration is the same size as the architectural differences the table is ranking.

results/signalled_BEST_ctc53.json, results/signalled_BEST128_ctc53.json and results/signalled_FINE12_ctc53.json, the three files behind the main paper's ladder table, each on the ckpt_eval or ckpt_step file of its own run; none of the three carries a hook count, so the three are on one basis with each other and about 0.8 points optimistic against the pinned row of Table 69. The epoch-2 comparison above is between two files that both carry one.

The spread is not only between runs. The watcher writes a measurement every time it finds a new checkpoint, so one epoch of one run yields several, and Table 70 prints six of them from a single epoch. They span 1.8 points on the four-rate mean and up to 3.0 at a single rate, on one test set, one budget and one code path, with every row hook-counted.

checkpoint, in order	q0	q16	q32	q48	mean
0819_0923	30.3	24.8	16.9	10.8	20.70
0819_1239	28.9	22.9	18.2	12.4	20.60
0819_1555	30.8	25.3	18.3	12.6	21.74
0819_1911	30.1	23.5	18.0	12.2	20.94
0819_2157	31.5	25.6	18.8	13.7	22.38
0819_2227	31.0	25.9	18.4	12.4	21.94

Table 70. Six checkpoints from one epoch of one run. FINE12 is the twelve-exit ladder, whose highest rate has no admissible allocation at this budget, so the mean is over the four rates that do. Take from it that a comparison at a matched epoch is not a comparison at a matched checkpoint, and that the difference between two rows here is as large as several of the differences this work reads as results.

The six results/signalled_FINE12_0819_*.json files listed in this module, all at epoch 2 of runs/FINE12 on 53 sequences at a 0.1 dB budget, all carrying a hook count. Five are ckpt_step and one is ckpt_eval; both are overwritten in place by the run, and the measurement files are what preserve them. A seventh epoch-2 measurement exists and is excluded because it carries no hook count.

What follows from the two tables is a constraint on how this work may be read. No configuration was trained twice at two seeds, and no seed is set in the decoder's trainer, so there is no interval on any architectural comparison in this document. Section A says so and gives the sources of variation; what these files add is a magnitude. Until a configuration is trained twice, a difference of a few points between two rows of the ladder table is not separable from the difference between two runs of

either of them.

H.6. Anchor drift

run	q0	q32	q63	frames
RECIPE512, pinned	0.0051	0.0145	0.0362	53
RECIPE512, later	0.0029	0.0150	0.0359	53
BEST	0.0002	0.0086	0.0179	53
BEST128	0.0036	0.0104	0.0216	53
FINE12	0.0075	0.0260	0.0642	53
VERBATIM, no anchor term	0.1101	0.1241	0.1594	40

Table 71. How far the deepest exit has moved from the released decoder, in decibels, one checkpoint per run. The deepest exit is supposed to be the released decoder and the training objective carries a term that holds it there. The term does real work: the run trained without it sits 0.110 to 0.159 dB away, more than the whole headline budget at every rate before a single tile has exited early. The term does not hold it exactly, and what it leaves behind grows with rate: 36% of a 0.1 dB budget at q63 on the pinned checkpoint and 64% on the twelve-exit run, whose floor exceeds that budget outright at the highest rate.

results/supp_anchor_PAPER.json on runs/RECIPE512/ckpt_PAPER.pth.tar;
results/anchor_RECIPE512.json, results/anchor_BEST.json,
results/anchor_BEST128.json, results/anchor_FINE12.json and
results/anchor_VERBATIM.json on the ckpt_eval or ckpt_step file of their runs, at the
epoch each happened to be at, which none of them records. Frame counts differ by file
and are printed, because two of these are 40 frames and are not comparable with the
53-frame rows at the fourth decimal.

Whether it grows with training is not settled by what is in results/. The two RECIPE512 rows are the pinned checkpoint and a later one on the same 53 frames, and at q63 they read 0.0362 and 0.0359 dB, which is no movement at all; at q0 the later checkpoint is the closer of the two, at 0.0029 against 0.0051. Neither file records the epoch of the checkpoint it read, so the pair does not bracket a known amount of training. The measured direction is with rate rather than with time, sevenfold from q0 to q63 on the pinned checkpoint. The open problem is the size, not the trend: at q63 a third of the budget is spent before the allocation begins, and any move to a later checkpoint has to remeasure this and report it as its own row rather than assume it carries over.

There is a structural fix and it was measured and rejected, which is row five of Table 68. Freezing the trunk makes the deepest exit the released decoder by construction, with no weight left that could move, and the drift measures exactly zero. The shallow exits then cannot improve either, because the exit heads alone cannot recover what six skipped blocks contained, and the oracle ceiling collapses to nothing above the lowest rate. Training the trunk is what makes the ladder worth having, and the drift is what training the trunk costs.

H.7. What is open, and one thing that closed

- **The reported checkpoint is early.** Table 69 says a later checkpoint of the same run saves more at every rate. Nothing in this document is measured on one, because 85 checked claims, every table and every figure rest on the pinned checkpoint, and moving it means remeasuring all of them. What a later checkpoint would change is the size of the headline and the anchor drift beneath it, in opposite directions.
- **No interval exists on any architectural comparison.** Two runs of one configuration are 4.7 points apart at a matched epoch, and six checkpoints inside one epoch of one run span 1.8. Both are the size of the differences the ladder table ranks. The remedy is repeated runs and it is a training cost, which the evaluation queue that produced this supplement was not permitted to spend.
- **The anchor is held by a loss term and not by construction.** The deepest exit is 0.036 dB from the released decoder at q63 on the pinned checkpoint, about a third of the headline budget, and the one construction that removes the drift removes the ladder's value with it.
- **Everything here is one decoder's intra path.** The ladder is built inside the intra decoder of one released codec and measured on intra

frames of video sequences at 1080p and below. No inter frame, no second codec rearranged into a ladder, and no resolution above 1080p, the last because the evaluation card has about 5 GB free and a 4K decode does not fit. The construction needs a residual trunk with a shared head and nothing else, which is an argument rather than a measurement.

- **The window is measured with the losses known.** The allocations in this section are the exact per-tile search, which bounds any decoder-side predictor. Section F measures how much of it a trained head reaches and how much a rule with no parameters reaches, and neither closes the gap at high rate.

One long-open check has closed, and the cause was not what it looked like. The partially signalled configuration is supposed to reduce to the fully signalled one when every tile is overridden, and for weeks the checker reported it missing by 0.09 to 0.21 points at matched delivered quality. Bisection noise is far too small to explain a gap that size, which was established and was correct, and the conclusion drawn from it, that one of the two code paths must be charging differently, was wrong. Both paths were right. The checker was comparing the hybrid file's endpoint against a different measurement of the fully signalled configuration from the one the hybrid run had been made against, and once both sides read the same file the endpoint reproduced to four decimals. The checker now goes through the same saving convention the generated tables use, and compares against modelled endpoints where the hybrid file carries no hook count, so the cost model's 0.8-point offset cannot present itself as a failure of the interpolation. As recorded in results/check_paper.json, 85 of 85 claims now pass.

results/check_paper.json. Section A.16 reports the same closure from the checker's side; what is added here is that the defect was in the comparison rather than in either measurement, which is the failure mode this project has met in several costumes and is the reason every table in this supplement names its file.